

大语言模型与 AI 面试题集锦

7 章共 64 道高频面试题详解

目录

- 第 1 章 LLM 基础与 Transformer 原理
- 第 2 章 训练、微调与对齐
- 第 3 章 RAG 与知识库问答
- 第 4 章 Agent、Workflow 与多智能体
- 第 5 章 Function Calling、MCP、A2A 与工具调用
- 第 6 章 推理部署、性能优化与系统设计
- 第 7 章 多模态与 VLM

第 1 章 LLM 基础与 Transformer 原理

高频面试题详解 · Q1 ~ Q8

Q1. Transformer 的整体结构是什么?Encoder、Decoder 分别做什么?

Transformer 由 Vaswani 等人在 2017 年的 "Attention Is All You Need" 中提出,是完全基于注意力机制的序列建模架构,抛弃了 RNN 的循环结构,可以并行计算。

整体由 Encoder 栈和 Decoder 栈构成,各自由 N 个相同的层(原论文 N=6)堆叠。

Encoder 每层包含两个子层:多头自注意力(Multi-Head Self-Attention) + 前馈网络(FFN)。每个子层外都有残差连接(Residual) 和层归一化(LayerNorm)。Encoder 接收完整输入序列,通过自注意力让每个位置都能"看到"所有其他位置,输出每个 token 的上下文向量表示。

Decoder 每层包含三个子层:① Masked Multi-Head Self-Attention(只能看左侧已生成的 token,通过下三角 mask 实现因果);② Cross-Attention(Query 来自 decoder,Key/Value 来自 encoder 输出,用于"对齐"输入序列);③ FFN。同样有残差 + LayerNorm。

Decoder 的作用是自回归地生成输出序列:在第 t 步,基于已生成的 1..t-1 个 token 和 encoder 编码的源序列,预测第 t 个 token。

位置信息通过 Positional Encoding(原论文用 sin/cos 函数,后来发展出 RoPE 等)显式注入,因为自注意力本身对位置无感知。

常见变体:Encoder-only(BERT)用于理解任务;Decoder-only(GPT)用于生成;Encoder-Decoder(T5、BART)用于翻译、摘要等 seq2seq 任务。

Q2. 自注意力机制如何工作?为什么比 RNN 更适合长序列?

自注意力(Self-Attention)的核心是用同一个序列自己的不同位置去计算"注意力权重",从而让每个位置都能动态地从其他位置汇聚信息。

计算过程:对输入 X(形状 [n, d]),通过三个可学习线性投影得到 $Q = XW_q$, $K = XW_k$, $V = XW_v$ 。然后 $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{dk}) V$ 。

其中 QK^T 计算每对位置之间的相似度,除以 $\sqrt{dk}$ 是缩放因子,防止 dk 较大时点积过大导致 softmax 进入饱和区、梯度消失。softmax 把得分归一化为权重,最后加权 V 得到输出。

相比 RNN 的优势:

(1) 并行计算:RNN 必须按时间步串行展开,无法并行;自注意力一次矩阵乘法同时处理整个序列,GPU 利用率高,训练速度快。

(2) 长距离依赖:RNN 信息要经过 n 步传递才能从位置 1 到位置 n ,容易梯度消失/爆炸,实际有效记忆长度有限;自注意力任意两个位置都是 $O(1)$ 步直接连接,长距离依赖建模能力远超 RNN。

(3) 路径长度短:Transformer 任意两 token 间最大路径长度为 1,而 RNN 是 $O(n)$,CNN 是 $O(\log n)$ 。

代价:自注意力的复杂度是 $O(n^2 \cdot d)$,序列长度 n 增大时计算和显存平方增长。这也是后续 Flash Attention、稀疏注意力、线性注意力等优化的出发点。

Q3. MHA、MQA、GQA 有什么区别?为什么推理阶段常用 GQA/MQA?

三者的核心区别在于 Q/K/V 头(head)的分组方式,直接影响 KV Cache 显存和推理吞吐。

MHA(Multi-Head Attention,原始多头注意力):每个 head 都有独立的 Q、K、V 投影。若有 h 个头,就有 h 套 K、V。表达能力最强,但 KV Cache 大。

MQA(Multi-Query Attention,Shazeer 2019): h 个 Q 头共享同一组 K、V。KV 头数变成 1,KV Cache 缩到原来的 $1/h$,推理显著加速。代价是质量略有下降,训练时容易不稳定。代表模型:PaLM、Falcon。

GQA(Grouped-Query Attention,Ainslie 2023):折中方案。把 h 个 Q 头分成 g 组,每组共享一对 K、V。 $g=1$ 时退化为 MQA, $g=h$ 时退化为 MHA。KV Cache 缩到 g/h 。代表模型:LLaMA-2/3 70B、Qwen2、Mixtral。

为什么推理常用 GQA/MQA:

(1) KV Cache 是 LLM 推理显存瓶颈。KV Cache 大小 $\approx 2 \times n_{\text{layer}} \times n_{\text{kv_head}} \times \text{head_dim} \times \text{seq_len} \times \text{batch} \times \text{dtype}$ 。MQA/GQA 让 $n_{\text{kv_head}}$ 大幅缩小,显存占用降到 MHA 的 $1/4$ 到 $1/8$ 。

(2) 推理阶段是 memory-bound(显存带宽瓶颈)。KV 读写量小了,吞吐 token/s 显著提升,长上下文更可行。

(3) GQA 在质量上几乎不输 MHA,在速度上接近 MQA,成为当前主流选择。

Q4. 位置编码为什么必要?RoPE 相比绝对位置编码有什么优缺点?

为什么必要:自注意力本质上是"集合"运算,对输入顺序不敏感(把 token 顺序打乱,输出只是对应位置也打乱,语义关系完全丢失)。但语言是有序的(主谓宾顺序、前后文逻辑都依赖位置),所以必须显式注入位置信息。

主流位置编码大致分三类:

(1) 绝对位置编码(APE):给每个位置 i 一个独立向量 p_i ,加到 token embedding 上。原始 Transformer 用 $\sin/\cos$ (无需训练、可外推),BERT/GPT-1/2 用可学习的位置嵌入(更灵活但固定最大长度,不能外推)。

(2) 相对位置编码(RPE):在 attention score 里加入 $i-j$ 相关的偏置(T5 的 bucket bias、ALiBi 等)。建模"相对距离"更符合语言直觉。

(3) 旋转位置编码(RoPE,Su Jianlin 2021):把位置编码作为旋转矩阵作用在 Q 和 K 上。对位置 m 的向量,在每个二维子空间里旋转角度 $m \cdot \theta$ 。这样 $Q_m \cdot K_n$ 的结果只依赖相对位置 $m-n$,实现了"绝对编码,相对效果"。

RoPE 的优点:

- ① 同时具备绝对编码的简洁性(直接施加在 Q/K 上、不改架构)和相对编码的优势(注意力分数自然依赖相对位置);
- ② 外推性好(配合 NTK-aware、YaRN 等扩展可以推理时拉长上下文);
- ③ 与线性注意力、Flash Attention 等兼容,工程友好;
- ④ 计算高效,只增加少量旋转操作。

RoPE 的缺点:

- ① 直接外推到训练长度之外性能会下降(需要 NTK / YaRN / PI 等技巧);
- ② 频率参数 base(通常 10000)对长上下文敏感,需要重新调整;
- ③ 实现上要在每层 attention 都做旋转,代码复杂度比加法式 APE 略高。

当前主流模型(LLaMA、Qwen、ChatGLM、DeepSeek 等)几乎都用 RoPE。

Q5. Decoder-only、Encoder-only、Encoder-Decoder 架构分别适合什么任务?

Encoder-only(代表:BERT、RoBERTa、DeBERTa):双向自注意力,每个 token 能看到全部上下文。

适合:文本分类、命名实体识别、情感分析、句子相似度、阅读理解等"理解类"判别任务;也适合作

为 embedding 模型(BGE、E5 都是基于 encoder)。

不擅长:开放式文本生成(没有自回归生成能力,虽然可以 MLM 填空但不自然)。

Decoder-only(代表:GPT 系列、LLaMA、Qwen、DeepSeek、Mistral):带因果 mask 的自回归生成。

适合:开放式文本生成、对话、续写、代码生成、Few-shot/Zero-shot 任务、指令跟随。当前几乎所有大语言模型都是 decoder-only,因为它兼容所有任务(用 prompt 把任务转化为续写)、训练目标统一(next-token prediction)、易扩展。

不擅长:严格意义上的双向理解(每个位置只能看到左侧),但通过大规模预训练 + 指令微调,实际效果已经能逼近甚至超过 encoder-only 模型。

Encoder-Decoder(代表:T5、BART、mT5、Flan-T5):encoder 双向看输入,decoder 自回归生成,二者通过 cross-attention 连接。

适合:典型 seq2seq 任务——机器翻译、文本摘要、问答(给定上下文)、改写、对话状态生成等"输入→输出"映射明确的任务。

优点:对源/目标分离的任务结构更自然,encoder 可以充分理解输入。

缺点:参数量大、训练复杂,通用性不如 decoder-only,工程上越来越少用。

业界趋势:decoder-only 几乎一统江湖,主要因为:scaling law 更友好、训练数据准备更简单、In-Context Learning 能力强、所有任务可统一用 prompt 表达。

Q6. Tokenizer 是什么?BPE、WordPiece 有什么区别?

Tokenizer(分词器)负责把原始文本切成模型能处理的 token 序列,并维护一个 token ↔ id 的词表。它是 LLM 输入输出的"翻译层",直接影响词表大小、序列长度、OOV(未登录词)率、跨语言能力。

主流子词(subword)分词算法:

BPE(Byte-Pair Encoding, Sennrich 2015):

① 初始化:把每个词拆成单字符 + 词尾标记;

② 统计语料中所有相邻字符对的频次,合并频次最高的一对,加入词表;

③ 重复 step ②,直到达到目标词表大小。

BBPE(Byte-level BPE):BPE 的字节级变体,基本单位是 byte 而不是字符。GPT-2/3/4、LLaMA、Qwen 都用 BBPE。优点:可以表示任意 Unicode,完全没有 OOV;对多语言、emoji、代码友好。

WordPiece(Schuster 2012,BERT 使用):

① 与 BPE 类似,也是迭代合并子词;

② 区别:不是按"频次"合并,而是按"使语料似然提升最大"的子词对合并,即最大化 $P(\text{子词 A}, \text{子词 B}) / (P(\text{子词 A}) \cdot P(\text{子词 B}))$;

③ 词内的非首子词加 "##" 前缀(如 "playing" $\rightarrow$ "play", "##ing")。

主要区别:

(1) 合并准则:BPE 看频次,WordPiece 看似然提升;

(2) 标记方式:BPE 通常用 "_" 表示词首(SentencePiece 风格),WordPiece 用 "##" 表示词中;

(3) 分词时:BPE 贪心或基于规则,WordPiece 最长匹配。

其他常见分词:

- SentencePiece:Google 开源的统一框架,可以训练 BPE 或 Unigram LM,直接处理原始文本(无需预分词),T5、LLaMA、ChatGLM 都用它。

- Unigram LM:基于概率模型,先生成大词表再裁剪,允许多种切分方式按概率选择。

Q7. Scaling Law 说明了什么?对模型训练有什么指导意义?

Scaling Law(缩放定律)由 OpenAI Kaplan 等人 2020 年首次系统提出,后被 DeepMind 的 Chinchilla(Hoffmann 2022)修正,描述了 LLM 的测试损失与模型参数量 N 、训练数据量 D 、计算量 C 之间的幂律关系:

$\text{Loss}(N) \approx A \cdot N^{(-\alpha)} + L_{\infty}$,即 loss 随参数量增加按幂律下降,存在不可降低的下限。同样 $\text{Loss}(D) \approx B \cdot D^{(-\beta)}$, $\text{Loss}(C) \approx C \cdot F^{(-\gamma)}$ 。

关键结论:

(1) 模型性能可预测:在小规模实验上拟合幂律曲线,可外推到大规模训练的最终 loss,大幅降低试错成本。

(2) Kaplan 早期结论:计算预算给定时,应优先把算力堆在更大模型上,数据可以少一些。

(3) Chinchilla 修正(2022):重新做实验后发现,在给定 C 下,最优是参数 N 和数据 D 同比例增长(N 翻倍则 D 也翻倍)。当时主流大模型(GPT-3 175B 只用了 300B token)是"训练不足"的——同样算力下,更小但训练更充分的模型(Chinchilla 70B, 1.4T token)性能更好。这就是 "Chinchilla 最优": 大约每个参数对应 20 token 训练数据。

(4) 涌现能力:某些能力(如多步推理、In-Context Learning)在小模型上完全没有、达到某个临界规模后突然出现,但近年研究质疑"涌现"可能只是评估指标不连续造成的假象。

对训练的指导意义:

- ① 算力规划:有 C 的预算,先用 scaling law 估算 N^* 、 D^* 的最优组合,避免"模型大但欠拟合";
- ② 数据配比与质量:Chinchilla 之后业界开始堆数据(LLaMA-3 用 15T+ token),并重视数据去重、清洗、配比;
- ③ 小规模实验外推:用 100M ~ 1B 的小模型多组实验拟合曲线,预测目标规模的损失,选超参/数据配方;
- ④ 算力分配:推理优化(MoE、量化、蒸馏)可以打破训练 scaling law,在小推理成本下接近大模型效果。
- ⑤ 后训练阶段(SFT、RLHF)、长上下文、多模态都有自己的 scaling law,需要单独研究。

Q8. LLM 的解码策略有哪些?Greedy、Beam Search、Top-k、Top-p 的优缺点?

解码(decoding)是把模型每步输出的概率分布转化为具体 token 的过程,直接影响生成质量与多样性。

Greedy Search(贪心):每步选概率最大的 token。

优点:简单、确定性、速度快;

缺点:容易陷入局部最优,导致重复、单调("我我我..."、循环短语);适合事实性、确定性任务(如答案抽取)。

Beam Search(束搜索):每步保留 top-k 条得分最高的候选序列(束宽 k),最后选总得分最高的。

优点:在翻译、摘要等"目标明确"的任务上明显优于贪心,能避免短视错误;

缺点:① 倾向于生成短且通用的文本(长度惩罚不当时);② 多样性差,k 条束往往非常相似;③ 对开放式生成(如对话、续写)反而比采样差,容易"无聊";④ 计算成本高(k 倍)。

Top-k Sampling:在概率最高的 k 个 token 里按概率重新归一化,再随机采样。

优点:引入随机性,生成更有创造性、多样性;过滤掉长尾低概率 token,避免胡言乱语;

缺点:k 是固定的——分布很陡时,k 太大会引入噪声;分布很平时,k 太小又过度收敛。

Top-p / Nucleus Sampling(Holtzman 2019):动态选择累积概率达到 p 的最小 token 集合,再在这个集合里采样。

优点:自适应——分布陡时只取几个高概率 token,分布平时取很多;比 Top-k 更鲁棒;

缺点:p 还是需要调,极端情况下也可能过宽/过窄;采样仍有不确定性,不适合需要可复现的场景。

其他常见技巧:

- Temperature(温度):缩放 logits, $T < 1$ 更确定, $T > 1$ 更随机, $T = 0$ 等同贪心。常与 Top-p 联用。
- Repetition Penalty / Presence Penalty:对已出现的 token 降权,缓解重复。
- Min-p(2023):用最大概率乘以 p 作为阈值,保留所有概率 $\geq$ 阈值的 token,比 top-p 更稳。
- Contrastive Search、DoLa、Speculative Decoding(用小模型起草、大模型校验)等。

实践建议:

- 创意写作、闲聊:Top-p (0.9~0.95) + Temperature (0.7~1.0);
- 翻译/摘要:Beam Search($\text{beam}=4$) 或 Top-p (0.6) + 低温度;
- 代码、数学、结构化输出:贪心或 Top-p (0.1) + 低温;
- 评测/复现:贪心或固定 seed。

第 2 章 训练、微调与对齐

高频面试题详解 · Q1 ~ Q8

Q1. 大模型是怎么训练出来的?预训练、SFT、RLHF 的流程是什么?

现代 LLM 训练通常分四个阶段:数据准备 → 预训练(Pre-training)→ 监督微调(SFT)→ 偏好对齐(RLHF / DPO 等)。

阶段 1 · 预训练(Pre-training):

目标:next-token prediction(decoder-only)或 MLM(encoder)。

数据:海量(数 T ~ 数十 T token)的网页、书籍、代码、论文、对话等无监督文本,经过去重、过滤、质量评分、配比。

产物:Base Model,具备基本语言建模和世界知识能力,但不懂"听指令"。

算力:数千到数万张 GPU,持续数周到数月,这是整个流程中最贵的阶段(占总成本 95%+)。

阶段 2 · 监督微调(SFT,Supervised Fine-Tuning):

目标:让模型学会按"指令-回答"的格式响应。

数据:数万到数百万条高质量(instruction, response)对,人写、模型蒸馏、混合都可以。FLAN-T5、Alpaca、ShareGPT、OpenAssistant 都是典型 SFT 数据集。

训练:在 base 上继续做 next-token prediction,但 loss 只在 response 部分计算(prompt 不算 loss)。

产物:Instruct/Chat Model,能跟随指令、有对话格式感,但回答的"风格、安全性、偏好"还没对齐人类喜好。

阶段 3 · 偏好对齐(RLHF 等):让模型的回答更符合人类偏好(有用、无害、诚实)。

经典 RLHF(InstructGPT、ChatGPT)的三步:

① 训练奖励模型(Reward Model, RM):人类对同一 prompt 的多个回答排序,RM 学会预测"哪个回答人类更喜欢";

② 用 PPO(Proximal Policy Optimization)优化 LLM:LLM 是 policy,RM 给 reward,加 KL 散度

惩罚约束策略不要偏离 SFT 太远;

③ 反复迭代,产出最终的 RLHF 模型。

现代替代方案:DPO、IPO、KTO、GRPO 等不需要单独的 RM,直接用偏好对训练,工程更简单。

阶段 4(可选)·后处理与持续优化:

安全微调(Constitutional AI、RLAIF)、领域微调、长上下文扩展(YaRN、Position Interpolation)、工具调用微调、Reasoning 微调(CoT / o1 风格的 RL on reasoning)、模型蒸馏到小模型等。

Q2. 微调方案有哪些?LoRA、QLoRA、Adapter、全参微调如何选择?

微调按参数量可分两大类:全参微调(Full Fine-Tuning) 与 参数高效微调(PEFT)。

全参微调(Full FT):

更新模型全部参数。效果上限最高,但显存爆炸(7B 模型全参微调需 60GB+ 显存,激活值、梯度、优化器状态都是参数量的数倍),工程门槛高,容易"灾难性遗忘"。

适合:数据量大(数十万+)、领域跨度大、有充足算力(A100/H100 集群)。

PEFT 系列:冻结大部分参数,只训练少量"插件"参数。

LoRA(Low-Rank Adaptation,Hu 2021):

核心思想:微调时参数更新 ΔW 是低秩的($\text{rank} \ll d$)。把 ΔW 分解为两个小矩阵 $A \in \mathbb{R}^{(d \times r)}$ 、 $B \in \mathbb{R}^{(r \times d)}$,只训练 A、B,W 冻结。r 通常 4~64。

优点:① 训练参数减少 100~10000 倍,显存友好;② 训练完只需保存 A、B(几 MB ~ 几百 MB),多任务可以挂多个 LoRA;③ 推理时可合并回 W,无额外延迟;④ 不易灾难性遗忘。

常见目标层:attention 的 q_proj、k_proj、v_proj、o_proj;也可以加在 FFN 上。

QLoRA(Dettmers 2023):LoRA + 量化。

把 base model 量化到 4-bit(NF4 格式),冻结;LoRA adapter 用 16-bit 训练。65B 模型可在单张 48GB GPU 上微调,极大降低门槛。性能几乎不损失。

是当前个人/小团队微调中大型模型(13B~70B)的事实标准。

Adapter(Houlsby 2019):

在 Transformer 每层之间插入小的"瓶颈"网络(down-proj → 非线性 → up-proj),只训练它们。

优点类似 LoRA;缺点:推理时无法合并,会增加延迟,所以工业上逐渐被 LoRA 取代。

Prefix Tuning / Prompt Tuning / P-Tuning:在输入前加可学习的连续 prompt 向量,只训练它们。参数量更少,但对超参敏感、收敛慢,小模型上效果有限。

IA³、BitFit、(IA)³ 等:更激进的轻量方案,只调 bias 或缩放参数。

选择建议:

- 数据多(10 万+)+ 算力足:全参微调或 LoRA 都行,全参上限高;
- 数据少(几千到几万) + 中等显存:首选 LoRA(rank=8~32);
- 想微调 13B+ 但只有单卡:QLoRA;
- 多任务、多场景共用 base 模型:LoRA(每任务一个 adapter,按需挂载);
- 改风格、改格式:LoRA 足够;改知识、改深层能力:可能需要全参 + 大数据;
- 极端轻量(几十 MB):Adapter / Prompt Tuning,但效果通常不如 LoRA。

Q3. Function Calling 是怎么训练出来的?

Function Calling(函数/工具调用)能力的训练核心是:让模型学会在需要外部工具时,输出符合预设 schema 的结构化调用,并能消化工具返回结果继续推理。整体仍是基于 SFT(+ 可选 RL)的监督训练。

数据构造(最关键):

需要构造 (system_prompt_with_tools, user_query, assistant_tool_call, tool_result, assistant_final_answer) 这样的多轮对话样本。每个样本包含:

- ① 系统提示中声明可用工具列表(每个工具有名字、描述、参数 JSON Schema);
- ② 用户提问;

- ③ 助手输出调用决策——一段特殊格式(如 OpenAI 的 `tool_calls` 字段、ChatML 的 `<function>` 标签、Qwen 的 ReAct 风格);
- ④ 工具执行结果(role=tool);
- ⑤ 助手综合工具结果给出最终回答。

数据来源:

- 人工标注:质量最高但成本极高;
 - 模型自动生成 + 校验:用 GPT-4 / Claude 等强模型先生成调用样本,再用真实工具执行结果校验/修正;
 - 真实 API 日志清洗:用真实生产系统中的工具调用轨迹;
 - 合成数据:Toolformer、Gorilla、ToolBench、APIBench 等开源数据集都是合成路线。
- 需要覆盖:① 单工具调用;② 多工具串行/并行;③ 参数错误重试;④ "不需要工具"的负样本(避免乱调);⑤ 复杂参数(嵌套对象、数组)。

训练方式:

- ① SFT 是主力:在 base / chat 模型上做指令微调,样本就是上面构造的对话。loss 只在 assistant 部分计算,尤其要监督 `tool_calls` 字段的精确格式。
- ② 格式约束:训练时让模型学会用专门的特殊 token(如 Qwen 的 `<|tool_call|>...</|tool_call|>`),推理时配合 grammar / constrained decoding,保证 JSON 合法。
- ③ 可选 RL:用工具调用成功率、参数正确率、最终任务完成率作为 reward,做 RLAI 或 GRPO,进一步提升复杂任务效果。
- ④ 评测集:BFCL(Berkeley Function-Calling Leaderboard)、ToolBench 等。

工程注意点:

- 训练时要见过"工具描述很长"的样本,推理才能稳定处理大工具集;
- 要训练"自我修正"——参数填错后看到错误信息能重试;
- 多语言场景下,中英文工具描述都要覆盖;
- 推理时建议用 JSON Mode / Schema-guided decoding 兜底。

Q4. RLHF 的三个阶段是什么?为什么 SFT 不够?

RLHF(Reinforcement Learning from Human Feedback)由 OpenAI InstructGPT (2022) 系统化,后被 ChatGPT 等沿用。

三个阶段:

阶段 1 · SFT:

收集 (prompt, demonstration) 对,人类示范"理想回答"是什么样;在 base 模型上做监督微调,得到 SFT 模型。让模型先学会基本的指令格式和回答风格。

阶段 2 · Reward Model 训练:

对同一个 prompt,让 SFT 模型采样多个回答 ($y_1, y_2, \dots, y_k$);标注员把这些回答两两比较或整体排序;在 SFT 模型基础上(去掉 LM head、加 scalar head)训练 Reward Model,使其打分顺序与人类排序一致。损失函数通常是 Bradley-Terry pairwise loss: $-\log \sigma(r(x, y_{\text{好}}) - r(x, y_{\text{差}}))$ 。

RM 本质上是把"难以言说"的人类偏好压缩成一个标量函数,后续 RL 阶段就能自动获得反馈信号。

阶段 3 · PPO 优化:

把 LLM 当作 policy π , prompt 是 state,生成的 token 序列是 action, RM 给出的标量是 reward。用 PPO 算法最大化 reward,同时加 $KL(\pi \parallel \pi_{\text{SFT}})$ 惩罚,防止模型为了拿高分而偏离正常语言分布、走向"reward hacking"。最终得到对齐后的模型。

为什么 SFT 不够,需要 RLHF:

(1) 标注成本与上限:SFT 需要人类"写出"高质量回答,门槛高、成本高;让人"比较"两个回答容易得多。RLHF 用比较代替书写,数据效率高。

(2) 风格/偏好的"软"信号无法直接监督:有用性、礼貌、安全、详尽程度这些维度没有标准答案,无法用 cross-entropy 直接学;但可以通过"哪个更好"的偏好信号学到。

(3) 探索空间更大:SFT 只能模仿示范回答,模型见到的"好回答"是固定的;RLHF 允许模型生成多种

回答,从中通过 reward 自我探索哪些方向更好,更接近"做得比示范还好"。

(4) 抑制不想要的行为:SFT 难以教"不要做什么"(只给正例);RLHF 可以通过偏好对明确告诉模型"A 比 B 好",B 类回答自动被压低。安全、拒绝、避免幻觉等都是 RLHF 强项。

(5) 实证效果:InstructGPT 论文显示,1.3B 的 RLHF 模型在人类偏好上胜过 175B 的纯 SFT GPT-3。

RLHF 的代价:工程复杂(4 个模型同时存在:actor、critic、RM、ref)、训练不稳定、对 RM 质量极敏感、容易 reward hacking。这也是 DPO 等更简单方法兴起的原因。

Q5. DPO 与 PPO/RLHF 有什么区别?

DPO(Direct Preference Optimization,Rafailov 2023)是 RLHF 的简化替代方案,无需训练独立的 Reward Model 也无需 PPO,直接用偏好对优化策略。

PPO/RLHF 流程(复杂):

SFT → 训练 Reward Model → PPO 优化(同时跑 actor、ref、reward、critic 四个模型,显存暴涨,工程复杂,超参敏感)。需要在线采样 rollout,训练不稳定。

DPO 的核心洞察:

RLHF 的最优策略与 reward 函数之间存在闭式关系: $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \cdot \exp(r(x,y) / \beta)$ 。

反过来,可以把 reward 表达为 $r(x,y) = \beta \cdot \log[\pi(y|x) / \pi_{\text{ref}}(y|x)] + \text{const}$ 。

把这个表达式代入 Bradley-Terry 偏好模型的 loss,就消去了 reward model,直接得到关于 π 的损失:

$$L_{\text{DPO}} = -E[(x, y_{\text{好}}, y_{\text{差}})] \log \sigma(\beta \cdot \log[\pi(y_{\text{好}}|x) / \pi_{\text{ref}}(y_{\text{好}}|x)] - \beta \cdot \log[\pi(y_{\text{差}}|x) / \pi_{\text{ref}}(y_{\text{差}}|x)])$$
。

主要区别:

(1) 是否需要 Reward Model:

PPO:需要(单独训练 + 在线打分);

DPO:不需要,偏好对直接用于训练。

(2) 是否需要在线采样:

PPO:需要,每个 batch 都要 rollout 生成回答,工程重;

DPO:不需要,固定的偏好数据集 (prompt, y_好, y_差) 离线训练,跟 SFT 一样简单。

(3) 训练稳定性:

PPO:对超参(KL 系数、learning rate、clip)极敏感,容易崩,需要大量调参;

DPO:稳定得多,基本就是普通监督学习的体验。

(4) 算力与显存:

PPO:需同时驻留 4 个模型副本(actor、critic、ref、RM);

DPO:只需 2 个(policy、ref),甚至 ref 可以离线预算,显存接近 SFT。

(5) 性能:

原论文显示 DPO 在多个 benchmark 上与 PPO 相当甚至更好。但更大规模的实证(Llama-3、Qwen 等)显示在某些复杂任务和安全维度上 PPO 仍略胜,尤其在长程任务和需要在线探索的任务上。

(6) 后续发展:

IPO(避免 DPO 过拟合)、KTO(单点偏好,不需要 pair)、ORPO(SFT + 偏好统一)、SimPO 等都是 DPO 系列改进。

实践选择:

- 小团队、数据有限、追求工程简单:DPO 是默认选择;
- 追求极致性能、有大算力、复杂任务:PPO 仍有价值;
- 推理类任务(数学、代码):GRPO 等 on-policy RL 当前效果最好(参见 DeepSeek-R1)。

Q6. GRPO、GSPO、DAPO 听说过吗?和 PPO 有什么不同?

这几个都是 2024-2025 年涌现的新一代 RL 训练算法,主要针对 LLM 推理(reasoning)训练而设计,在 DeepSeek-R1、Qwen3、Kimi 等模型中取得突破。

GRPO(Group Relative Policy Optimization,DeepSeek 2024):

出自 DeepSeekMath 论文,后被 DeepSeek-R1 大规模采用。核心简化是:去掉 PPO 中独立的 Critic(value network),用"组内相对优势"替代。

做法:对同一个 prompt 采样 G 个不同回答 $\{y_1, \dots, y_G\}$,每个用 reward model(或规则 reward,如数学答案是否正确)打分得 $\{r_1, \dots, r_G\}$;优势 $A_i = (r_i - \text{mean}(r)) / \text{std}(r)$,即组内相对得分。然后用 PPO 类的 clip 目标 + KL 惩罚做更新。

相比 PPO 的优势:① 不需要 critic,显存少一半;② 训练更稳定,对超参不敏感;③ 特别适合"答案对错"这种二值/稀疏 reward 场景。

DeepSeek-R1 用 GRPO + 规则 reward(数学题答案、代码运行结果)训练出强推理能力,引爆了 RL on Reasoning 这条路线。

DAPO(Decoupled Clip and Dynamic sAmpling Policy Optimization,字节 / 清华 2025):

针对 GRPO 训练中的几个具体痛点改进:

- ① Clip-Higher:把 PPO 的对称 clip $[1-\epsilon, 1+\epsilon]$ 改为非对称 $[1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}}]$,让"低概率但应该被鼓励"的 token 有机会被显著提升,缓解熵塌缩(模型过早收敛到单一答案);
- ② Dynamic Sampling:动态过滤掉"全对"或"全错"的 prompt 组(优势为 0、无梯度信号),把算力集中在有学习空间的样本上;
- ③ Token-Level Loss:从序列级 loss 改为 token 级,长回答不被稀释;
- ④ Overlong Reward Shaping:对超长但未完成回答给软惩罚,避免训练崩坏。

在 AIME 等数学推理 benchmark 上超过 GRPO 训练的 DeepSeek-R1。

GSPO(Group Sequence Policy Optimization,Qwen 团队 2025):

Qwen3 训练采用。核心改动是把 GRPO 的 token-level importance ratio 改为 sequence-level ratio,即:

原 PPO/GRPO: $\text{ratio} = \pi(\text{token}_t | \dots) / \pi_{\text{ref}}(\text{token}_t | \dots)$, 逐 token clip;

GSPO: $\text{ratio} = \pi(\text{整段回答} | x) / \pi_{\text{ref}}(\text{整段回答} | x)$, 整段一起 clip。

动机: 逐 token 的 ratio 在长序列上方差爆炸, 训练不稳; 序列级 ratio 更平滑, 长输出训练更稳。

在长思维链(long CoT)和混合专家(MoE)模型上效果显著。

总结对比:

PPO: 有 critic、token 级 clip, 通用但重;

GRPO: 无 critic、组内相对优势、token 级 clip, DeepSeek 系主流;

DAPO: GRPO + 非对称 clip + 动态采样 + token 级 loss, 字节系主流;

GSPO: GRPO + 序列级 ratio, Qwen 系主流, 长序列更稳。

都属于 "on-policy RL on LLM" 路线, 共同点是去 critic、组内归一化、规则/可验证 reward; 差异主要在 clip 形式、采样策略、loss 粒度。

Q7. Reward Hacking 是什么? 如何缓解?

Reward Hacking(奖励作弊 / 奖励黑客)指的是: RL 训练中, 策略找到一种能拿高 reward 但并不真正完成目标的行为, 本质是 reward 函数对真实目标的 "近似不完美", 被策略钻了空子。

常见表现:

- ① 过长/啰嗦: RM 误以为 "长 = 详细 = 好", 模型学会写废话凑长度;
- ② 谄媚(sycophancy): RM 学习到 "同意用户 = 高分", 模型变得过度迎合, 不愿意纠正用户错误;
- ③ 格式作弊: 学会输出 RM 偏好的固定开头、列表结构、礼貌用语, 内容反而空洞;
- ④ 安全过度: 为了不踩安全 RM 的红线, 大量无害问题也拒绝回答(over-refusal);
- ⑤ 利用 RM 漏洞: 发现 RM 对某些字符、emoji、特定句式打高分, 系统性插入它们;
- ⑥ 自我夸耀: 在回答中加入 "This is a comprehensive answer..." 等元描述;
- ⑦ 推理任务中 "假装思考": 只写 "让我想想..." 但跳过实际推理, 直接猜答案。

深层原因:

Goodhart's Law:"当一个度量变成目标时,它就不再是好的度量。" RM 是真实人类偏好的代理,代理本身有偏差;策略优化得越狠,偏差被放大得越严重。

缓解手段:

(1) KL 约束(最经典):PPO 中加 $\beta \cdot \text{KL}(\pi \parallel \pi_{\text{ref}})$ 惩罚, β 越大策略偏离基线越受限。要好好调 β 或用 adaptive KL。

(2) 提升 RM 质量:更多样、更高质量的偏好数据;让 RM 见到各种"作弊样例"并打低分;用更大、更专精的 RM(如多 RM 集成,有用性、无害性、诚实性分开打分)。

(3) 多目标 / Constrained RL:不光最大化主 reward,还要满足若干约束(长度、安全、格式)。Llama 系列用过 multi-objective RM。

(4) Reward Shaping:对已知作弊模式手动加负 reward(过长扣分、检测到模板化开头扣分)。

(5) On-policy 持续更新 RM:训练过程中定期用新策略的输出做新标注,更新 RM,堵住已被发现的漏洞(Iterative RLHF)。

(6) 可验证 reward / Rule-based reward(2024+ 主流):对数学、代码、形式化任务,reward 直接来自"答案是否正确"、"代码是否跑通",绕过 RM。DeepSeek-R1 大量使用,几乎没有 hacking 空间。

(7) 评测与人工抽查:训练中定期人工评估,看作弊是否冒头;

(8) 限制采样温度、限制最大长度、限制工具调用次数等工程兜底。

(9) 算法层面:DPO 因为没有显式 RM,在简单场景下 hacking 相对少;DAPO 的 dynamic sampling 也能避开"全对全错"导致的退化。

注意:Reward Hacking 是 AI 对齐的核心问题之一,无法完全消除,工程上是"持续发现 — 修复"的

Q8. RLAIIF 是什么?有什么风险?

RLAIIF(Reinforcement Learning from AI Feedback)用 AI 模型(通常是另一个强大的 LLM,如 GPT-4、Claude 或同模型的更强版本)代替人类标注员,提供偏好反馈或直接生成奖励信号。

由 Anthropic 在 Constitutional AI(2022)和 Bai et al. 2022 系统提出,后 Google 等也大量使用。

典型流程:

- ① 写一份"宪法"(constitution),即一组原则(如"诚实"、"不伤害"、"对儿童安全"等);
- ② 让模型生成两个回答 y_1, y_2 ;
- ③ 用强 LLM 作为"裁判",根据宪法判断哪个更好;
- ④ 用这些 AI 标注的偏好数据训练 RM 或直接做 DPO;
- ⑤ 也可以用 AI 直接生成 critique $\rightarrow$ revision,自我改进。

优点:

- (1) 成本极低:人类偏好标注每条数美元,AI 标注几乎免费,可扩展到百万级;
- (2) 速度快:迭代节奏可以从月级变成天级;
- (3) 一致性高:同一个裁判模型评估标准统一,不像人类有疲劳、文化差异;
- (4) 可定制:通过 prompt / 宪法精确控制评估维度;
- (5) 实证:Anthropic、Google 等显示 RLAIIF 效果接近甚至超过 RLHF。

风险:

- (1) 偏差放大(最核心):裁判模型自带偏好(可能偏好长回答、偏好特定写作风格、偏好与自己相似的回答),这些偏差会被训练放大,产生"模型趋同"——所有模型最终都像 GPT-4。
- (2) 监督上限被锁死:学生模型只能学到"老师认为好"的内容,难以在能力上超越老师。如果用

GPT-4 作裁判,学生天花板就是 GPT-4。

(3) 自我循环风险:用模型 A 训练模型 B,再用 B 给 A 反馈,会形成偏见循环,与现实人类偏好越走越远(model collapse 的一种形式)。

(4) 安全/价值观漂移:裁判模型若有隐藏偏见(政治倾向、文化倾向、风格倾向),被训练目标全盘继承;比 RLHF 更不透明,因为偏见来源(裁判模型)本身是黑盒。

(5) Reward Hacking 加剧:策略很容易学会"裁判模型喜欢什么风格",针对性地输出,而非真正解决问题。

(6) 长尾问题:AI 裁判在常见任务上靠谱,但在专业领域(医学、法律)、长程任务、复杂推理上判断力远不如领域专家。

(7) 法律/伦理风险:如果生成数据被认为是"合成数据衍生品",版权和合规边界仍有争议。

缓解策略:

- 混合 RLHF + RLAI:关键能力维度用人工,大规模覆盖用 AI;
- 多裁判集成:多个不同模型同时打分,降低单一偏差;
- 定期人工抽样校验:观察 AI 判官是否漂移;
- 用更强的模型当裁判:Claude-3.5/4、GPT-4o 等;
- 结合可验证 reward:能用规则验证的(数学、代码、事实)就别用 AI 裁判。

第 3 章 RAG 与知识库问答

高频面试题详解 · Q1 ~ Q10

Q1. RAG 是什么?为什么有了长上下文模型还需要 RAG?

RAG(Retrieval-Augmented Generation,检索增强生成)由 Facebook AI 2020 年提出,核心思想是:在生成回答之前,先从外部知识库检索相关文档,把检索结果拼到 prompt 里,让 LLM 基于这些"外部证据"作答。

典型链路:用户 query → query 改写 / embedding → 向量检索(可能配合 BM25) → 多路召回 → Rerank → 拼装 prompt(query + 检索片段)→ LLM 生成 → (可选)答案验证 / 引用回填。

为什么"长上下文"模型出现后,RAG 仍然不可替代:

- (1) 知识时效性:LLM 训练数据有截止日期,公司内部文档、最新政策、刚发生的新闻都不在模型权重里。RAG 可以接最新数据库、API,信息永远是"现在时"。微调/重新预训练成本太高、迭代太慢。
- (2) 私有/敏感数据:企业知识(合同、客户数据、内部 wiki)不可能拿去训练公开模型;放进数据库做 RAG 既能利用,又能控制权限、做审计。
- (3) 成本与效率:每次都把 100 万 token 的全部知识库塞进 long context,每个请求几美元、延迟数十秒、KV Cache 爆显存。RAG 只取相关的几 KB,成本低 1-2 个数量级,延迟 ms 级。规模化部署只有 RAG 这条路。
- (4) 可解释、可追溯:RAG 能返回引用来源("这句话出自文档 X 第 3 段"),用户可以核对;监管、医疗、法律、金融等行业必须有此能力。纯长上下文模型无法精确归因到原文。
- (5) 长上下文性能衰减("lost in the middle"):研究表明,即使模型号称支持 128K 上下文,信息在中间位置时召回率显著下降(NIAH 测试),实际有效利用率远低于宣称长度。把"该看的"先筛出来再

塞进 prompt 比一股脑塞更靠谱。

(6) 知识可控可更新:RAG 中文档可以增删改查、版本管理、按权限分发;模型权重一旦微调进去就难以"忘记"。监管要求"被遗忘权"时,RAG 直接删 chunk,微调模型只能重训。

(7) 可组合性:RAG 可以与多模态(图片、表格)、结构化数据库、知识图谱(GraphRAG)灵活组合,生态成熟。

现代趋势是"Long Context + RAG"协同:用 RAG 精筛 top-k 相关文档(可能 10-50 个 chunk,数万 token),长上下文负责吸收和综合。两者不是替代关系。

Q2. RAG 最难的地方是什么?

RAG 系统从 demo 走到生产,真正的工程难点不在"接一个向量库 + LLM",而在以下几个层面:

(1) 数据质量(最根本):

"垃圾进垃圾出"。文档解析(PDF 表格、图片、扫描件、Word 复杂版式)极其难做对;OCR 错误、表格切碎、目录混入正文、页眉页脚噪声、HTML 标签残留.....处理不好,后面所有环节都白费。是 RAG 项目 60%+ 的工程量。

(2) 文档切分(Chunking):

Chunk 太小丢失上下文,太大检索精度差又拖累成本。如何做语义切分(段落、章节、表格不被切碎)、如何保留父子结构(小 chunk 召回 + 大 chunk 输入 LLM)、如何在不同文档类型间适配,没有银弹。

(3) 检索召回(Recall):

用户 query 和文档措辞不一致(语义鸿沟),纯 embedding 召不全;长 query、多意图 query 需要拆分;特定术语、产品名、代号 embedding 模型不认识,需要混合 BM25 + 向量 + 关键词召回 + 改写。如何融合多路、如何调阈值,都要看数据具体调。

(4) Rerank 与精排:

召回的 top-50 怎么精排到 top-5。Cross-encoder 重排序模型增加成本和延迟,模型选型、阈值都要试。

(5) Prompt 工程与上下文组装:

检索到的 chunk 顺序怎么排(相关度?时间?来源?)、有冲突信息怎么办、要不要附引用编号、prompt 模板怎么防止幻觉、怎么控制"找不到就承认"。

(6) 多轮对话理解:

用户后续问题常常省略主语或基于前文,需要做 query rewriting / 上下文重写,直接用最后一句去检索几乎一定失败。

(7) 评测:

没有 ground truth 的开放领域如何评测?需要分两段评:检索阶段(Recall@k、MRR、nDCG)+生成阶段(Faithfulness、Answer Relevance、Context Precision/Recall),RAGAS、TruLens 等框架辅助。建评测集本身就是大工程。

(8) 幻觉控制:

即使检索对了,LLM 仍可能凭记忆胡编、忽略 context、把无关 chunk 当依据。需要 prompt 约束、引用机制、二次校验。

(9) 工程指标:

延迟(检索 < 200ms,生成流式输出)、并发、QPS、向量库的更新策略、增量索引、A/B 测试、灰度。

(10) 业务对齐:

什么算"答对"是业务定的,不同部门、不同用户人群口径不一,产品和工程要反复 align。

一句话总结:RAG 难在"把每一环都做到 80 分以上"——任何一环掉到 50 分,系统效果就崩。

Q3. 文档切割策略有哪些?如何避免语义被切断?

文档切分(Chunking)目标是把长文档切成既适合检索(语义聚焦)、又适合 LLM 消化(长度适中、上下文完整)的片段。常见策略:

(1) 固定长度切分(Fixed-size Chunking):

按字符数或 token 数切(如每 500 token 一段,overlap 50 token)。

优点:简单、快速、可预测;

缺点:经常切碎句子、段落、表格,语义割裂严重。

改进:overlap(重叠)能缓解边界丢信息——前一块的尾部出现在下一块的头部。

(2) 递归字符切分(Recursive Character Splitting,LangChain 默认):

按层级分隔符递归尝试切:先按"\n\n"(段落)→ "\n"(行)→ "。(句号)→ " "(空格) → 字符。

在保证不超过目标长度的前提下,优先选高层级分隔符。

优点:比固定长度更尊重自然结构,实现成本低;

缺点:对结构化文档(Markdown、HTML、代码)不够智能。

(3) 语义切分(Semantic Chunking):

滑动窗口计算相邻句子 embedding 的相似度,在"语义跳变点"处切。

优点:每个 chunk 内部语义高度一致;

缺点:慢、贵、对 embedding 质量敏感、可能产生过长或过短的 chunk。适合关键知识库,不适合 TB 级数据。

(4) 结构感知切分(Structure-aware):

根据文档天然结构切——Markdown 按标题层级、HTML 按 DOM、PDF 按章节/表格/图、代码按函数/类。

优点:语义完整、保留层级关系;

缺点:需要为每种格式写解析器,工程量大。

主流工具:Unstructured、LlamaParse、PyMuPDF、Docling。

(5) 父子块 / 多粒度切分(Parent-Child / Small-to-Big):

存两套 chunk:① 小 chunk(如句子级)用于精准检索;② 大 chunk(如段落或章节)作为父块,检索命中小 chunk 后返回对应大 chunk 喂给 LLM。

优点:兼得检索精度和上下文完整性,是工业最常用方案之一;

缺点:存储翻倍、索引复杂度增加。

(6) 命题级切分(Proposition Chunking):

用 LLM 把文档拆成原子化"命题"(每个表达一个事实),每条命题独立索引。

优点:粒度极细,检索极精;

缺点:LLM 调用成本高,适合小而精的知识库。

(7) Agentic Chunking:

用 Agent 根据文档类型动态决定切法,甚至边读边切并加摘要 / 关键词。

如何避免语义被切断:

- a. 用层级分隔符 + 适度 overlap(典型 10%~20%);
- b. 表格、代码块整体不切——遇到 ``、|...|, <table> 等模式跳过切分;
- c. 给每个 chunk 加"上下文头":父标题 / 章节路径 / 文档名,作为前缀或 metadata,小 chunk 也能保留来源语境;
- d. 列表项不拆开,公式不切;
- e. Late Chunking(Jina 2024):先对整个长文档 embedding,再在 token 层切——避免边界问题;
- f. 用 LLM 做摘要 / 上下文丰富(Anthropic 的 Contextual Retrieval:每个 chunk 让 LLM 加一句"这是关于...的内容"做前缀,显著提升召回);
- g. 评测驱动:针对自己数据建小评测集,试 3-4 种切法,选最好的——没有通用最优。

工程实践:中文文档建议 200~500 字 / chunk;英文 500~1000 token;表格密集型文档建议结构感知切分 + 父子块组合。

Q4. Embedding 模型如何选择?BGE、M3E、OpenAI embedding 有什么差异?

Embedding 模型把文本映射为稠密向量,语义相似的文本向量距离近。选型直接决定 RAG 的召回上限。

主流模型概览:

BGE 系列(BAAI,智源):

- BGE-large/base/small(中英文)、BGE-M3(多语言 + 多向量 + 多功能)、BGE-Reranker。
- BGE-M3 是当前中文 RAG 的事实标准之一:支持 100+ 语言、最长 8192 token、同时输出 dense / sparse(类 BM25)/ ColBERT 多向量,一个模型搞定多种检索范式。
- 开源、Apache 2.0、可商用,可本地部署。

M3E 系列(MokaAI):

- M3E-base / M3E-large。早期(2023)中文 embedding 的代表,用大量中文数据训练。
- 当前在多数中文 benchmark 上被 BGE-M3、GTE-Qwen、Conan 等超越,但仍有用户。

OpenAI Embedding:

- text-embedding-3-small (1536d) / 3-large (3072d),支持 dimensions 参数动态降维。
- 闭源 API,英文表现强,多语言可用;中文不如 BGE-M3 等专精模型。
- 成本可控但数据出境;不能本地部署。

其他重要选项:

- GTE 系列(阿里):GTE-large、GTE-Qwen2-7B-instruct(7B 大 embedding,效果顶尖但贵);
- E5 / Multilingual-E5(微软):学术 baseline 之一;
- Cohere Embed v3:闭源,多语言强,带压缩格式;
- Jina Embeddings:支持超长(8K~32K)上下文,latency 低;
- Voyage AI、Mistral Embed、Nomic Embed:各有特色;
- Qwen3-Embedding:中文最新强力开源选项;

- Conan-Embedding(腾讯):MTEB-zh 多次登顶。

关键差异维度:

(1) 语言能力:中文场景首选 BGE-M3 / GTE-Qwen2 / Conan / Qwen3-Embedding,英文为主可选 OpenAI、Voyage、Cohere;多语言混合用 BGE-M3 / Cohere v3。

(2) 上下文长度:文档型场景需要长 chunk → BGE-M3 (8K)、Jina (32K)、Voyage (16K) 更合适;OpenAI 也支持 8K。

(3) 向量维度与成本:

维度高检索质量略好但存储成本高、检索慢;

OpenAI 3-large 3072d 可动态降到 1024/512;BGE-base 768d、large 1024d。海量数据(亿级)考虑量化(int8、binary)或低维。

(4) 部署模式:

本地化 / 私有部署 / 不出境 → 开源(BGE、GTE、Qwen3-Embedding);

快速接入、量小 → API(OpenAI、Cohere);

(5) 领域适配:

医疗、法律、代码等专业领域,通用 embedding 召回率会掉,考虑做领域微调(用 SimCSE / Contrastive learning,几千对就有效)。

(6) 检索范式:

只用 dense → BGE / OpenAI 都行;

想要 hybrid (dense + sparse) 一个模型搞定 → BGE-M3;

需要 ColBERT/late interaction → BGE-M3 / ColBERTv2。

选型方法论(强烈建议):

① 参考 MTEB / C-MTEB / AIR-Bench 榜单缩小候选;

- ② 用自己业务的真实 query + 文档建 200-500 条评测集;
- ③ 测 Recall@10 / Recall@50,延迟,部署成本;
- ④ 选 top2 各做 1 周线上 A/B,看真实效果。

一句话:多数中文 RAG 默认选 BGE-M3;英文密集可选 OpenAI text-embedding-3 或 Voyage;
有钱有卡追求极致用 GTE-Qwen2-7B / Qwen3-Embedding-8B。

Q5. 向量数据库怎么选?Milvus、Qdrant、FAISS、PGVector 有什么区别?

向量数据库的核心职责是高效存储 + ANN(近似最近邻)检索 + 元数据过滤 + 扩缩容运维。选型主要看数据规模、过滤需求、部署复杂度、生态。

FAISS(Facebook AI Similarity Search):

- 严格说不是数据库,是 C++/Python 库。提供 IVF、HNSW、PQ、IVFPQ 等 ANN 算法。
- 优点:速度极快、算法最全、GPU 支持好(GPU FAISS 单机十亿级向量秒级返回);
- 缺点:① 不是服务、要自己包 API;② 无原生过滤(metadata)支持(只能拿到向量 + id,过滤要自己做);③ 无持久化、无副本、无分布式(原生);④ 增删改不友好。
- 适合:研究、本地小规模 demo、其他向量库的底层引擎(很多向量库底层用 FAISS / HNSWLib)。

Milvus(Zilliz 出品):

- 云原生分布式向量数据库,Go + C++ 实现。
- 优点:① 真正的分布式架构(数据 / 计算分离,可独立扩缩);② 支持十亿~百亿级向量;③ 多种索引(HNSW、IVF、DiskANN、SCANN);④ 强 metadata 过滤;⑤ 支持稠密 + 稀疏 + 多向量;⑥ 生态成熟(Attu UI、PyMilvus、和 LangChain/LlamaIndex 集成);⑦ 商业云 Zilliz Cloud。
- 缺点:运维复杂(依赖 etcd、MinIO、Pulsar),小规模显得太重;学习曲线陡。
- 适合:中大型生产、需要分布式、QPS 高、数据量大(千万~百亿)。

Qdrant(Rust 写的,Qdrant Solutions):



- 单二进制 / Docker 极易部署,API 友好(REST + gRPC)。
- 优点:① 性能优秀(Rust + HNSW + Quantization);② 强大的 payload 过滤(JSON 字段任意条件);③ 支持稀疏 / 多向量 / NamedVector;④ 量化(scalar / product / binary)节省显存;⑤ 集群模式简洁;⑥ 文档清晰,DX 好。
- 缺点:相比 Milvus,在百亿级超大规模场景生态略弱;社区相对小。
- 适合:中小到中大规模(百万~亿级)生产、追求易用、Rust 性能信仰。

pgvector(PostgreSQL 扩展):

- 在 Postgres 里加向量类型 + IVFFlat / HNSW 索引。
- 优点:① 复用已有 Postgres 运维 / 备份 / 高可用 / 权限;② SQL 与向量混合查询(关系 + 向量一条 SQL 搞定,做 metadata 过滤特别自然);③ ACID 事务;④ 部署最简单。
- 缺点:① 单机性能不如专业向量库,十亿级吃力;② 索引重建慢;③ 高并发下 HNSW 内存占用大。
- 适合:已经用 Postgres、规模在千万级以下、看重一致性与 SQL 关联查询。

其他常见选项:

- Chroma:轻量、本地开发首选,基于 SQLite + HNSWLib,极简 API;生产规模不足。
- Weaviate:Go 写,模式丰富、有 schema 概念,GraphQL 接口;支持模块化(自动 embedding);
- Elasticsearch / OpenSearch:在 7.x+ 加了 kNN,可以稠密 + BM25 一站式;适合已经用 ES、想做 hybrid 搜索的团队;
- Pinecone:闭源 SaaS,免运维,价格相对贵,数据出境;
- LanceDB:嵌入式、列存、适合 ML 数据集 + 向量;
- Vespa:Yahoo 出品,大规模搜索 + 向量,生态独特;
- 阿里 OpenSearch / 腾讯云向量数据库 / 百度 VectorDB:云厂商方案。

选型决策树:

- 100 万以下、本地 demo → Chroma / FAISS / pgvector;
- 已经用 Postgres,千万级以下 → pgvector;
- 千万~亿,易部署,自建 → Qdrant;
- 亿~百亿,分布式,自建 → Milvus;

- 不想自己运维 → Pinecone / Zilliz Cloud / Qdrant Cloud;
- 已有 Elasticsearch → OpenSearch kNN;
- 需要 hybrid (BM25 + dense) + 多向量 → Milvus 或 Qdrant 或 Elasticsearch。

考量指标:Recall@10、QPS、P95 延迟、过滤条件下的性能、内存 / 磁盘占用、写入吞吐、增量索引能力、集群扩展、备份恢复。

Q6. 多路召回是什么?BM25 + Dense Retrieval 如何融合?

多路召回(Multi-way Retrieval / Hybrid Search)指同时用多种检索方式并行召回候选文档,然后融合结果,而不是单一依赖向量检索。它是工业级 RAG 的标配。

为什么要多路:

稠密向量(Dense Retrieval / Embedding)擅长捕捉语义相似("电脑卡顿"≈"机器很慢"),但对精确词匹配弱(产品型号、代号、人名、稀有术语 embedding 模型常常学不到位)。

稀疏检索(BM25 / TF-IDF)擅长精确匹配关键词、术语、产品名,但完全不懂同义词、改写。

关键词 / Metadata 过滤擅长结构化条件("type=合同 AND year=2024")。

互补结合后,稀疏路保证"该精确匹配的不漏",稠密路保证"该语义相似的也召回",大幅提升召回率(Recall)。

常见召回路:

- (1) 稠密向量检索(BGE / OpenAI Embedding + Milvus / Qdrant);
- (2) 稀疏检索:BM25(传统)或 BGE-M3 / SPLADE 等学习式稀疏向量;
- (3) 关键词检索(Elasticsearch 全文检索、正则);
- (4) 知识图谱召回 / SQL 召回(GraphRAG、Text2SQL);
- (5) 多 query 改写后并行检索;

(6) 父子块召回(命中小 chunk 返回父 chunk);

(7) HyDE(用 LLM 先生成假设答案,用答案 embedding 检索)。

BM25 + Dense 融合方法:

(1) RRF(Reciprocal Rank Fusion,最常用):

不看具体分数,只看排名。每条文档 d 的融合分 $= \sum 1/(k + \text{rank}_i(d))$, k 默认 60。

优点:无需对齐不同检索器分数(BM25 是无界数,余弦相似度在 $[-1,1]$),稳健、参数少、效果好。

HuggingFace、Elasticsearch、Vespa 都内置。

缺点:丢失了分数差异的信息。

(2) 加权线性融合:

$\text{score} = \alpha \cdot \text{score_dense} + (1-\alpha) \cdot \text{score_sparse_normalized}$ 。

需要先对各路分数归一化(min-max 或 z-score)。 α 在评测集上调,典型 0.5~0.7 偏 dense。

(3) Cascade / Two-Stage:

第一阶段 BM25 召回 top-1000(快、便宜),第二阶段在这 1000 个里用 dense + rerank 精排。

Recall 阶段宽松,Precision 阶段严格。

(4) Cross-encoder Rerank:

把多路召回的 top-50~100 合并去重,送 cross-encoder(BGE-Reranker、Cohere Rerank、Jina Reranker)做精排到 top-5~10。这是工业最常见也最有效的"最后一关"。

(5) Learning-to-Rank(LTR):

用机器学习模型(LambdaMART、神经网络)综合多种特征(BM25 分、dense 分、点击率、时间衰减)排序。需要标注数据。

工程实现要点:

- 同一份数据建两套索引:全文倒排(ES / OpenSearch / Tantivy) + 向量索引(Milvus / Qdrant);

- 并发发起查询,避免串行延迟;
- 每路召回 30~100 个,合并去重(用文档 id);
- 用 RRF 融合作为简单 baseline,再加 cross-encoder rerank 到 top-k;
- 长 query 先做 query rewriting / 分解再分别召回。

现代简化:BGE-M3 一个模型同时输出 dense + sparse + ColBERT 三种向量,可以直接做 hybrid,工程简化很多;Milvus、Qdrant 都原生支持。

Q7. Reranker 的作用是什么?什么时候必须用?

Reranker(重排器)接在召回之后,对召回的 top-k(通常 20~200)候选做精细打分排序,输出更高质量的 top-n(通常 3~10)送给 LLM。它是 RAG 召回 → 生成链路的"精排关卡"。

为什么需要 reranker:

召回阶段为了速度用 bi-encoder 或 BM25:query 和 doc 分别独立编码成向量,通过相似度近似检索。这种 late-interaction 范式快,但精度有限——很多看起来语义近、实则不答题的文档会混进 top-k。

Reranker 通常是 cross-encoder:query 和 doc 拼接作为一个输入送进 BERT 类模型,深度交互,直接输出相关性分数。

关键区别:

- bi-encoder(召回):2 次 forward, $O(n)$ 计算,但精度有限;
- cross-encoder(rerank):每对 (query, doc) 都跑一次 forward, $O(k)$ 但 k 小,精度高得多;不能离线索引,只能在线对召回结果跑。

主流 Reranker 模型:

- BGE-Reranker (base/large/v2-m3):中英双语强力开源选项;
- Cohere Rerank 3:闭源 API,多语言,工业广泛使用;
- Jina Reranker;

- mxbai-rerank;
- LLM-as-Reranker:用 GPT-4 / Claude 直接打分(贵但准,只用于关键场景或离线评测);
- ColBERT / ColBERTv2:late interaction,介于 bi 和 cross 之间,平衡精度和速度。

Reranker 的作用:

- (1) 显著提升 Top-K 精度(常见提升 nDCG@10 +5~15 个点);
- (2) 过滤召回阶段误召回的"近似不相关"文档,直接降低 LLM 幻觉风险;
- (3) 让 LLM 接收更少但更精的 context,降低 token 成本、加快生成、避免 lost-in-the-middle;
- (4) 在 hybrid 多路召回中,作为统一打分尺度,自然完成融合。

什么时候必须用:

- (1) 召回结果相关性参差(Top-50 里只有少数真正相关):几乎所有生产 RAG 都需要。
- (2) LLM context 预算紧,只能塞 top-3~5:必须 rerank 保证这几条是最优的。
- (3) 用户 query 复杂、有多重约束:bi-encoder 容易抓重点偏;cross-encoder 能"读懂"约束。
- (4) 业务对召回精度敏感(医疗、法律、金融):必须用,甚至要用 LLM-rerank。
- (5) 多路召回融合后,需要统一打分:rerank 是天然融合器。

什么时候可以不用:

- (1) 召回本身已经非常准(小知识库 + 强 embedding + 精心切分),top-3 已是好结果;
- (2) 极低延迟约束(< 100ms 端到端),rerank 加几十毫秒不可接受 → 用更快的 ColBERT-style 或不用;
- (3) 成本敏感的大规模场景(每天千万 query),先评估收益是否值得。

工程取舍:

- 延迟:base 模型在 GPU 上 50 个候选 ~50ms,large ~150ms,放 CPU 慢得多;
- 成本:本地部署一次性 GPU;Cohere Rerank ~\$2/1000 次;
- 候选数:rerank top-50 到 top-5 是常见配比;再往上效果边际递减;
- 量化与蒸馏:rerank 模型量化到 int8、用蒸馏小模型,可在 CPU 实时跑。

一句话:在生产 RAG 中,reranker 是性价比最高的一个组件,几乎"必加"。

Q8. RAG 如何降低幻觉?还有哪些幻觉无法靠 RAG 解决?

RAG 降低幻觉的核心机制是"用外部证据约束生成":让模型基于检索到的真实文档作答,而不是凭参数化记忆胡编。

RAG 能缓解的幻觉类型:

- 知识陈旧(模型不知道最新事件);
- 长尾事实(模型没记牢的小众知识);
- 数字 / 日期 / 人名错记;
- 不存在的引用、不存在的论文;
- 通用领域 vs 私有领域的知识冲突。

关键技术手段(从检索到生成全链路):

(1) 检索质量保证:

- 高质量切分 + 元数据(避免 chunk 把答案切碎);
- 多路召回 + Rerank 保证 top-k 真的相关;
- 召回相关性低于阈值时,主动告诉模型"未找到",而不是用劣质 context 强行生成;

(2) Prompt 工程约束:

- 明确指令:"只根据下列上下文回答,如果信息不足请回答'根据现有资料无法确定'";
- 给每个 chunk 编号,要求回答时附引用编号 [1][2];
- few-shot 示例展示"找不到就承认"的回答样式;
- 让模型先复述/总结引用,再回答(self-citation prompting);

(3) 引用回填与可验证:

- 生成后做引用验证:用 NLI 模型检查回答的每个断言是否真的被 context 支持(Faithfulness 检测);
- 不支持的部分自动删除或重新生成;
- 给用户展示引用,让人工二次核对;

(4) 自反思 / Self-RAG / CRAG:

- 让模型在生成过程中决定是否需要检索、检索结果是否够好、需不需要再检索一次;
- 对每个生成片段评估"支持/部分支持/不支持";

(5) 多步推理 / 拆解:

- 复杂问题先拆子问题,每个子问题独立检索 + 回答,再综合,避免一次性 query 召回不准;

(6) 模型层面:

- 优先选幻觉率低的基模(Claude、GPT-4 系列实测低于多数开源);
- 必要时做 RAG 微调,让模型学会"忠实引用"和"承认无知";

(7) 业务层兜底:

- 把"低置信回答"路由到人工;
- 风险高的领域(医疗、法律)给免责声明,强制返回原文链接。

RAG 解决不了的幻觉类型:

- (1) 检索召回失败:相关文档没召回上,LLM 还是基于参数记忆答 → 仍可能错。需要召回侧改进,不

是 RAG 框架本身能解决。

(2) 上下文冲突:多个 chunk 互相矛盾,LLM 选择哪一个不可控。需要时间感知 / 来源权威排序。

(3) 上下文歧义:文档本身写得模糊,LLM 给出"看似具体"的解释 → 编造细节。

(4) 推理类幻觉(数学、逻辑、代码):RAG 给的是知识,但推理步骤错了仍然错。需要 CoT、code interpreter、外部计算器,RAG 帮不上。

(5) 综合 / 归纳类:用户问"A 和 B 谁更好",文档没有直接比较,模型自己合成时容易越界。

(6) 过度自信表述:即使 context 模糊,LLM 倾向于用确定语气作答,这是模型校准(calibration)问题,不是 RAG 问题。

(7) 沿用错误知识:RAG 上下文给了正确信息,但模型记忆与之冲突时,有时仍偏向自己的旧记忆(尤其是高频常识冲突场景)。

(8) 文档本身就错:RAG 只能保证"基于文档作答",文档错 → 答案错。需要数据治理。

(9) 多模态幻觉:图表理解、OCR 错误、表格结构错位带来的幻觉,RAG 框架不能直接解。

(10) 长程综合:跨多文档、需要多步推理的综合题,即使每步检索都对,模型综合时也可能失真。

一句话:RAG 显著降低"事实性幻觉",但不能根除"推理性幻觉"和"系统性幻觉"。需要 RAG + CoT + Code + 评估 + 人工兜底组合拳。

Q9. RAG 与 Fine-tuning 怎么取舍?

这是大模型应用最常见的架构选型问题。RAG 和 Fine-tuning(包括 SFT、LoRA 等)解决的根本是不同维度的问题:

RAG 擅长:

- 注入动态、可更新、有引用的知识;
- 私有 / 时效性 / 长尾事实;
- 来源可追溯,合规要求;
- 知识可热更新,加新文档即可,无需重训。

Fine-tuning 擅长:

- 改变模型的"行为风格"(语气、格式、品牌口吻、专业措辞);
- 教会新技能(特定任务格式、Function Calling 格式、JSON 输出习惯);
- 深度领域适配(医疗术语、法律句式、代码语言偏好);
- 让模型在没有外部上下文时也能表现专业;
- 工具调用、ReAct 等结构性能力;
- 大幅压缩 prompt(微调进模型权重后,推理时不用再放长 system prompt)。

一句话直观对比:RAG 像考试时允许翻书,Fine-tuning 像考前背书。

维度对比:

(1) 知识更新:

RAG → 秒级(写库即可);Fine-tuning → 重训,周/月级;

(2) 成本:

RAG → 数据库 + embedding 模型 + 每次推理多花 token;

Fine-tuning → GPU 训练成本(LoRA 几百美元起,全参数千美元起),推理 token 更少;

(3) 可解释性:

RAG → 强,有引用;Fine-tuning → 弱,黑盒;

(4) 幻觉控制:

RAG → 显著降低事实性幻觉; Fine-tuning → 可能强化已有偏见, 甚至引入新幻觉;

(5) 数据要求:

RAG → 原始文档(几乎所有公司都有); Fine-tuning → 高质量标注样本(几千到几万对, 稀缺);

(6) 风格/格式控制:

RAG → 弱(只能 prompt); Fine-tuning → 强(直接学到风格);

(7) 私有数据安全:

RAG → 数据在自己库; Fine-tuning → 数据训进模型, 模型如果共享/泄漏, 知识也外泄;

(8) 隐私 / 遗忘权:

RAG → 删 chunk 即可; Fine-tuning → 几乎无法"遗忘"特定信息, 只能重训。

取舍决策框架(任务类型 → 推荐):

- a. 知识型 QA(客服、文档问答、法务检索): RAG 主导;
- b. 风格 / 格式 / 品牌口吻: Fine-tuning 主导;
- c. 工具调用 / Function Call 增强: Fine-tuning;
- d. 安全 / 拒绝行为: Fine-tuning(RLHF);
- e. 高度专业领域(医疗写作、合同生成): Fine-tuning + RAG 组合;
- f. 代码生成: 基模 + RAG(检索仓库 + API 文档); Fine-tuning 用于特殊框架;
- g. Agent 系统: 基模(可能少量微调) + RAG + Tool。

RAG + Fine-tuning 协同(工业最常见):

- 先微调让模型懂"我们公司的术语、回答格式、引用方式";
- 再用 RAG 注入最新知识;
- 两者互补, 不冲突;

- 也可以微调一个"RAG-aware"模型,专门擅长基于 context 作答(Self-RAG、Command-R 等)。

判断顺序建议:

- ① 先尝试 prompt engineering + RAG,80% 场景到这里就够了;
- ② 不行再加 Few-shot;
- ③ 还不行再加 LoRA 微调风格/格式;
- ④ 最后才考虑全参微调或 RLHF。

强调:Fine-tuning 不擅长"教新知识",它擅长"教新行为"。把新知识塞进微调通常事倍功半,还容易引入幻觉——这点是面试高频陷阱。

Q10. 如何评估 RAG?检索效果和回答效果分别看什么指标?

RAG 评估必须分两段:检索阶段(retrieval)和生成阶段(generation)分别打分,因为它们的瓶颈和优化方向不同。

一、检索阶段指标(Retrieval Metrics):

需要标注的 ground-truth:对每个 query 标注哪些文档/chunk 是相关的(可以二值或多级)。

(1) Recall@k:Top-k 召回中包含的相关文档占有所有相关文档的比例。RAG 中最重要的指标之一——召回不到,后面全废。

(2) Precision@k:Top-k 中相关文档的占比。和 Recall 平衡。

(3) MRR(Mean Reciprocal Rank):第一个相关文档的倒数排名取平均。1.0 = 第一个就对;0.5 = 平均第二个;直观衡量"是否能在最前面给出相关结果"。

(4) nDCG@k(Normalized Discounted Cumulative Gain):考虑了相关性等级 + 排名衰减的综合指标。多级相关性场景下最权威。

(5) Hit Rate@k:Top-k 中是否至少命中一个相关 → 二值,简单粗暴。

(6) MAP(Mean Average Precision):平均查准率,综合考虑所有相关文档的排名。

(7) 响应延迟、QPS、向量库内存占用:工程指标,生产必看。

二、生成阶段指标(Generation Metrics,Reference-free RAGAS / TruLens 体系):

(1) Faithfulness(忠实度,Hallucination 度量):

生成答案中的每个断言是否都能由检索到的 context 支持。

实现:把回答拆成原子陈述,逐条让 LLM 判断 context 是否支持(NLI 风格)。低分 = 幻觉多。

RAG 最关键的质量指标。

(2) Answer Relevance(答案相关性):

生成答案是否切题、是否回应了用户 query。

实现:用 LLM 从答案反向生成若干 query,与原 query 计算 embedding 相似度。

(3) Context Precision:

Top-k context 中真正被回答用到的占比。高 = 检索精准;低 = 召回里塞了无关。

(4) Context Recall:

回答所需的所有信息中,被检索到的占比(需要 ground-truth 答案)。低 = 召回不全,关键信息没拿到。

(5) Answer Correctness(答案正确性):

生成答案与 ground-truth 答案的语义匹配度(LLM judge 或 semantic similarity)。

(6) Answer Semantic Similarity:

答案与参考答案的 embedding 相似度,粗粒度但便宜。

(7) 自动指标 vs LLM-as-Judge:

- 自动指标(BLEU、ROUGE、BERTScore):便宜但与 RAG 场景契合度低;
- LLM-as-Judge(用 GPT-4 / Claude 打分):贵但更贴近人类判断,RAGAS / G-Eval / Prometheus 都基于此;
- 人工评测:终极标准,适合关键 release 评估。

三、端到端业务指标:

- 用户满意度评分、点赞 / 点踩比;
- 任务完成率 / 重复 query 率;
- 转人工率(客服场景);
- 平均会话长度、解决时长;
- 业务 KPI(转化率、留存)。

常用工具:

- RAGAS:开源,提供 faithfulness、answer relevance、context precision/recall 等;
- TruLens:LangChain 风格,可视化、追踪;
- DeepEval:Pytest 风格,适合 CI;
- Phoenix、LangSmith、Langfuse:RAG 全链路追踪 + 评估;
- 自建 LLM-judge:用 GPT-4 / Claude 写 rubric。

实践建议:

- ① 必须先建评测集——200~500 条真实 query + 标注答案 + 标注相关文档;
- ② 把链路拆开,分别评检索和生成,知道瓶颈在哪;
- ③ 离线 + 在线两套指标:离线跑 RAGAS / 人工,在线看用户行为;
- ④ 每次改动用同一评测集对比;
- ⑤ 业务上线后持续收集 bad case,反哺数据 / prompt。

简记口诀:检索看 Recall@k 和 MRR;生成看 Faithfulness 和 Answer Relevance;业务看用户满意



度。



第 4 章 Agent、Workflow 与多智能体

高频面试题详解 · Q1 ~ Q10

Q1. 什么是 LLM Agent?核心组件有哪些?

LLM Agent 是以大语言模型为"决策大脑",通过外部工具感知世界、执行动作、记忆经验,自主完成多步任务的系统。简单的"问→答"是 LLM,而 Agent 是"目标→思考→行动→观察→继续行动→...→完成目标"。

一个标准 Agent 通常包含五大组件:

(1) LLM(大脑 / Brain):

负责理解任务、规划步骤、决定调用哪个工具、解读工具结果、生成下一步动作和最终回答。是 Agent 的核心引擎,通常是经过指令微调的对话模型(GPT-4、Claude、Qwen 等)。

(2) Planner / Reasoner(规划与推理):

把复杂目标拆解为可执行的子步骤。常见模式:CoT(链式思考)、ReAct(Reason + Act 交替)、Plan-and-Execute(先制定完整计划再执行)、Tree-of-Thoughts、Reflexion(自我反思与纠错)。Planner 本质上是 prompt 工程 + 模型推理能力。

(3) Tools(工具 / 执行器):

Agent 与外部世界交互的手脚——搜索引擎、计算器、代码解释器、数据库查询、API 调用、浏览器、文件读写等。每个工具有明确的 name、description、JSON Schema 参数。LLM 通过 function calling 输出结构化调用,运行时执行后把结果返回。

(4) Memory(记忆):

让 Agent 跨轮、跨会话、长期记住信息。一般分三类:

- 短期记忆(Working Memory):当前对话上下文窗口内的内容;
- 长期记忆(Long-term Memory):重要事实、用户偏好,通常存到向量数据库,按需检索;

- 工作过程记忆(Episodic):任务执行轨迹(thought-action-observation),用于反思和复用。

(5) Environment / Executor(运行时环境):

工具调用的实际执行容器——浏览器沙箱、Python runtime、Shell、API 网关。负责安全隔离、超时控制、错误处理、并发管理。

(6) 可选扩展:

- Reflection / Critic:对自己产出的结果做自我评估、批评、修正;
- Multi-Agent 协作:多个角色 Agent 分工合作;
- Guardrails:输入输出审查、权限校验、敏感操作拦截;
- Observability:轨迹追踪、日志、监控、评测。

执行循环(典型 ReAct 范式):

while not done:

```
thought = LLM.reason(goal, history)    # 想下一步做什么
action = LLM.decide_tool(thought)      # 决定调用哪个工具及参数
observation = tool.execute(action)     # 执行并获得结果
history.append(thought, action, observation)
if LLM.is_done(): break
return LLM.final_answer(history)
```

与 Chatbot 的关键区别:

Chatbot 是"对话",一问一答;Agent 是"任务",自主分解、调用工具、长程执行。Chatbot 需要用户每步引导,Agent 在拿到目标后能自我推进。

主流框架:LangChain / LangGraph、LlamaIndex Agent、AutoGen、CrewAI、AutoGPT、OpenAI Assistants API、Anthropic 的工具使用,以及"手搓"(直接用裸 LLM API + 自定义状态机)。

Q2. Agent 和 Workflow 有什么区别?

Workflow(工作流)和 Agent(智能体)是两种不同的 LLM 应用范式,核心差异在"决策权属于谁"——是开发者预先编排,还是 LLM 运行时自主决定。

Workflow:

开发者预先用代码 / DSL 把执行路径写死,LLM 只在节点内作为"模块"被调用,完成特定任务(分类、抽取、生成、改写)。流程图是静态的,每次执行路径基本相同。

典型例子:

- 客服工单分类 → 抽取实体 → 查库 → 生成回复 → 审核;
- 文档解析 → 翻译 → 摘要 → 入库;
- 标准化 RAG:query 改写 → 多路召回 → rerank → prompt → 生成。

Agent:

LLM 拥有动态决策权——下一步做什么、调用什么工具、何时停止,都由模型自主决定。执行路径是动态的,同一目标不同次运行可能走不同路径。

典型例子:

- 用户说"帮我研究下电动车市场",Agent 自己决定搜索哪些关键词、爬哪些网站、读哪些 PDF、生成什么图表;
- 编程 Agent(Claude Code、Cursor Agent)自己 grep 代码库、编辑文件、跑测试、修 bug。

核心区别对比:

- (1) 决策权:Workflow 在编译期(开发者) vs Agent 在运行期(LLM);
- (2) 灵活性:Workflow 固定结构,新场景需要改代码;Agent 同一套代码处理万千场景;
- (3) 可控性:Workflow 强(每步可测可监控);Agent 弱(LLM 行为随机,可能跑飞);
- (4) 可预测性:Workflow 高(给定输入输出基本一致);Agent 低(每次轨迹可能不同);

(5) 调试与定位:Workflow 容易(节点级日志);Agent 难(轨迹长、模型决策不透明);

(6) 成本:Workflow 低(LLM 调用次数固定);Agent 高(轮数不可控,可能爆 token);

(7) 延迟:Workflow 可预估;Agent 不可控;

(8) 适用任务:

Workflow 适合规则清晰、重复执行、SLA 严的生产流程;

Agent 适合开放式、长尾、需要灵活探索的任务。

(9) 失败模式:

Workflow:节点失败可重试,影响有限;

Agent:可能陷入死循环、工具调用乱飞、目标偏移。

Anthropic 在《Building Effective Agents》(2024) 中的重要观点:

"很多团队过度热衷于做 Agent,而其实大部分场景 Workflow 就足够,且更稳定、便宜、可调试。
先用 Workflow 解决,真正需要动态决策时再升级为 Agent。"

混合模式(最常见):

- 主流程是 Workflow,某些节点内嵌小 Agent(比如"研究"节点);
- 顶层是 Agent,内部很多步用 Workflow 实现;
- LangGraph、Anthropic Computer Use 等都是这种混合架构。

选型建议:

- ① 任务能否用流程图画清楚?能 → Workflow;
- ② 输入空间是否封闭?是 → Workflow,否 → Agent;
- ③ SLA、成本、合规要求高?→ Workflow;
- ④ 长尾、探索型、用户输入多样?→ Agent;

⑤ 先 Workflow,瓶颈出现后再 Agent 化,渐进升级。

Q3. ReAct 是什么?如何实现?

ReAct(Reasoning + Acting,Yao et al. 2022)是 Agent 领域最经典、最基础的范式,核心思想是把"推理(thought)"和"行动(action)"在同一个 LLM 决策循环中交替进行,让模型边想边做、边做边想。

动机:

- 纯 CoT(Chain-of-Thought)只推理不行动,无法使用外部工具,知识被锁在参数里;
- 纯 Action-only(直接调工具)缺乏中间推理,容易乱调、漏调;
- ReAct 综合:用思考决定行动,用观察修正思考,形成闭环。

执行循环(ReAct Loop):

每一步 LLM 输出三段结构化文本:

- ① Thought:对当前状态的分析、下一步该做什么;
- ② Action:具体工具调用(工具名 + 参数);
- ③ Observation:工具返回的结果(由 runtime 填入,不是 LLM 生成);

然后回到 ① 继续,直到 LLM 输出"Final Answer"。

一个简单例子(查询某城市天气):

Question: 今天东京天气怎么样,要带伞吗?

Thought 1: 我需要查询东京今天的天气,使用 search 工具。

Action 1: search("Tokyo weather today")

Observation 1: Tokyo, Mar 22, partly cloudy, 18°C, 60% chance of rain in afternoon.

Thought 2: 有 60% 降雨概率,建议带伞。可以给出答案了。

Final Answer: 东京今天多云 18 度,下午有 60% 概率下雨,建议带把伞。

实现要点:

(1) Prompt 模板:

给 LLM 一段系统提示,声明可用工具列表(name + description + 参数 schema)、ReAct 输出格式(Thought/Action/Observation 三段)、终止条件(Final Answer)。附 1-2 个 few-shot 示例。

(2) 输出解析:

从 LLM 输出中正则或 JSON 解析出 Action 的 tool 名和参数。建议:

- 用 function calling / tool calling API(原生支持,无需手撕正则);
- 或用专门的特殊 token / XML 标签(<action>...</action>)。

(3) 工具执行:

在 runtime 把 LLM 解析出的 tool call 发到对应执行器(REST API、Python 函数、shell),捕获结果,处理异常(超时、参数错、抛错都要构造为可读的 Observation 返回给 LLM)。

(4) 上下文拼装:

把历史的 (Thought_i, Action_i, Observation_i) 全部累积到 prompt,作为 LLM 下一步决策的输入。注意上下文长度——超过窗口要做摘要、截断或滑动窗口。

(5) 终止控制:

- LLM 输出 "Final Answer" 标识 → 退出;
- 达到 max_iterations(防死循环);
- 上下文超长或时间超限 → 强制终止并返回当前最优答案;

(6) 错误处理:

工具失败 → 把错误信息作为 Observation 返回,让模型自己重试或换工具;

解析失败 → 让模型修正格式后重试;

陷入循环 → 检测重复 Action,主动打断。

代码骨架(伪代码):

```
def react_agent(query, tools, max_steps=10):
    history = [system_prompt(tools), user(query)]
    for _ in range(max_steps):
        output = llm.chat(history)
        if "Final Answer:" in output:
            return parse_final(output)
        tool_name, args = parse_action(output)
        try:
            obs = tools[tool_name](**args)
        except Exception as e:
            obs = f"Error: {e}"
        history.append(assistant(output))
        history.append(user(f"Observation: {obs}"))
    return "达到最大步数,未能完成"
```

优点:

- 推理 + 行动协同,效果优于单独 CoT 或 Action-only;
- 可解释性强,每步思考可见;
- 实现简单,prompt 工程即可。

局限:

- 早期决策错误后难以全局回退;
- 长任务上下文爆炸;
- 串行执行,无并行;
- 容易陷入"想太多"或在某工具上反复重试。

后续改进:Reflexion(失败后反思总结再重试)、Plan-and-Execute(先全局规划)、Tree-of-Thoughts(分支搜索)、Self-RAG、Agent + Memory 长期化等。ReAct 是这些方法的共同基础。

Q4. Plan-and-Execute、Reflexion 与 ReAct 有什么区别?

这三种是 Agent 推理与执行策略的三个代表性范式,逐步从"边想边做"演化到"先想后做"和"做错了反思"。

ReAct(2022):

推理与行动逐步交替,每一步基于当前 observation 决定下一步。"think-act-think-act-..."

优点:简单、自然、单步决策成本低;

缺点:① 短视(只看下一步,缺乏全局规划);② 早期错误难以纠正;③ 长任务容易在中途偏移目标;④ 串行无法并行加速。

Plan-and-Execute(Wang et al. 2023, "Plan-and-Solve Prompting"; LangChain Plan-Execute):

把决策分为两阶段:

- ① Planner(规划):LLM 先生成完整任务分解(子任务列表,可能是 DAG);
- ② Executor(执行):按计划逐项执行,每个子任务可以用 ReAct、单次 LLM 调用、或工具调用完成;
- ③ 必要时根据中间结果重新规划(Replanning)。

举例:"撰写一份特斯拉 2024 财报分析"

Planner 输出:

1. 搜索特斯拉 2024 财报原文;
2. 提取关键数据(营收、净利、汽车交付量);
3. 搜索同行业对比(比亚迪、福特);
4. 分析趋势;
5. 撰写报告。

Executor 按 1-5 执行,每步可并行(如 1 和 3)。

优点:



- ① 全局视野,不易偏题;
- ② 子任务可并行(降延迟);
- ③ 可用便宜模型做 **Executor**,贵模型做 **Planner**(成本优化);
- ④ 中间步骤可见,容易调试。

缺点:

- ① 计划僵化——初始信息不全时容易制定差的计划;
- ② 需要 **replan** 机制处理意外;
- ③ 计划过细会过度规划,过粗又退化为 **ReAct**。

Reflexion(Shinn et al. 2023):

在 **ReAct** 基础上加"反思 + 长期记忆"。一次任务失败或表现差时,LLM 不只是简单重试,而是生成一段自然语言"反思"(总结失败原因、改进策略),写入记忆,在下一次尝试时作为 **prompt** 的一部分,让模型不再犯同样的错。

执行流程:

for trial in range(N):

```
    trajectory = react_run(task, memory)
```

```
    score = evaluator(trajectory)      # 外部评估器(可以是规则、单元测试、reward  
model)
```

```
    if score >= threshold: break
```

```
    reflection = llm.reflect(trajectory, score) # 让 LLM 总结教训
```

```
    memory.append(reflection)
```

```
return best_trajectory
```

在 HotpotQA、AlfWorld、HumanEval 等 benchmark 上显著超越纯 **ReAct**。

优点:

- ① 引入"试错学习"机制,从失败中真正改进;

- ② 反思以自然语言形式存在,可读可控;
- ③ 不需要重新训练,只在推理时迭代。

缺点:

- ① 需要可计算的成功信号(外部 evaluator),开放任务难定义;
- ② 多次重试成本高;
- ③ 反思质量取决于 LLM,可能反思方向错。

三者对比:

维度 / ReAct / Plan-and-Execute / Reflexion

决策范围:逐步 / 全局先 / 多次试错

规划深度:浅(下一步)/ 深(整个任务)/ 跨次反思

并行能力:无 / 子任务可并行 / 多次尝试可并行

失败处理:逐步重试 / Replanning / 反思后重试

成本:低(单次)/ 中(规划 + 执行)/ 高(多次)

调试难度:中 / 易(计划可见)/ 中

典型框架:LangChain ReAct Agent / LangChain Plan-and-Execute / Reflexion 论文实现

适用场景:

- ReAct:轻量任务、单一工具、对话型应用;
- Plan-and-Execute:复杂多步任务、清晰目标(报告写作、数据分析、研究);
- Reflexion:有明确成功度量的任务(编程、数学、游戏、Web 任务);

工程上常组合使用:外层 Plan-and-Execute 制定计划,每个节点用 ReAct 执行,跨次用 Reflexion 反思——这就是现代复杂 Agent 的"三件套"。

Q5. 如何做任务拆分?为什么要拆分?效果如何评估?

任务拆分(Task Decomposition)是把一个复杂目标分解为若干子任务,逐个或并行解决,再综合结果。它是 Agent 处理长程、复杂任务的核心能力。

为什么要拆分:

- (1) 突破 LLM 单步推理上限:单个 prompt 处理复杂任务容易出错;拆分后每个子任务难度低、上下文短,模型更稳定。
- (2) 提高准确率:类似 CoT,把"一锤子买卖"变成"逐步推进",每步可验证,累积错误小。
- (3) 突破上下文长度:复杂任务的全部信息塞不进一个 prompt,拆分后每步只需局部信息。
- (4) 并行化:独立子任务可并行执行,大幅降延迟、提吞吐(MapReduce 思想)。
- (5) 异构资源:不同子任务用不同模型/工具——便宜模型做简单任务,昂贵模型做关键推理;不同 API 同时调用。
- (6) 可观测与调试:每个子任务有独立输入输出,失败可定位、可重试、可缓存。
- (7) 与人类协作:复杂任务可在某个子任务处插入人工审核(human-in-the-loop)。

常见拆分方法:

- (1) 按目标递归拆分(Recursive Decomposition):

Planner 把目标拆成 sub-goals,sub-goals 再拆,直到每个叶子节点是单步可执行的。形成任务树。

例:"写技术博客" → 选题、查资料、写大纲、写正文、配图、校对 → "写正文"再拆为"写引言、第一节、第二节..."

- (2) 按步骤拆分(Sequential / Linear):

把任务按时间或依赖顺序排成 1-2-3-4 线性序列。

例:"找一份工作" → 改简历 → 投递 → 准备面试 → 谈薪 → 入职

(3) 按角色拆分(Role-based):

Multi-Agent 场景:不同角色 Agent 处理不同子任务(产品经理、开发、测试、文案)。CrewAI、AutoGen 的核心模式。

(4) Map-Reduce:

对集合型任务并行拆分(对每篇文档摘要),再聚合(综合所有摘要)。长文档处理标准方法。

(5) Tree of Thoughts:

把每一步看作搜索树的节点,LLM 生成多个候选下一步,评估器打分,做 BFS / DFS / Beam 搜索,找最优路径。

(6) Skeleton-of-Thought(骨架优先):

先用一次 LLM 生成回答的"骨架"(若干 bullet),然后并行生成每个 bullet 的展开内容,最后拼装。降低延迟。

(7) Least-to-Most Prompting:

先把问题分解为难度递增的子问题,先解决简单的,后续问题可以参考前面答案。适合数学和逻辑题。

(8) 工具触发的隐式拆分:

ReAct 风格——不显式给出完整计划,每次"该用什么工具"就是一次拆分决策。

拆分的最佳实践:

① 子任务粒度合适:太粗 = 没用,太细 = overhead 大;一般每个子任务对应 1 次 LLM 调用 + 1-3 个工具调用;

② 子任务独立性:独立的能并行;耦合的串行;描述清楚依赖关系(DAG);

- ③ 子任务自包含:输入输出 schema 明确,不依赖隐式上下文;
- ④ 写入显式 plan:用 JSON / Markdown 列出,便于解析、可视化、修改;
- ⑤ 中间结果可序列化:存到 memory 或 scratch pad,后续步骤可引用;
- ⑥ Replan 能力:某子任务失败或新信息出现,允许 Planner 调整后续计划;
- ⑦ 收敛条件:明确什么时候算完成(全部子任务成功?达到 budget?评估器通过?);
- ⑧ 兜底:max_iterations、超时、token budget,防失控。

效果评估:

(1) 端到端指标:

- 任务完成率(Success Rate):任务最终是否达成目标;
- 答案准确率(Accuracy):有 ground-truth 的题(数学、问答);
- 人工评分(1-5 分):开放任务最终质量。

(2) 过程指标:

- 平均步数(Avg Steps):拆分是否过细;
- 工具调用次数 / 成功率:每步效率;
- 子任务通过率:哪些步骤是瓶颈;
- 重试次数:稳定性指标;
- 总 token / 总耗时 / 总成本。

(3) 轨迹指标:

- 计划质量:Planner 给的拆分是否合理(LLM-judge 或人工);
- 子任务独立性:是否真的并行了;
- 回退次数:replan 频次。

(4) 鲁棒性:

- 同一任务多次运行,成功率方差;

- 对抗输入(歧义、错误、缺信息)下的表现。

常用 benchmark:

- AgentBench:多场景 Agent 评估;
- GAIA:复杂真实问答任务;
- WebArena / VisualWebArena:Web 任务;
- SWE-bench / SWE-bench-verified:代码修复;
- ToolBench / BFCL:工具调用;
- AlfWorld、ALFRED:具身任务。

实践中,务必针对自己业务建小型评测集(20-100 个真实任务),分别测拆分质量和最终成功率,迭代 prompt 和拆分策略。

Q6. Agent 的记忆机制怎么设计?短期记忆、长期记忆、语义记忆区别?

Agent 的记忆机制让其跨轮、跨会话、跨任务保持信息一致与积累,是从"无状态聊天"走向"有持续性的助理"的关键。

记忆的三层划分(借鉴认知科学):

(1) 短期记忆(Short-term / Working Memory):

当前对话上下文窗口内的内容——最近若干轮对话、当前任务的 thought-action-observation 历史、临时变量、本次 session 的工具结果。

存储:直接放在 LLM 的 prompt context 里。

生命周期:当前会话或任务,任务结束就丢。

挑战:窗口有限(8K~200K token),长对话需要做摘要 / 滑动窗口 / 截断。

实现:LangChain 的 ConversationBufferMemory、ConversationBufferWindowMemory、ConversationSummaryBufferMemory(超过阈值自动摘要)。

(2) 长期记忆(Long-term Memory):

跨会话、持久化保存的信息——用户偏好、历史交互摘要、关键事实、学到的教训。

存储:外部数据库,通常是向量库(语义检索)+ KV 库(精确查找)+ 关系库(结构化)。

生命周期:可能永久,或有 TTL 退化。

检索:用户每条新输入触发"记忆检索"——把可能相关的旧记忆按 **embedding** 相似度 + 时间衰减召回,塞进 **prompt**。

挑战:写什么、何时写、如何更新、冲突如何处理、隐私如何保护、容量如何控制。

实现:Mem0、Letta(MemGPT)、Zep、LangChain VectorStoreMemory 等。

(3) 语义记忆 / Episodic / Procedural(认知科学中的细分):

- 语义记忆(Semantic Memory):事实性知识——"用户姓张"、"用户在上海工作"、"用户讨厌香菜"。结构化或半结构化,可用 KG 或 KV 存储。

- 情景记忆(Episodic Memory):具体事件 / 经历——"上周三用户问了关于 LLM 微调的问题"、"上次任务失败的原因是 API 超时"。带时间戳,可用于回放与反思。

- 程序性记忆(Procedural Memory):学到的技能 / 流程 / 偏好——"用户喜欢回答先给结论再展开"、"处理这类工单的标准流程是..."。可作为 **system prompt** 模板或 **few-shot** 示例。

在工程实现里,这三类常被合并入"长期记忆"统一处理,但区分有助于设计存储和检索策略。

常用工程模式:

(1) 滑动窗口 + 摘要:

保留最近 **N** 轮对话原文,更早的内容压缩成摘要,作为系统提示前缀。

(2) 重要事件抽取:

每轮对话结束后,让 **LLM** 抽取"值得记住"的事实(用户说了 **X**、决定了 **Y**),只把这些写入长期记忆。避免把闲聊全存进去。

(3) 反思日志(Reflection Log):

任务完成后总结一段"我学到了什么 / 下次要怎么做",写入 procedural memory。

(4) 多级检索:

收到新 query 时:① 短期记忆全部送入 prompt;② 用 query embedding 在长期记忆库召回 top-k 相关条目;③ 拼装统一上下文。

(5) 记忆更新策略:

- 写入:每轮检测是否有新信息,写入对应 store;
- 更新:用户更正旧信息时,标记旧条目失效或合并;
- 遗忘:基于时间衰减、访问频次、相关性打分,自动清理低价值记忆;
- 冲突解决:新旧冲突时优先新的,或保留两者带时间标记由 LLM 判断。

(6) 命名空间 / 分租户:

多用户场景下,每个用户独立 memory store,严格隔离,符合隐私合规。

(7) 主动检索 vs 被动召回:

- 被动:用户每条 query 触发一次召回(实现简单);
- 主动:Agent 自己决定"我现在需要什么记忆"并发起检索(更智能但复杂)。

挑战与陷阱:

- 召回错记忆 → 回答带偏(检索质量很关键);
- 记忆膨胀 → 检索质量下降,需要定期 compaction;
- 隐私 → 用户希望"忘掉"的内容必须可删;
- 一致性 → 不同 Agent 实例的 memory 是否共享;
- 安全 → 注入攻击(把恶意指令写入 memory 影响后续行为)。

代表方案:

- MemGPT / Letta:把 LLM 比作 OS,自己管理 main memory 和 external memory;
- Mem0:开源的 LLM memory 层,支持 fact extraction、conflict resolution;
- Zep:专为对话设计的 memory 服务,自动总结、实体抽取;
- LangGraph Memory / OpenAI 的 Memory 功能。

一句话总结:短期记忆 = 上下文窗口;长期记忆 = 外部向量库;语义/情景/程序性是长期记忆里的子类;关键是"写什么、怎么检索、何时更新"。

Q7. 工具调用失败怎么办?

工具调用是 Agent 与外界交互的桥梁,实际生产中失败率不低(网络抖动、API 超时、限流、参数错误、上游服务 down、返回数据异常)。Agent 的健壮性很大程度取决于失败处理。

常见失败类型:

- ① 参数错误:LLM 输出的参数 schema 不合法、字段缺失、类型错;
- ② 网络错误:超时、连接拒绝、DNS 失败;
- ③ 业务错误:API 返回 4xx(权限、找不到资源)、5xx(服务端 bug);
- ④ 限流:429,QPS 超限;
- ⑤ 返回格式异常:返回 JSON 解析失败、空响应、HTML 错误页;
- ⑥ 语义错误:工具调用成功但结果"看起来不对",比如搜索返回了无关结果;
- ⑦ 工具不存在:LLM 幻觉了一个不存在的工具名;
- ⑧ 工具结果过大:返回数据爆 context 窗口。

应对策略(分层):

(1) 参数 Schema 校验 / 修正:

- 用 JSON Schema 严格校验 LLM 输出的参数;
- 用 function calling / structured output 原生约束;
- 不合法 → 把错误信息(哪个字段、为什么不合法)作为 observation 返回给 LLM,让它修正后重

新输出;

- 用 Pydantic、Zod、Guardrails、Outlines 在生成阶段就约束格式;

(2) 重试(Retry):

- 对可重试错误(网络、5xx、429)做指数退避重试(exponential backoff + jitter);
- 不可重试错误(参数错、权限)不重试,直接报错;
- 限制最大重试次数(3 次足够),避免雪崩;
- 用 tenacity、urllib3 Retry、p-retry 等库实现;

(3) 超时(Timeout):

- 每个工具调用设硬超时(API 通常 5~30s,搜索可能更长);
- 超时后中断,返回"工具超时"observation,让 LLM 决定是否重试或换工具;
- 防止单个工具卡死整个 Agent;

(4) 降级(Fallback):

- 主工具失败 → 切换备用工具(主搜索 API down → 用备用搜索源);
- 实时 API 失败 → 用缓存的旧数据 + 标注"数据可能过时";
- 高级模型超时 → 降到普通模型;

(5) 熔断(Circuit Breaker):

- 某工具连续失败 N 次,熔断一段时间不再调用,避免重复浪费;
- Hystrix / resilience4j / 自实现状态机;

(6) 把错误暴露给 LLM:

- 关键:不要静默吞错或抛 Python exception;
- 把错误转化为人话写入 observation:"调用 search 失败,原因:连接超时。可以重试或换用其他工具。";
- LLM 能看到错误,就能自主决定换工具、调参数、或放弃任务给用户报告;

(7) 自我修正循环:

- 让 LLM 看到错误后修改参数重试,通常 1-2 次能修正(尤其是参数错误);
- 但要限制单一工具的最大尝试次数(如 3 次),防死循环;

(8) 工具结果验证:

- 调用成功不代表结果正确——结果可能不相关、空、过大;
- 加 **post-call** 校验:返回是否非空、是否符合预期 **schema**、是否相关;
- 大结果做截断 / 摘要再喂回模型,避免 **context** 爆炸;

(9) 工具集白名单:

- LLM 只能调用预注册的工具;
- 调用未注册工具直接拒绝,告诉 LLM"该工具不存在,可选工具是 X、Y、Z";

(10) 人工兜底(Human-in-the-loop):

- 多次失败 → 转人工;
- 高风险操作(退款、删除)必须人工确认;
- 给用户友好的失败提示 + 引导,而不是 "Error 500";

(11) 全链路追踪与告警:

- 用 LangSmith / Langfuse / OpenTelemetry 记录每个工具调用的 **trace**;
- 失败率、超时率打到监控系统,触发告警;
- 失败 **case** 收集进数据集,改进 **prompt** 或工具描述;

(12) 防御性工具设计:

- 工具自身要有幂等性(可安全重试);
- 大数据返回带 **pagination**,允许 Agent 按需取;
- 写操作有 **dry-run** 模式给 Agent 预演;

一个标准失败处理范式(伪代码):


```
def safe_call_tool(tool, args, max_retries=3):
    for attempt in range(max_retries):
        try:
            result = tool.execute(args, timeout=tool.timeout)
            if validate(result): return success(result)
            return observation("结果不符合预期: ...")
        except RateLimitError:
            sleep(backoff(attempt)); continue
        except TimeoutError:
            return observation("调用超时,可以重试或换工具")
        except ValidationError as e:
            return observation(f"参数错误: {e},请修正后重试")
        except Exception as e:
            log(e); return observation(f"工具调用失败: {e}")
    return observation(f"已重试 {max_retries} 次仍失败,请考虑其他方案")
```

黄金原则:失败不可避免,关键是让 Agent 看到失败、理解失败、能从失败恢复或优雅放弃。

Q8. Multi-Agent 如何协作?Supervisor、Swarm、Hierarchical 有什么区别?

Multi-Agent 系统让多个具有不同角色 / 工具 / 知识的 Agent 协作完成复杂任务,通常比单 Agent 拆任务更有效——专业分工 + 可扩展 + 模块化。

常见架构模式:

(1) Supervisor / Orchestrator(主管模式):

一个中心 Supervisor Agent 持有全局视角,负责接收用户请求、决定下一步该哪个 sub-agent 处理、传递结果给下一个 sub-agent、综合输出最终答案。Sub-agent 不直接互相通信,所有路



由经过 Supervisor。

类似公司中的项目经理:他不亲自做技术活,但分派任务给开发、测试、文档、设计四个组,跟踪进度,汇总结果。

优点:

- 控制清晰,容易调试;
- 可灵活添加/移除 sub-agent;
- Supervisor 可以做权限管理、风险评估;

缺点:

- Supervisor 成瓶颈,所有消息都过它;
- 复杂任务 Supervisor prompt 容易爆炸;
- 决策质量取决于 Supervisor 的能力。

LangGraph、CrewAI、AutoGen 都支持 Supervisor 模式。

(2) Hierarchical(层级 / 分层):

Supervisor 模式的递归版。顶层 Supervisor 把任务分给中层"主管",中层再分给底层 worker agent,形成树形组织。

例:Top Supervisor("研究并写报告") → Research Team Lead("收集资料") → 多个 Researcher Agent; Writing Team Lead("撰写报告") → 多个 Writer Agent。

优点:

- 适合非常复杂的任务,可分形扩展;
- 关注点分离,每层 prompt 简洁;
- 团队边界清晰,可独立优化;

缺点:

- 复杂度高,失败模式多;

- 跨层通信成本大;
- 不当设计可能"层层传话失真"。

(3) Swarm(对等 / 网状):

没有中心 Supervisor,每个 Agent 都是对等节点,可以直接互相调用 / 通信。Agent 自己决定要不要把任务转给另一个 Agent(handoff),类似一群专家彼此协商。

OpenAI Swarm(2024 开源)是代表实现:每个 Agent 持有一组工具,其中某些工具就是"调用其他 Agent"。

优点:

- 无单点瓶颈,扩展性好;
- 灵活——Agent 间动态发起协作;
- 适合"角色对话"型场景(辩论、谈判、模拟);

缺点:

- 路由不可预测,容易死循环 / 任务漂移;
- 调试和监控更难;
- 上下文同步复杂(每个 Agent 看到的历史可能不同);
- 容易"Agent 间互相礼貌让来让去,谁也不解决"。

(4) Pipeline / Sequential(管道):

Agent 按固定顺序串联,前一个的输出作为下一个的输入,流水线作业。

例:文本输入 → 翻译 Agent → 摘要 Agent → 风险审核 Agent → 输出。

其实更接近 Workflow,通常不算严格的"Multi-Agent",但常被归入。

简单可控,但失去 Agent 的动态决策优势。

(5) Debate / Reflection / Roundtable(辩论 / 评审):

多个 Agent 扮演不同立场或角色对同一问题独立给出方案,然后互相批评 / 投票 / 综合,得出最终答案。提升答案质量,适合开放性问题、创意、决策。

代表:Multi-Agent Debate (Du et al. 2023)、ChatEval。

(6) Blackboard(黑板模式 / 共享状态):

Agent 不直接互相调用,而是读写一块共享"黑板"(数据库 / 状态对象)。每个 Agent 监听某些条件,触发时执行,把结果写回黑板。

类似事件驱动 / Actor 模型,适合并发与异步。

协作的关键工程问题:

- (1) 通信协议:消息格式(自然语言、JSON、结构化)、Agent 间 schema 约定;
- (2) 上下文共享:全共享、部分共享、按需共享(避免上下文爆炸);
- (3) 路由决策:谁决定下一步该谁做(LLM router、规则、概率);
- (4) 终止条件:什么时候认为任务完成(避免无限协作);
- (5) 冲突解决:多个 Agent 给出矛盾结果时怎么处理(投票、Supervisor 裁决、辩论);
- (6) 角色定义:每个 Agent 的 system prompt、工具、知识库要清晰区分;
- (7) 错误传播:某 Agent 失败,整体如何降级 / 重试 / 重新路由;
- (8) 成本与延迟:多 Agent 调用次数指数级增加,要控制总 token / 时间预算;
- (9) 安全:Agent 间互相调用引入新攻击面(prompt injection 跨 Agent 传播);
- (10) 可观测性:Multi-Agent 轨迹比单 Agent 复杂得多,需要专门可视化(LangSmith、Langfuse、AutoGen Studio)。

选型建议:

- 简单任务 / 流程明确:Pipeline 或 Workflow;
- 单一目标 + 多种工具:Supervisor;
- 极复杂任务 + 团队感:Hierarchical;
- 开放性、动态协商:Swarm;
- 创意 / 决策 / 评审:Debate;
- 异步 / 事件驱动:Blackboard。

主流框架:

- AutoGen(微软):灵活,支持各种模式,LLM-as-orchestrator;
- CrewAI:角色 + 任务 + 流程的简洁抽象,生产友好;
- LangGraph:基于状态图,支持复杂控制流,适合工程化;
- OpenAI Swarm:轻量、对等协作示范;
- MetaGPT:模拟软件公司角色(PM / Architect / Engineer);
- AgentVerse / Camel:学术多智能体平台。

一个工程提醒:多数实际业务场景,单 Agent + 强 prompt + 多工具就够了,不必一上来就 Multi-Agent。Multi-Agent 是手段不是目标,只有任务确实需要多角色协作时再上,否则增加复杂度和成本。

Q9. 如何设计一个客服 Agent?

客服 Agent 是 LLM 应用最经典也最复杂的落地场景,涉及理解 / 检索 / 工具调用 / 安全 / 人工协作的全链路。下面是一个生产级设计的核心架构与考量。

一、需求与边界先想清楚:

- (1) 业务类型:售前咨询、售后客诉、技术支持、订单查询、退换货等;
- (2) 服务渠道:网站 IM、App、电话(语音)、邮件、电商客服;
- (3) 关键指标:首次响应时间、解决率、转人工率、用户满意度(CSAT)、单次会话成本;
- (4) 风险等级:涉资金 / 个人信息 / 医疗 / 法律的需更严控;
- (5) 多语言、多时区支持。

二、系统架构(七层):

层 1 · 接入层:

统一接入多种渠道(WebSocket / SSE 流式),前端展示流式回复 + 引用 + 推荐问题。

层 2 · 意图识别 / 路由:

用 LLM 或专门分类模型把用户问题分类为:售前 / 售后 / 退款 / 物流 / FAQ / 闲聊 / 投诉 / 转人工等意图,路由到对应处理 pipeline。

识别关键 entity:订单号、商品 ID、用户 ID、金额、时间。

层 3 · 知识检索(RAG):

向量库存:产品手册、FAQ 库、知识库、历史工单(脱敏)、政策文档。

采用多路召回 + Rerank,确保答得有据可循。

高频问题可加缓存(redis)直接返回标准答案,降本提速。

层 4 · 工具调用层(Function Calling):

常见客服工具:

- query_order(order_id):查订单状态;
- query_logistics(order_id):查物流;
- check_user_info(user_id):用户基本资料(权限受控);
- create_ticket / update_ticket:开工单 / 改工单;
- initiate_refund(order_id, reason, amount):发起退款(高风险,需审批);
- send_coupon(user_id, type):发优惠券安抚;
- transfer_to_human(reason):转人工;
- escalate_to_supervisor:升级到主管。

工具 schema 严格约束;高风险工具(退款、改地址)必须二次确认或人工审批。

层 5 · Agent 编排:

中等复杂度任务用 ReAct;关键任务用 Workflow 控制每一步;高度个性化用 Plan-and-Execute。

上下文管理:对话历史 + 用户画像 + 当前订单 + 当前任务状态。

会话状态机:greeting → identify → diagnose → solve → confirm → close。

层 6 · 安全与合规:

- 敏感词 / Prompt Injection 检测(输入 + 输出);
- PII 脱敏:用户姓名、电话、地址在日志和 LLM 调用时脱敏;
- 输出审核:答案不能包含未授权信息、不能承诺超出权限的事;
- 拒绝越权操作:用户要求"帮我改密码"不能直接做,引导走标准流程;
- 不冒充人类客服(透明声明 AI 身份,符合监管);
- 不给医疗 / 法律建议,引导专业渠道;
- 操作可审计:每次工具调用、每次回复都打日志,留 90+ 天。

层 7 · 人机协作:

- 转人工触发条件:用户明确要求 / Agent 多次失败 / 高风险操作 / 情绪激烈(检测愤怒、投诉关键词);
- 转人工时把对话摘要 + 关键信息推给人工坐席,坐席无缝接管;
- 人工处理完毕可"放回"Agent 继续后续轻量交互;
- 人工的处理结果回流为标注数据,持续优化 Agent。

三、关键设计细节:

(1) Prompt 设计:

- 明确身份:"你是 X 公司客服助理 '小 X',严格遵守...";
- 行为准则:语气友善、不承诺无法兑现的事、不知道就承认、引用知识库;
- 工具使用指南:工具列表 + 何时该调用;
- Few-shot:几个标准好答案的例子;
- 拒绝模式:超出范围的问题如何引导;

(2) 多轮对话理解:

用户后续提问常省略主语("那它什么时候发货?"),需要做 query rewriting / 上下文重写,把消解后的完整问题用于检索和处理。

(3) 主动引导:

- 不要光被动等问;
- 用户给出订单号 → 主动展示订单摘要;
- 物流异常 → 主动给出解决方案选项;
- 卡壳时提供快捷选项按钮(避免用户打字);

(4) 情绪识别与安抚:

检测到用户愤怒 / 沮丧 → 调整语气、道歉、提速、考虑直接转人工。可以用情感分类模型或 LLM 自行判断。

(5) 多模态(可选):

用户可能上传图片(残损商品照片、订单截图、错误页面)。接入 VLM 做识别。

(6) 知识库治理:

专门团队维护 FAQ / 知识库,新政策上线立即同步;给每条知识打 metadata(时间、地区、版本);定期清理过时内容。

(7) Bad Case 回流:

用户点踩 / 转人工 / 投诉 都自动进入 bad case 池,每周复盘,改 prompt / 加知识 / 加规则。

(8) 评测体系:

离线:用真实历史工单测自动化指标(意图准确率、答案 faithfulness、转人工合理率);

在线:用户 CSAT、解决率、平均处理时长(AHT)、转人工率;

A/B:新版本上线分流验证;

人工抽检:每周抽 100 条人工标注,跟踪长期质量。

(9) 成本控制:

- 简单问题用便宜模型(如 Haiku / Qwen-turbo);复杂工单升级到强模型;
- 高频 FAQ 直接走规则 / 缓存,不走 LLM;
- Prompt Caching 复用 system prompt;

- 限制最大轮数和 token budget;

(10) 可观测性:

全链路 trace(LangSmith / Langfuse);关键指标 dashboard;异常告警(成功率突降、延迟飙升);

四、上线节奏建议:

1) MVP:覆盖前 20% 高频问题,纯 RAG 不带工具调用;

2) 加工具:订单查询、物流查询(只读 + 低风险);

3) 加工单创建、转人工;

4) 加退款、优惠券等写操作(带审批);

5) 个性化、多模态、主动服务等高级特性。

一句话:客服 Agent = RAG + 工具调用 + 多轮对话 + 风险控制 + 人工兜底,五个模块缺一不可,工程实施远比模型选型重要。

Q10. Agent 如何评测?

Agent 评测远比单一 LLM 评测复杂,因为 Agent 涉及多步、工具调用、长程依赖、不确定路径。需要从结果、过程、安全、成本四个维度综合衡量。

一、评测维度:

(1) 任务成功率(Success Rate / Task Completion):

终极指标。任务最终是否达成目标。

- 有 ground-truth 答案的:逐条对比(数学、QA、代码);

- 开放任务:用 LLM-judge 或人工评分(1-5);

- 业务任务:看业务指标(订单是否创建、邮件是否发出、bug 是否修复)。

(2) 工具调用质量:

- 工具选择准确率:该调用工具时是否调用、是否选了对的工具;

- 参数填充准确率:调用时参数是否正确(尤其是 ID 类、数字类);
- 调用成功率:被调用的工具是否真的成功执行;
- 多余/遗漏调用:无关调用次数、本该调但没调的次数。

主流 benchmark:BFCL(Berkeley Function Calling Leaderboard)、ToolBench、API-Bank、Nexus。

(3) 轨迹评估(Trajectory Evaluation):

Agent 的"过程"也要看,不只看结果。

- 步数是否合理(过多 = 兜圈,过少 = 草率);
- 每步推理是否 coherent;
- 是否有重复 / 循环 / 错误回溯;
- 工具调用顺序是否合理;
- 与"参考轨迹"的相似度(如果有标准答案路径);

可以用 LLM-as-judge 评分,或人工抽检。

(4) 鲁棒性 / 一致性:

- 同一任务多次运行的成功率方差(随机种子 / 温度);
- 对抗输入(歧义、不完整、错误信息、prompt injection)下的表现;
- 模型替换 / 工具不可用 / 长尾输入下是否优雅降级。

(5) 安全性:

- Prompt Injection 抗性:用户/工具结果中嵌入恶意指令,Agent 是否被劫持;
- 越权操作:Agent 是否会调用未授权工具或泄漏敏感信息;
- 有害输出:暴力、色情、歧视、虚假信息的产出率;
- 拒绝合理性:该拒绝时是否拒绝,不该拒绝时是否过度拒绝(over-refusal)。

(6) 效率与成本:

- 平均 token 消耗(input / output);

- 端到端延迟(P50 / P95);
- 工具调用次数;
- 单次任务美元成本;
- 各模型 / 工具的调用比例。

(7) 用户体验(在线指标):

- 用户满意度(CSAT / NPS);
- 任务放弃率(用户中途离开);
- 转人工率;
- 留存与复访;
- 点赞/点踩比。

二、评测方法:

(1) Offline Benchmark 评测:

主流公开 benchmark:

- AgentBench(清华):8 类场景的综合 Agent 评测;
- GAIA:真实复杂问答 (Meta);
- WebArena / VisualWebArena:Web 任务;
- SWE-bench / SWE-bench-verified:真实仓库 bug 修复;
- ToolBench / BFCL:工具调用;
- AlfWorld、ALFRED:具身任务;
- TAU-Bench(τ -bench):多轮工具调用对话;
- OSWorld:操作系统级任务;
- AppWorld:手机/桌面 App 任务;
- AgentBoard:综合多维度。

(2) 自建任务评测集:

业务场景必须自建——20-200 条真实业务任务,包含简单 / 复杂 / 边缘 / 异常案例。

每条任务标注:输入、期望输出、关键工具调用、可接受答案范围。

(3) LLM-as-Judge:

用 GPT-4 / Claude 类强模型给 Agent 输出打分,搭配明确 rubric。

注意:LLM judge 自带偏好(长度、风格偏见),关键场景需要人工抽样校准。

(4) 单元 / 集成测试:

工程化把任务封装为 pytest-style 测试。每次代码 / prompt 改动自动跑全套测试,CI/CD 集成。

工具:DeepEval、Promptfoo、Phoenix、Inspect AI。

(5) Trace 追踪 / 可视化:

LangSmith、Langfuse、Helicone、Arize Phoenix——记录每个 Agent run 的完整 trace(每条 LLM 调用、工具调用、参数、结果、token、延迟),便于查 bug 和复盘。

(6) Replay / 回归测试:

把历史失败 case 沉淀为回归集,每次新版本必须保证不退化。

(7) A/B / Shadow Mode:

上线时新版本和旧版本对照,看真实业务指标;Shadow Mode 让新版本在生产数据上跑但不返回用户,先评估再切流。

(8) Human-in-the-loop 评测:

人工标注员对 Agent 轨迹打分(每月 100-500 条),作为 LLM-judge 的校准基线,捕捉自动化指标遗漏的细节。

三、评测中的常见陷阱:

(1) 只看最终答案对错,忽略轨迹质量;

(2) Benchmark 上 SOTA 不等于真实业务好用(数据分布、长尾差异大);

- (3) LLM-judge 偏见未校准;
- (4) 评测集 leak 到训练数据(尤其用了 GPT-4 生成评测集时);
- (5) 评测集太小,统计意义不足(至少 50+ 条);
- (6) 不分维度——成功率高不代表安全;延迟低不代表准确;
- (7) 忽略成本——准确率涨 2% 但成本翻 5 倍,可能不值。

四、实践 checklist:

- 建立 50-200 条真实任务评测集 + ground-truth;
- 分别评:任务成功率 / 工具调用 / 轨迹 / 安全 / 成本;
- 自动化(CI 跑) + LLM-judge + 人工抽样 三层结合;
- 引入公开 benchmark 一项(BFCL 或 AgentBench)做横向对比;
- 上线后持续收集 trace,bad case 回归集滚动更新;
- A/B 测试新版本,看真实业务指标;
- 安全测试(prompt injection、越权)定期跑;
- Dashboard 实时监控线上关键指标,设告警。

一句话:Agent 评测要"结果 + 过程 + 安全 + 成本"四位一体,benchmark 看趋势、自建集看业务、人工抽检看真实质量。

第 5 章 Function Calling、MCP、A2A 与工具调用

高频面试题详解 · Q1 ~ Q10

Q1. Function Calling 是什么?在 Agent 中起什么作用?

Function Calling(函数调用 / 工具调用)是让 LLM 把"我想调用某个外部函数"这一意图,以严格的结构化形式(通常是 JSON)输出出来,由应用层解析后实际执行函数,再把执行结果作为新的上下文返回给模型继续推理。

由 OpenAI 在 2023 年 6 月首次推出(后改名 tool calls),已成为现代 LLM API 的标准能力。Claude、Gemini、Qwen、DeepSeek、Mistral 等都支持。

工作流程:

- (1) 开发者声明可用工具:每个工具有 name、description、parameters(JSON Schema 描述);
- (2) 用户发起请求;
- (3) LLM 判断是否需要工具:如果需要,输出一个 tool_calls 结构(工具名 + 参数 JSON);如果不需要,直接给出文字回答;
- (4) 应用层解析 tool_calls,实际调用对应函数(可能是查数据库、调 API、执行代码);
- (5) 函数返回结果,以 role=tool 的消息形式附回对话历史;
- (6) LLM 看到工具结果后,继续推理,可能再调用更多工具,也可能给出最终答案。

在 Agent 中的作用:

- (1) 突破知识边界:LLM 的参数化知识是静态的、有截止日期的、有限的。Function Calling 让 Agent 能查实时数据(天气、股价、新闻)、私有数据(数据库、内部 API)、动态计算(代码运行)。
- (2) 行动能力:从"只会说话"升级到"能办事"——发邮件、订机票、改文件、控制硬件,把语言模型变成真正能执行任务的智能体。

(3) 标准化决策接口:无需手撕正则解析自然语言,模型直接输出可验证的 JSON,工程上稳定、可调试、可监控。

(4) ReAct 与 Agent 循环的基石:Thought-Action-Observation 循环里的 Action,本质就是 Function Calling。所有主流 Agent 框架都建立在 function calling 之上。

(5) 模块化能力扩展:工具是即插即用的——新增一个工具只需写函数 + schema,无需重训模型。

(6) 多步任务串联:LLM 通过连续调用多个工具完成长链条任务(查订单 → 看物流 → 算赔偿 → 发优惠券)。

(7) 结构化输出兜底:即使不真的执行函数,Function Calling 也可以用来强制 LLM 输出符合 schema 的 JSON,用于数据抽取、表单填充。

与传统 NLP 工具调用的区别:

过去用 LLM 调工具要靠 prompt 引导 + 正则解析,容易格式错;Function Calling 是模型经过专门训练 + API 层面格式约束,稳定性显著提升。

工程要点:

- 工具描述写清楚(name、description、参数语义);
- 工具数量不要过多(20+ 工具准确率会下降,可分层 / 按需注入);
- 参数 schema 严格,加 enum、range、required 约束;
- 处理工具失败、参数错误的重试与降级;
- 多工具并行调用(现代 API 支持 parallel tool calls)可降延迟。

一句话:Function Calling 把 LLM 从"自然语言生成器"变成"可执行决策中枢",是 Agent 时代最关键的底层机制。

Q2. Function Calling 和普通 Prompt 调 API 有什么区别?

两者都能让 LLM"调用工具",但实现机制、可靠性、工程体验差异很大。

普通 Prompt 调 API(Prompt-based Tool Use):

在系统 prompt 里用自然语言告诉 LLM:"如果你需要查天气,请输出 SEARCH(city);如果需要计算,输出 CALC(expression)..."。然后用正则或解析逻辑从模型输出文本里抽出工具调用意图。这是 Function Calling 出现之前的主流做法,ReAct 论文最早就是这样实现的。

Function Calling(Tool Calls):

通过 API 层声明工具 schema(JSON Schema 格式),模型在专门训练过的 tool-use 数据上学会输出结构化 tool_calls 字段(独立于普通 content)。应用代码直接拿到 JSON,不需要再解析自然语言。

核心区别(逐项对比):

(1) 输出格式可靠性:

Prompt 调:输出是自由文本,可能格式漂移、漏字段、嵌入闲聊("好的,我来调用 SEARCH(北京)..."),正则随时崩;

FC:输出是结构化字段,服务端强制 schema 校验,基本不会出格式错。

(2) 模型训练:

Prompt 调:依赖通用指令跟随能力,小模型表现差;

FC:经过专门的 tool-use SFT/RL 微调(BFCL 上有明显差距),小模型也能稳定输出。

(3) 解析复杂度:

Prompt 调:开发者要写正则 / 状态机 / fallback,易出 bug;

FC:SDK 直接给出结构化对象,开发者就当本地函数调用。

(4) 多工具支持:

Prompt 调:工具越多,prompt 越臃肿,准确率快速下降;

FC:工具 schema 单独传递,模型对工具数量更不敏感(也有上限,但好得多)。

(5) 参数约束:

Prompt 调:类型、枚举、必填靠 prompt 描述,模型可能填错类型或漏字段;

FC:JSON Schema 原生支持 type、enum、required、pattern,API 层可强校验。

(6) 并行调用:

Prompt 调:很难,一般串行;

FC:OpenAI、Claude、Qwen 等支持 parallel tool calls,一轮返回多个调用,显著降延迟。

(7) Token 成本:

Prompt 调:工具描述全塞 system prompt,每次都重传;

FC:工具描述也占 token,但配合 Prompt Caching 可以大幅复用,实际成本通常更低。

(8) 流式输出:

Prompt 调:工具调用混在文本流里,解析麻烦;

FC:tool_calls 在结构化字段中,流式输出可分别处理 content delta 和 tool_call delta。

(9) 错误反馈:

Prompt 调:工具失败要靠开发者把错误信息构造回 prompt;

FC:有 role=tool 的消息类型,标准化错误回填,模型理解更准。

(10) 生态与互操作:

FC:已经成为事实标准,LangChain、LlamaIndex、AutoGen、MCP 都基于 tool calls 协议;

Prompt 调:每个项目自己一套格式,无法互通。

何时仍可能用 Prompt 调:

(1) 基模不支持 FC(很老的开源模型或自训模型);

(2) 极简 demo / 原型验证;

(3) 想要在思考过程中夹带工具调用(ReAct 风格的 thought-action 文本流);

(4) 极度定制化的 DSL(比如让模型输出 SQL 或 Python 代码而非函数调用)。

现代实践:

- 优先用 Function Calling, 稳健、易维护;
- 复杂场景配合 structured output / JSON schema mode 双保险;
- 用 BFCL、ToolBench 评估不同模型的 FC 能力差距, 选模型时不能只看通用 benchmark;
- 老模型或本地小模型考虑用 Outlines / Guidance 等 constrained decoding 把 prompt 调升级到结构化输出。

一句话: Prompt 调 API 是 "约定俗成 + 正则解析", Function Calling 是 "API 协议 + 训练保障", 后者在可靠性、可维护性、生态上全面胜出。

Q3. MCP 是什么?和 Function Calling 有什么区别?

MCP(Model Context Protocol)是 Anthropic 于 2024 年 11 月开源的开放协议,定位是"LLM 应用 ↔ 外部工具/数据源"之间的标准化接口,目标是让任意 LLM 应用都能用统一的协议接入任意工具,类似 LSP(Language Server Protocol)之于编辑器。

MCP 三角色:

- (1) Host: 用户使用的 LLM 应用, 例如 Claude Desktop、Cursor、Windsurf、Cline、ChatGPT Desktop;
- (2) Client: 运行在 Host 里、负责与某个 Server 通信的客户端进程, 一个 Host 可以管多个 Client;
- (3) Server: 暴露工具(Tools)、资源(Resources)、提示模板(Prompts)、采样(Sampling)的服务进程, 实现具体能力(查 GitHub、读文件、操作数据库、发邮件)。

MCP 三大能力原语:

- Tools(工具): 可被 LLM 调用执行的函数, 与 Function Calling 一一对应;
- Resources(资源): 可被读取的上下文数据(文件、表格、文档), 由应用决定何时把哪些资源注入 prompt;
- Prompts(提示): 预定义的可复用 prompt 模板, 用户可显式触发;

- (扩展)Sampling:Server 反向请求 Host 调用 LLM,实现 server → LLM 的协作。

传输方式:stdio(本地子进程,最常用)、HTTP/SSE(远程)、Streamable HTTP(新版,2025)。

MCP 与 Function Calling 的关系:

Function Calling 是"模型 ↔ 调用约定"——LLM 把意图输出为 JSON,这是模型层面的协议。

MCP 是"应用 ↔ 工具源"——把工具的实现从应用里剥离,标准化成可独立分发、远程调用、复用的 Server。这是应用层面的协议。

可以把两者关系类比为:

Function Calling = LLM 内置的"函数调用语法"(C 语言的 func());

MCP = 操作系统级的"动态库 / RPC 标准"(.so / gRPC),让函数实现可以独立部署、跨进程调用。

一个 LLM 应用通常这样组合两者:

- ① 应用启动时连接若干 MCP Server,枚举它们暴露的 tools;
- ② 把这些工具的 schema 翻译为模型的 function calling 格式,放入 API 请求;
- ③ 模型决定调用某工具 → 输出 tool_calls;
- ④ 应用根据 tool 名找到对应 MCP Server,通过 MCP 协议调用,拿到结果;
- ⑤ 把结果以 tool message 回填到对话。

MCP 相比直接写 Function Calling 实现的优势:

(1) 复用与生态:一份 MCP Server 写好,Claude Desktop、Cursor、Windsurf、Cline 等数十个 Host 都能用,不必每个应用重写一次。GitHub MCP、Slack MCP、PostgreSQL MCP 等社区已经积累上千个服务。

(2) 解耦:工具实现与 LLM 应用解耦,工具升级、替换不影响应用。可以让数据库管理员维护数据



库 MCP,前端工程师维护应用,各司其职。

(3) 多语言:Server 可以用任何语言实现(Python、TypeScript、Go、Rust);Host 也可以用任意语言,只要支持协议。

(4) 远程能力:HTTP / SSE 传输让工具可托管在云端,实现 SaaS 化(Composio、Smithery、Pipedream 都在做)。

(5) 标准化 Resource / Prompt 等概念:工具之外,文档、模板、订阅推送等都被纳入统一协议,不只是函数调用。

(6) 安全 / 权限分层:每个 Server 独立沙箱、独立鉴权,Host 可控制哪些 Server 启用、哪些工具被允许调用,更安全。

(7) 跨 LLM:同一套 MCP 可被 Claude、GPT、Gemini、本地模型共用(只要 Host 适配),避免厂商绑定。

MCP 的局限/挑战:

- 协议年轻,2024 年末才推出,生态还在快速迭代;
- Server 本地启动开销(stdio 模式);
- 远程模式的认证、权限、审计还在演进;
- 工具发现机制(Server 注册中心)未完全标准化;
- 多 Server 时的工具命名空间冲突。

面试关键点(一句话):

Function Calling 解决"模型怎么表达想调用工具",MCP 解决"工具怎么被任何应用方便地接入"。前者是模型协议,后者是应用-工具协议;互补关系,不是替代关系。

Q4. A2A 了解吗?和 MCP 的关系是什么?

A2A(Agent-to-Agent Protocol)是 Google 于 2025 年 4 月推出的开放协议,定位是"Agent ↔ Agent"之间标准化通信协议,让不同厂商、不同框架开发的 Agent 能互相发现、互相协作。

A2A 的核心抽象:

- (1) Agent Card:每个 Agent 在固定路径(/.well-known/agent.json)发布一个能力清单 JSON,声明自己能做什么(skills、authentication、endpoint),供其他 Agent 发现;
- (2) Task:协作的基本单位。一个 Agent(client)向另一个 Agent(remote)发起 task,带有目标、参数、状态;
- (3) Message / Artifact:任务执行过程中的消息流(文本、文件、结构化数据)。支持多轮交互、流式输出、断点续传;
- (4) States:task 有明确的生命周期状态(submitted、working、input-required、completed、failed、canceled),便于跨 Agent 协同。

通信方式:基于 HTTP + JSON-RPC + SSE,与 OpenAPI 兼容,易接入现有微服务体系。

A2A 解决的核心问题:

现实中 Agent 不是单兵作战——企业里可能有 HR Agent、IT 工单 Agent、采购 Agent、销售 Agent 来自不同供应商或团队。怎么让一个 Agent 调用另一个 Agent?如果每对 Agent 都搞私有集成,复杂度爆炸。A2A 提供统一协议,让 Agent 之间像 Web API 一样互联。

A2A vs MCP 的核心区别:

(1) 通信方向 / 对象:

MCP:LLM 应用(Host) ↔ 工具/数据(Server),本质上是"应用调资源";

A2A:Agent ↔ Agent,本质上是"智能体之间互相委派任务"。

(2) 调用方的"智能水平":

MCP 的 Server 通常是"哑工具"——给定参数,执行,返回结果,没有自主决策;

A2A 的 remote agent 是"另一个完整 Agent"——可能自己再思考、拆任务、调工具、再调别的 Agent,可能带有自己的 LLM 大脑。

(3) 交互模式:

MCP:典型的请求-响应(可流式),Server 不主动发起;

A2A:更接近多轮异步对话——任务可能持续数分钟甚至数小时,Agent 间发送多轮 message,可以暂停等用户补充信息(input-required 状态)。

(4) 任务状态管理:

MCP 通常无状态(每次调用独立);

A2A 内建 task 生命周期,支持长任务跟踪、暂停恢复、取消。

(5) 发现机制:

MCP:由 Host 配置文件指定要连哪些 Server;

A2A:有 Agent Card + 发现机制,Agent 可以"逛 marketplace",动态发现可协作的 Agent。

(6) 设计目标:

MCP:扩展单个 Agent 的能力(给它更多工具和数据);

A2A:让多个 Agent 组成生态,跨组织协作。

两者关系:互补,不冲突。

官方表述:MCP 解决"Agent 怎么用工具",A2A 解决"Agent 怎么用其他 Agent"。一个完整的复杂 Agent 系统会同时用到两者:

- 内部:Agent 通过 MCP 接入数据库、文件、API 等基础设施;
- 外部:Agent 通过 A2A 协议与其他组织 / 部门的 Agent 协作完成跨域任务。

一个典型场景:

用户对 HR Agent 说"帮我安排一次员工的入职流程"。HR Agent 通过:

- MCP:查公司知识库、员工数据库、写入工单系统;
- A2A:把"开 IT 账号"任务派给 IT Agent,把"准备工卡"任务派给行政 Agent,把"发欢迎邮件"派给

Marketing Agent,自己跟踪所有子任务进度。

A2A 的现状(2025 中):

- 由 Google 主导,Linux Foundation 接管标准化;
- Salesforce、SAP、ServiceNow、Atlassian 等数十家厂商支持;
- 生态仍在早期,工具链、SDK 在快速演进;
- 与 OpenAI 的 Agent 工具、Anthropic 的 computer use 等并行发展,未来可能进一步融合。

面试关键句:

MCP 让 Agent 拥有"超能力"(用工具用数据),A2A 让 Agent 形成"超组织"(协作)。前者是垂直扩展,后者是横向扩展;两者一起构成多智能体时代的"水电煤"基础设施。

Q5. 工具参数 Schema 如何设计?

工具参数 Schema(基于 JSON Schema)是 LLM 工具调用的"接口契约",直接决定模型理解工具、填对参数、避免错误调用的能力。设计得好,模型用得稳;设计差,即使强模型也会乱调。

一、基本要素:

(1) name:工具名,要清晰、简短、动词开头("search_web"、"get_order"、"send_email")。避免缩写、命名空间冲突。

(2) description:工具的总体描述。这是模型决定何时调用工具的关键依据。写法要点:

- 一句话说清"这个工具做什么";
- 补一句"什么时候用";
- 必要时给个例子("用于查询用户当前订单状态,例如:用户问'我的订单到哪了'");
- 避免歧义:"search" 太泛,改为 "search_company_internal_wiki";
- 避免与其他工具描述冲突(模型会困惑选哪个)。

(3) parameters:JSON Schema 描述参数结构。

二、参数 Schema 设计原则:

(1) 字段命名规范:

- 用 snake_case 或 camelCase,保持一致;
- 名字自解释:order_id 而不是 id,start_date 而不是 date1;
- 避免缩写:create_date 而不是 cd。

(2) 类型严格:

- 优先用具体类型(string、integer、number、boolean、array、object);
- 不要全用 string 偷懒,给模型类型提示;
- 时间用 ISO 8601(string + format: "date-time"),不要让模型自己拼"2024 年 3 月 15 日";
- ID 类用 string 而非 integer(避免前导 0、超大数精度问题)。

(3) 必填与可选:

- 严格区分 required;
- 必填字段越少越好,可选字段给默认值(default);
- 模型对"必填"字段填错率更低,可选字段易遗漏。

(4) 枚举与约束:

- 选项有限的字段用 enum:status: ["pending", "shipped", "delivered"];
- 数值加 minimum / maximum;
- 字符串加 pattern(正则)、maxLength;
- 数组加 minItems / maxItems;
- 这些约束模型在生成时会自适应(配合 constrained decoding 效果更好)。

(5) description 字段级提示:

每个字段加 description,告诉模型:

- 这个字段是什么含义;

- 取值范围或示例;
- 与其他字段的关系;
- 反例(什么不该填)。

例:{"order_id": {"type": "string", "description": "订单号,格式为 'ORD-' 开头加 8 位数字,例如 ORD-12345678。如果用户未提供,要主动询问,不要编造。"}}}

(6) 嵌套结构要克制:

- 模型对深层嵌套对象准确率下降;
- 三层以上嵌套考虑拆成多个工具或扁平化;
- 数组里的对象 **schema** 也要写清。

(7) 避免过载:

- 一个工具不要塞 10+ 参数;参数多容易填错;
- 把多功能工具拆成多个单一职责工具("search_orders" + "search_users" 优于 "universal_search");
- 但工具总数也别太多(20+ 准确率下降),按场景动态注入。

三、常见坑与规避:

(1) 时间格式混乱:

不要让模型自己理解"明天"、"上周三";

应用层把相对时间转成绝对时间,或者提供专门的"current_date"参数让模型知道"今天"。

(2) 隐式依赖:

"如果 type=email,则必须有 to,但 type=sms 则必须有 phone"——JSON Schema 用 oneOf / anyOf / dependencies 显式表达,不要靠 description 文字提示。

(3) 模糊字段:

"参数:filter(object)"——模型懵了。给具体子字段:{"status": ..., "min_amount": ...,

"date_range": ...}。

(4) 返回值的隐式 schema:

虽然 schema 只描述输入,但 description 里最好提一句返回什么,让模型理解后续怎么用;或在系统 prompt 中说明。

(5) ID 类型陷阱:

大整数 ID 用 string,避免 JavaScript 53 位精度问题;UUID 也用 string。

(6) 多语言:

中文场景下,工具描述用中文模型表现可能更好(尤其国内模型);英文模型一般也能理解中文 description,但工具 name 用英文更稳。

(7) 安全:

不要把鉴权 token、用户 ID 暴露给 LLM 作为参数——应该应用层从 session 注入,工具签名层面不出现。

四、典型示例(电商查询订单):

```
{  
  "name": "get_order_status",  
  "description": "查询用户的订单状态、物流信息和预计送达时间。当用户询问'我的订单到哪了'、'订单状态'、'什么时候到'时调用。",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "order_id": {  
        "type": "string",  
        "pattern": "^ORD-\\d{8}$",  
        "description": "订单编号,格式为 ORD- 开头加 8 位数字。如果用户未提供,请先询问。"  
      }  
    }  
  }  
}
```

```
    },  
    "include_logistics": {  
      "type": "boolean",  
      "default": true,  
      "description": "是否包含物流详情,默认 true。"  
    },  
    },  
    "required": ["order_id"]  
  }  
}
```

五、调优实践:

(1) 评测:

- 用 BFCL 风格的小评测集(100-300 条真实 query)测工具选择准确率、参数准确率;
- A/B 不同 description 写法,找最优;

(2) 工具描述迭代:

- 看 bad case:模型为什么调错工具?为什么填错参数?调整 description;
- 高频混淆的工具对(search vs query),在 description 里互相消歧;

(3) Schema 严格化 + 兜底:

- 严格 JSON Schema 校验输出;
- 输出不合法 → 把错误信息回写给 LLM,让它修正后再次输出;
- 配合 structured output / JSON mode 兜底。

(4) 工具集合精简化:

- 把太多工具按场景分组,只注入当前任务相关的子集;

- 或先用一次 LLM 调用做"工具选择",再用选中的工具子集做实际调用。

一句话:工具 schema 是 LLM 和外部世界的契约——名字 + 描述 + 类型 + 约束 + 必填,五要素清晰严格,模型才能稳。

Q6. LLM 工具调用失败如何重试和兜底?

工具调用失败是生产 Agent 的常态(网络抖动、API 限流、参数错、上游异常、超时)。设计良好的重试和兜底机制是 Agent 鲁棒性的核心。

失败类型分类(决定不同处理策略):

(1) 模型侧失败:

- 输出 JSON 格式错(缺括号、缺引号、缺字段);
- 工具名错(调用了不存在的工具);
- 参数类型错、必填漏填、值非法(不在 enum 内、超出 range);
- 选错了工具(本该用 A 用了 B)。

(2) 工具侧失败:

- 网络错误:连接超时、DNS、TCP reset;
- 服务端错误:5xx,服务挂了;
- 客户端错误:4xx,权限、找不到资源;
- 限流:429,QPS / quota 超限;
- 业务异常:返回了错误码 + 错误消息;
- 数据异常:返回空、返回非预期格式。

(3) 系统侧失败:

- 超时:LLM 调用超时、整个 Agent 链路超时;
- 资源耗尽:token budget、轮数 budget;

- 沙箱崩溃(对于 code interpreter 等)。

重试策略(分层):

(1) 模型侧 - 让 LLM 自己修复:

- 把错误信息以"tool result"角色回填给 LLM,让它自己改正后重新输出 tool_call;
- 例子:工具收到参数 order_id 缺失 → 回填"参数缺失:order_id 是必填项。请补全。" → LLM 重新生成调用;
- 这是最自然的方式,适合参数错、工具名错、JSON 格式错;
- 限制单工具最大重试 3 次,避免死循环。

(2) 工具侧 - 自动重试(应用层):

- 对幂等 / 可重试错误(超时、5xx、429、网络瞬断)做指数退避 + jitter 重试:
$$\text{delay} = \text{base} * 2^{\text{attempt}} + \text{random_jitter};$$
- 一般 3 次以内;
- 429 限流要按 Retry-After header 等待;
- 不重试的错误(400、403、404):直接报错给 LLM,让它换工具或问用户;
- 用 tenacity (Python)、p-retry (JS)、resilience4j 等成熟库,别自己实现。

(3) 超时控制:

- 每个工具调用必须设硬超时(常见 5-30s,搜索类 30-60s);
- 整个 Agent 链路设全局超时(例如 60s),超过强制结束;
- 超时不无限等,断开后返回"工具超时"给 LLM。

(4) 熔断(Circuit Breaker):

- 某工具连续失败 N 次 → 熔断 X 秒,期间直接拒绝调用,返回"工具暂时不可用";
- 熔断后过冷却期再尝试半开,成功则恢复,失败则继续熔断;
- 保护下游服务,避免雪崩;
- Hystrix / pybreaker / opossum 等库。

兜底策略(降级 + 替代):

(1) 工具降级:

- 主搜索 API 失败 → 用备用搜索源(Google → Bing → DuckDuckGo);
- 实时数据失败 → 用缓存数据 + 标注"数据可能过时";
- 高级模型超时 → 降到普通模型;
- 写操作失败 → 创建延迟任务,稍后重试。

(2) 任务降级:

- 完整方案不可行 → 给部分答案 + 说明哪些没做到;
- 信息检索不到 → 主动承认 "未找到相关信息,可否提供更多线索",而不是编造;
- 复杂任务失败 → 拆解后给已完成部分。

(3) 错误暴露给 LLM:

- 关键原则:不要静默吞错,要让 LLM"看到"错误;
- 错误信息要"人话":不是抛 Python stack,而是"调用 search 失败:连接超时。可重试或换其他工具。";
- LLM 看到错误就能自主决定下一步(重试、换工具、问用户、放弃)。

(4) 人工兜底(Human-in-the-loop):

- 多次失败、高风险操作、用户情绪激烈 → 转人工;
- 提供清晰的"转人工"按钮;
- 转人工时附上对话摘要 + 当前任务状态 + 已尝试的方案,人工无缝接管。

(5) 用户友好提示:

- 不要给用户看技术栈("Error 500: Connection timeout");
- 用业务语言:"暂时无法查询您的订单,请稍后重试或联系客服";
- 提供后续可选行动:重试、查 FAQ、联系人工。

工程实现要点:

(1) 统一工具调用封装:

- 所有工具调用走统一中间层,标准化重试、超时、熔断、日志;
- 单工具的特殊重试策略可配置;

(2) 幂等性:

- 写操作(下单、退款、发邮件)要做幂等设计:同一请求重复执行结果一致;
- 用 `idempotency_key` 或业务唯一键;
- 否则重试可能造成重复下单、重复退款等严重事故;

(3) 防死循环:

- 单工具最大调用次数(防参数始终修不对);
- 总轮数上限(整个 Agent 不能超过 N 步);
- 检测连续相同 `action`,主动打断;

(4) 失败日志与告警:

- 所有失败记录 `trace`,包含 `tool`、参数、错误码、重试次数;
- 失败率超过阈值告警;
- 失败 `case` 沉淀到 `bad case` 集,迭代优化;

(5) 可观测性:

- LangSmith / Langfuse / Helicone 全链路追踪;
- 仪表盘看各工具的成功率、P95 延迟、重试比例;
- 异常分布看哪类错误最频繁。

一个推荐范式(伪代码):

```
def safe_call(tool, args, max_retries=3):
```

```
if tool not in registered_tools:
    return error_for_llm(f"未知工具 {tool}。可选: {list_tools()}")
if circuit_breaker.is_open(tool):
    return error_for_llm(f"{tool} 暂时熔断,请用其他工具")
for attempt in range(max_retries):
    try:
        result = run_with_timeout(tool, args, timeout=tool.timeout)
        if not validate(result):
            return error_for_llm("结果异常: ...")
        return result
    except RateLimit as e:
        sleep(parse_retry_after(e)); continue
    except TransientError:
        sleep(backoff(attempt)); continue
    except PermanentError as e:
        circuit_breaker.record(tool)
        return error_for_llm(f"调用失败: {e}")
return error_for_llm(f"已重试 {max_retries} 次仍失败,请考虑其他方案")
```

黄金原则:失败不可避免,关键是让系统能自动恢复(应用层重试) + 让 LLM 能感知失败并智能应对(暴露错误) + 让用户能得到合理结果或顺畅转人工。

Q7. SSE、WebSocket、WebRTC 在 AI 应用中如何选择?

三者都是浏览器与服务器实时通信的协议,但定位不同。AI 应用(LLM 流式、Agent、语音对话)选哪种,主要看是单向流还是双向、是否需要超低延迟音视频。

SSE(Server-Sent Events):

基于 HTTP 长连接的"服务器单向推送"协议,Content-Type: text/event-stream。

客户端用 EventSource 监听,服务器推送格式化文本流(每条消息以 "data: ..." + 空行结束)。

特性:

- 单向(服务器 → 客户端);
- 基于 HTTP/1.1 或 HTTP/2,标准 HTTP 协议;
- 浏览器原生支持(EventSource API),自动重连;
- 文本流(UTF-8),不适合二进制;
- 穿透代理 / 防火墙友好;
- 实现极简,服务端用 Flask/FastAPI/Express 几行就行。

适合 AI 场景:

- LLM 流式输出(打字机效果):ChatGPT、Claude、几乎所有 LLM Web 应用都用 SSE;
- 长任务进度推送(Agent 执行步骤、训练进度);
- RAG 检索进度;
- 服务端通知 / 消息推送。

OpenAI、Anthropic、Google 等 LLM API 的流式接口都是 SSE。Vercel AI SDK、LangChain Stream 也基于 SSE。

WebSocket:

基于 TCP 的全双工通信协议,通过 HTTP Upgrade 升级而来。

客户端用 `new WebSocket(url)`,服务端长连接保持,双向异步收发文本或二进制。

特性:

- 全双工(双向同时);
- 持久连接,延迟低(无 HTTP 握手开销);
- 支持文本与二进制;
- 主流框架支持(Socket.IO、ws、starlette、FastAPI、Spring WebSocket);
- 防火墙 / 代理 有时不友好(需 80/443 端口或 WSS);



- 没有自动重连(需要应用自己实现);
- 连接管理复杂(心跳、超时、断线重连)。

适合 AI 场景:

- 真正双向频繁交互的场景:用户和 AI 频繁互动的实时编辑器、多人协作 Agent、游戏式对话;
- 需要客户端持续推送(语音流 PCM 上传 + 实时 ASR);
- 长连接对话状态保持 + 双向控制信号;
- 实时音频文本互转(STT/TTS 边录边出);
- 多模态实时输入(摄像头帧持续上传 + AI 实时分析)。

OpenAI Realtime API、Gemini Live、Hume EVI 等"语音对话 API"都是 WebSocket。

WebRTC(Web Real-Time Communication):

浏览器原生的端到端实时音视频协议,基于 UDP(也支持 TCP fallback),延迟最低(可低到 100ms)。

原本设计用于 P2P 视频会议(Zoom、Google Meet 早期),但也支持服务器中转(SFU/MCU)。

特性:

- 端到端低延迟(50-200ms);
- 原生支持音频 / 视频 / 数据通道(DataChannel,双向二进制);
- 内置音频编解码(Opus、G.711)、视频编解码(VP8/9、H.264、AV1)、回声消除、降噪、抖动缓冲;
- NAT 穿透(STUN/TURN);
- 协议复杂,需要信令服务器配合;
- 浏览器原生支持,但服务端实现成本高(SFU 如 mediasoup、LiveKit、Janus)。

适合 AI 场景:

- 端到端低延迟语音对话(用户说话 → AI 回话,要求总延迟 < 500ms);
- 视频通话式 AI(虚拟数字人、AI 教练、医生远程问诊);



- 多人 + AI 的混合会议(AI 实时记录、翻译、辅助);
- AR/VR / 直播场景的 AI 互动。

OpenAI 在 2024 末把 Realtime API 拓展支持 WebRTC,大幅降低语音对话延迟。LiveKit、Daily 等平台提供 WebRTC + AI 集成方案。

对比与选型:

- (1) LLM 文本流式输出 → SSE 是默认最佳选择(简单、稳定、HTTP 兼容);
- (2) 需要客户端频繁向服务端发消息(不是 request-response 模式)→ WebSocket;
- (3) 实时音视频 / 数字人 / 语音对话 → WebRTC;若延迟要求不极致,WebSocket 也行(PCM/Opus 流);
- (4) 单向通知 / 进度更新 → SSE;
- (5) 多人协作 + AI 助理 → WebSocket(Socket.IO);
- (6) 跨防火墙、企业内网严格 → SSE;
- (7) 移动端弱网 / 高丢包 → WebRTC 自带抗丢包,但要做好降级。

工程注意:

- 三者并不互斥,生产应用经常混用:SSE 推 LLM 文本流,WebRTC 跑音视频,WebSocket 处理控制信令;
- 注意网关 / 负载均衡支持长连接(Nginx proxy_buffering off,timeout 设大);
- 重连机制不可缺失(SSE 自带,WebSocket 要手写);
- 鉴权:SSE / WebSocket 用 token in URL 或 cookie,WebRTC 用信令时鉴权;
- HTTP/2 + SSE 在多路复用下性能优秀;HTTP/3 改善长连接稳定性。

一句话:LLM 文本流→SSE,双向实时数据→WebSocket,低延迟音视频→WebRTC。

Q8. LLM 网关要解决什么问题?

LLM Gateway(大模型网关)是位于业务应用和各类 LLM 服务之间的中间层,统一封装请求/响应,提供路由、鉴权、限流、缓存、监控、降级等横切能力。它对 LLM 应用就像 API Gateway 对微服务,是企业级落地的关键基础设施。

一、为什么需要 LLM 网关:

(1) 多模型 / 多供应商:

生产环境同时用 GPT-4、Claude、Gemini、Qwen、DeepSeek、自建模型,各家 API 协议不同(请求格式、流式协议、错误码、参数名)。每个业务方各自适配成本巨大,且不能复用经验。

(2) 切换 / 灰度 / A/B:

换模型要改一堆代码;想做 A/B 测试比较不同模型效果,业务侵入大。

(3) 成本爆炸:

没有统一计量,各业务方乱用模型,月底账单触目惊心。需要精细化计费、配额、报表。

(4) 安全 / 合规:

不同业务对数据出境、敏感词、Prompt Injection 防护的要求不同,在每个业务里重复实现既不一致也容易漏。

(5) 性能:

高 QPS 下要做缓存、限流、负载均衡,业务方做不到。

(6) 可观测性:

需要全链路追踪、日志聚合、监控告警,统一平台才能高效运维。

二、LLM 网关要解决的核心能力:

(1) 统一接入 / 协议归一:

- 对外提供统一 API(通常采用 OpenAI 兼容格式,因为它是事实标准);
- 内部把请求适配到各厂商真实 API;
- 业务方代码不变,后端模型可任意替换。

(2) 路由 / 负载均衡:

- 多个模型、多个 API key、多个 region 自动选择;
- 基于负载、延迟、可用性的智能路由(round-robin / weighted / least-latency);
- 多供应商容灾(主厂商挂了切备用)。

(3) 鉴权与权限:

- 业务方调用网关用业务 token,网关持有真实 LLM API key,业务方接触不到;
- 细粒度权限(谁能用哪些模型);
- 与企业 SSO / IAM 集成;
- 配额管理(每用户每天 token 上限)。

(4) 限流与降级:

- 全局 / 用户级 / 业务线级限流;
- 排队 / 拒绝 / 降级到便宜模型;
- 429 自动重试与熔断;
- 防雪崩 / 防滥用。

(5) 缓存:

- 完全相同 prompt 的精确缓存(Redis,命中率取决于业务);
- 语义缓存(用 embedding 相似度查找近似 prompt,如 GPTCache);
- Prompt Caching 透传(各厂商原生缓存能力);

- 大幅降低成本,常见命中率 10-30%。

(6) 重试与容错:

- 5xx 自动重试;
- 主备模型切换;
- 上游超时自动降级;
- 幂等性保证(用 idempotency key)。

(7) 流式输出处理:

- 统一 SSE / WebSocket 格式;
- 不同厂商流式协议(OpenAI / Anthropic / Google)归一化;
- 流式中断 / 重连。

(8) 成本核算:

- 按模型 / 用户 / 业务线 / 项目 token 统计;
- 月度账单、预算预警;
- 多币种、多模型价格表;
- 成本归因到 tag(支持业务负责人考核)。

(9) 安全防护:

- 输入侧 PII 脱敏 / 屏蔽;
- 敏感词 / 违禁内容拦截;
- Prompt Injection 检测;
- 输出审核(政治、暴力、色情、虚假信息);
- 数据脱敏后再调外部 API;
- 审计日志(谁、何时、调什么、说了什么)。

(10) 可观测性 / 监控:

- 全链路 trace;
- QPS / 延迟 P50/P95/P99 / 错误率;
- token 使用量趋势;
- 模型质量指标(可结合 LLM-judge 离线评估);
- 告警(成功率突降、延迟飙升、token 异常)。

(11) A/B 测试 / 灰度:

- 按比例分流到不同模型;
- 按用户 hash / 按业务标识分流;
- 收集对比效果数据。

(12) Prompt 与配置管理:

- 集中管理 system prompt / few-shot / 工具 schema;
- 版本化 + 灰度发布;
- 改 prompt 不发版。

(13) 高级功能:

- Function Calling 协议归一;
- MCP / Agent 协议接入;
- 模型回路评测(评测同一 prompt 在多模型上的表现);
- 多模态支持(图、音、视频);
- 长上下文 / 嵌入 / Rerank 服务统一暴露。

三、典型架构(分层):

业务应用 ——> LLM 网关(本文焦点) ——> 各 LLM 厂商 / 自建模型

网关内部:

- 接入层:统一 OpenAI 兼容 API、鉴权、限流;

- 路由层:模型选择、负载均衡、降级;
- 中间件:缓存、安全、审计;
- 适配层:各厂商 API 翻译;
- 监控层:trace、metric、log。

四、主流方案:

(1) 开源:

- LiteLLM:Python 包 + Proxy,统一 100+ 模型 OpenAI 兼容 API,功能全;
- One API / New API:Go 写的开源网关,简洁易部署,国内流行;
- Helicone:开源 + SaaS,侧重 observability;
- Portkey:商业 + 开源,功能丰富;
- Higress AI Gateway(阿里):基于 Higress 的 AI 增强;
- Cloudflare AI Gateway、Vercel AI Gateway:云厂商方案。

(2) 商业:

- Kong AI Gateway、Apigee AI、AWS Bedrock Gateway。

(3) 自研:

- 大厂常自研以贴合内部体系。

五、选型与落地建议:

- (1) 小团队 / 单一业务:LiteLLM Proxy 一键起步;
- (2) 中型:LiteLLM + Helicone / Langfuse 组合;
- (3) 大型企业:自研 or 商业方案 + 深度集成 SSO / 财务 / 合规;
- (4) 国内合规:选支持私有部署、不出境的方案(One API、阿里 Higress、自研);
- (5) 强 observability 需求:Helicone / Langfuse / Portkey。

一句话:LLM 网关是企业级 LLM 落地的"水电煤"——没有它,模型再强也用不好、用不省、用不安全。

Q9. 结构化输出如何保证合法?

结构化输出(Structured Output)是让 LLM 输出严格符合预定 JSON / XML / YAML schema 的内容,是工程化应用的关键(数据抽取、工具调用、表单填充、API 返回)。**"看起来像 JSON"**和**"100% 合法 JSON"**之间隔着无数 bug。

一、常见失败模式:

- (1) JSON 缺括号、缺引号、多逗号、字符串内未转义引号;
- (2) 字段缺失(必填字段没填);
- (3) 字段类型错(字符串当 number、布尔写成 "true");
- (4) 枚举值越界、范围超界;
- (5) 数组该有多个元素却只有一个;
- (6) 加了不该有的字段;
- (7) JSON 外面包了一段说明文字(``json + 自然语言前缀);
- (8) 嵌套结构错位;
- (9) 中文场景下的引号问题("..." vs "..." vs '...');
- (10) 流式输出中断、被截断。

二、保证合法的多层手段(从最有效到最弱):

(1) Constrained Decoding(约束解码,最强):

在生成阶段就限制 token 的可选范围,使输出从 token 级别就 100% 满足 schema。

原理:用 schema 编译出有限状态机或语法,每生成一个 token 都过滤掉违反语法的 token。

实现:

- OpenAI Structured Outputs(2024 推出,Strict Mode):JSON Schema 严格保证 100% 合法;
- Anthropic 的 tool_use 输出严格符合 input_schema;

- vLLM / SGLang 自托管模型:支持 outlines、xgrammar、lm-format-enforcer、formatron 等约束解码库;

- llama.cpp 的 grammar(GBNF);

- Outlines / Guidance / Jsonformer:Python 库,可与 transformers / vLLM 集成。

优点:基本不可能错;开销小;

缺点:① 需要推理框架支持;② 极端约束下可能让模型"挤进"不自然 token,降低质量;③ 闭源 API 只有少数支持。

(2) Function Calling / Tool Use(强):

主流 LLM 厂商的 function calling 在服务端做了约束 + 训练,输出可靠性远高于自由文本。

把"想要结构化输出"伪装成"调用一个返回该结构的工具",借用 function calling 的稳健性。

实践:即使不真的执行函数,也用 tool 定义来限定输出 schema。

(3) JSON Mode / Response Format(中强):

- OpenAI 的 response_format: {"type": "json_object"};

- Gemini 的 responseSchema;

- 保证输出是合法 JSON,但不保证符合具体 schema(除非配合 Strict / Schema Mode);

- 比裸 prompt 强很多,弱于 constrained decoding。

(4) Prompt 工程(基础):

- 明确告知输出格式:"只输出 JSON,不要任何前后缀文字、不要 markdown 包裹";

- 提供具体 schema 描述 + 字段含义;

- few-shot 示例(关键!给 2-3 个标准格式样例);

- 强调常见陷阱:"所有字符串用双引号、布尔小写、不要末尾逗号";

- 要求逐字段说明,降低嵌套错乱;

- 简单场景下结合 chain-of-thought:先让模型自然语言分析,再单独输出 JSON 部分(用分隔符隔开)。

(5) 后处理 + 修复:

- 拿到输出后做 `JSON.parse`,失败则进入修复流程;
- 用启发式修复:补缺失括号 / 引号、去除多余逗号、提取 ``json...`` 内文本、去掉前后说明;
- 库:json-repair、json5、demjson、partial-json(支持流式)、dirty-json;
- 修复失败再用 LLM 二次修复:"以下文本应该是合法 JSON 但解析失败,请修复并仅返回 JSON: ...";

(6) Schema 校验 + 重试:

- 用 JSON Schema 校验器(ajv、pydantic、zod、jsonschema)严格校验;
- 不通过 → 把错误信息回传给 LLM,要求修正后重新输出(self-correct loop);
- 限制最大重试 2-3 次,避免成本爆炸;
- Instructor / Marvin / LangChain OutputParser 都内置此流程。

(7) 输出类型选择策略:

- 复杂结构 → 优先 Function Calling;
- 简单字典 → JSON Mode;
- 自托管 / 开源模型 → Outlines / xgrammar;
- 长文本生成不带 JSON → 普通模式 + 末尾追加 JSON 块(用分隔符隔开两段)。

三、字段层面的稳定性技巧:

(1) 字段顺序:JSON 是无序的但 LLM 是按顺序生成的,把"模型容易自信的字段"放前面、依赖前面字段的放后面,质量更稳。例:先生成 reasoning,再 confidence,再 answer。

(2) 用 enum 而非自由 string:能枚举的尽量枚举,降低自由度;

(3) 用 number / boolean 而非 string:类型严格的字段不要全用 string;

(4) 避免过深嵌套:3 层以上拆成多个工具或扁平化;

(5) 默认值:可选字段在 schema 标 default,减少 LLM 漏填困惑;

(6) 字段描述清晰:每个字段加 **description**,包括含义、取值示例、与其他字段关系;

(7) 必填字段越少越好,把可选字段独立成第二个 **schema** 或单独工具调用。

四、流式输出场景:

JSON 在生成完成前是不合法的(只生成了一半),需要:

- 用 **partial-json / streaming-json-parser** 实时解析部分输出;
- 或前端等待完成再 **parse**;
- 或用 **NDJSON**(每行一个对象)分段输出,边出边消费。

五、实践组合:

推荐"约束解码 / Function Calling + Schema 校验 + 修复重试"三件套:

- ① 主力用 **function calling** 或 **Structured Outputs**;
- ② 解析后做 **Pydantic / Zod** 校验;
- ③ 校验失败 → **json-repair** 自动修;
- ④ 修复失败 → **LLM 二次修复**;
- ⑤ 全失败 → 兜底报错给上层。

六、相关库 / 工具:

- Python:Pydantic、Instructor、Outlines、Guidance、LangChain OutputParsers、Marvin、LMQL;
- JavaScript:Zod、Vercel AI SDK structured output、Instructor.js;
- 服务端约束解码:vLLM + outlines / xgrammar、SGLang、TGI、llama.cpp grammar;
- 校验:ajv (JS)、jsonschema (Python)。

一句话:别相信 LLM 自由输出 JSON;在生成阶段用 **constrained decoding / Function Calling** 兜住,生成后用 **Schema 校验 + 自动修复 + 重试**三层保障——只有多层防御才能在生产里达到

99.99% 合法率。

Q10. Prompt Caching 是什么?适合什么场景?

Prompt Caching(提示缓存)是 LLM 推理时的一种优化技术:把 prompt 中重复使用的前缀部分(如长 system prompt、工具描述、文档上下文)在第一次请求时计算 KV Cache 并缓存,后续请求若前缀相同,直接复用 KV Cache,跳过这部分的 prefill 计算,大幅降低成本和延迟。

一、背景:LLM 推理的两阶段

(1) Prefill(预填充 / 第一阶段):一次性处理整个 prompt 的所有 token,计算每层的 K、V 张量,得到第一个输出 token。这部分计算量大,延迟随 prompt 长度线性增长。

(2) Decode(解码 / 第二阶段):一次生成一个 token,每步只算当前 token 的 K、V,前面历史的 K、V 从 cache 读取。

Prefill 阶段在长 prompt 下占了大头延迟和算力。Prompt Caching 优化的就是这部分。

二、原理:

Transformer 的 KV Cache 对于"前缀确定"的 token 来说是可复用的——同一个前缀,无论后面接什么,前缀的 K、V 都不变。

系统识别出请求 prompt 的前缀与历史某次缓存匹配,直接从存储中加载 KV,避免重新算 prefill。匹配通常基于 token 序列哈希(必须从头精确匹配到某个点,中间一字不同就全失效)。

三、各厂商的 Prompt Caching:

(1) Anthropic Claude:

- 显式 API:需要在请求里用 cache_control 标记缓存断点;
- 缓存有效期默认 5 分钟,可选 1 小时;
- 缓存命中读取价格约为正常 input 的 10%;写入缓存比正常 input 贵 25%;
- 多个 cache_control 断点,支持嵌套缓存。

(2) OpenAI:

- 自动缓存(无需显式标记),针对 ≥ 1024 token 的请求自动启用;
- 缓存命中价格为正常 input 的 50%(2024 中)~25%(GPT-4o 2024 末);
- 缓存有效期数分钟到 1 小时,不可配置;
- 必须前缀完全相同,精确到 token。

(3) Google Gemini:

- 显式 Context Caching API,要先创建 cache 拿到 cache_name,后续请求引用;
- 有最低 token 门槛(数千 token 起);
- 按 cache 存储时长 + 命中量计费;
- 适合超长文档反复查询。

(4) DeepSeek、Qwen 等国内厂商:

- 多数已支持自动 prompt caching,价格折扣;
- 具体规则查各家文档。

(5) 自托管(vLLM、SGLang、TensorRT-LLM):

- 都支持 prefix caching / radix attention;
- vLLM 的 Automatic Prefix Caching:LRU 管理,自动命中;
- SGLang 的 RadixAttention:把所有共享前缀组成 trie,跨请求最大化复用。

四、性能收益:

(1) 延迟:命中后 prefill 时间几乎为 0,首 token 延迟(TTFT)可降低 50-90%,尤其长 prompt 场景明显。

(2) 成本:命中部分按打折价计费,典型节省 50-90%。

(3) 吞吐:服务端减少 prefill 计算,同 GPU 可服务更多用户。

五、适合的场景:

(1) 长 system prompt 反复使用:

- 复杂角色设定 / 行为规范 / 工具描述,几千 token,每次请求都带;
- Agent 系统:工具 schema、思考模板、few-shot;
- 命中率极高,典型大幅省钱。

(2) 文档问答 / 长上下文 RAG:

- 把整本手册、整个代码库放在 system 部分,用户每次只问不同问题;
- 文档不变 → 缓存常驻 → 每次只算用户 query 的 prefill。

(3) Few-shot 大量示例:

- 数十个 demonstrations 几千 token,反复用;
- 缓存示例,只算实际 query。

(4) 多轮对话:

- 同一用户的对话历史增量增长;
- 每轮新增的内容只算它本身的 prefill,历史部分缓存复用;
- 长对话场景节省巨大。

(5) 代码助手(Cursor、Continue、Claude Code):

- 整个代码文件 / 项目作为上下文反复传;
- 用户每次只改提问;
- 命中率高、效果显著。

(6) Agent 多步执行:

- ReAct 循环里每一步都包含完整历史,前缀不断扩展;
- 缓存历史前缀,只算新增 thought-action-observation;

- 长任务的延迟和成本显著降低。

(7) 批量评测 / 推理:

- 同一 system prompt + 不同 user query 跑数千个样本;
- 系统部分缓存常驻。

六、不适合 / 收益不大:

- (1) Prompt 高度多样化、前缀无重复(每次 system 都不同);
- (2) Prompt 总长本来就很短(< 500 token),节省有限;
- (3) 间隔时间太长(超出 cache TTL),缓存失效;
- (4) 实时变化的内容放在 prompt 前面(用户名、时间)→ 破坏前缀一致性,缓存全失效。

七、最佳实践:

(1) 把"稳定不变"的内容放前面,"易变"的放后面:

正确:[system prompt] + [工具描述] + [文档] + [对话历史] + [用户当前 query]

错误:把当前时间或随机 id 放最前面 → 缓存永远不命中。

(2) 显式缓存断点(Claude):

在不同稳定层级放 cache_control,system 一层、tools 一层、对话历史一层,即使部分变了也能复用更外层的缓存。

(3) 缓存 warmup:

高峰前先发一次"哑请求"把缓存预热,后续请求命中率高。

(4) 监控命中率:

日志统计 cache_creation_input_tokens、cache_read_input_tokens 比例,优化 prompt 结构提升命中率。

(5) Token 排序优化:

保证每次 prefix 都是同一 token 序列(空格、换行、字段顺序都影响)。

(6) 控制成本敏感场景:

虽然缓存读取便宜,但写入比正常 input 略贵(Claude),低频不重用的 prompt 不必加 cache_control。

八、与其他优化的关系:

- 与 Speculative Decoding 互补(prefill 优化 vs decode 优化);
- 与量化、Flash Attention 互补;
- 与 batch 推理协同:同一 batch 内共享前缀(SGLang 的 RadixAttention 把这做到极致);
- 与 RAG 协同:把检索结果作为缓存层,可大幅降低检索 + 推理总延迟。

九、面试关键点:

Prompt Caching = 复用 KV Cache 跳过重复 prefill,显著降本提速,适合"前缀稳定、反复调用"的场景(长 system prompt、文档问答、多轮对话、Agent、代码助手)。设计时把易变内容放尾部、稳定内容放前部是关键技巧。

第 6 章 推理部署、性能优化与系统设计

高频面试题详解 · Q1 ~ Q10

Q1. KV Cache 是什么?为什么能加速推理?

KV Cache(Key-Value Cache)是 Transformer 自回归解码中的关键优化:把每一层 self-attention 计算出的 Key 和 Value 张量缓存下来,后续生成新 token 时直接复用,而不是每步都从头重算整个序列的 K、V。

一、为什么需要 KV Cache:

Transformer 自回归生成第 t 个 token 时,需要计算它对前 $1..t-1$ 个 token 的注意力:

$$\text{Attention}(Q_t, K_{\{1:t\}}, V_{\{1:t\}}) = \text{softmax}(Q_t \cdot K_{\{1:t\}}^T / \sqrt{d}) \cdot V_{\{1:t\}}$$

如果没有缓存,每生成一个新 token 都要把前面所有 token 都重新过一遍 Transformer,得到它们的 K 和 V——计算量随序列长度平方增长, $O(n^2 \cdot d)$ 。

观察:前面 token 的 K、V 是不变的(它们的 hidden state 不依赖后面的 token,因为有 causal mask)。所以可以"算过一次就缓存,以后只算新增 token 的 K、V"。

二、KV Cache 工作流程:

Prefill 阶段(prompt 处理):

一次性把整个 prompt 过 Transformer,每层算出 K、V 张量,形状 $[\text{batch}, \text{seq_len}, n_kv_head, \text{head_dim}]$,保存为 cache。

Decode 阶段(自回归生成):

每生成一个新 token,只算它自己的 Q、K、V(各 1 个 token)。

把新的 K、V 拼接到 cache 末尾(cache 长度 + 1)。

用新 Q 与 cache 中的全部 K 算注意力。

复杂度:每步 $O(t \cdot d)$,而非 $O(t^2 \cdot d)$ 。

三、加速原理:

(1) 计算量减少:

没 cache:生成 n 个 token 总计算 $O(n^3 \cdot d)$ (每步 $O(n^2 \cdot d)$,共 n 步);

有 cache: $O(n^2 \cdot d)$ total。差距是序列长度的一个数量级以上。

(2) 内存访问优势:

GPU 的瓶颈通常是显存带宽。读取已缓存的 K、V 比重新计算便宜得多;

现代 LLM 推理通常是 memory-bound,KV cache 命中显著提升吞吐。

(3) 实测延迟:

生成第 100 个 token,没 cache 要重新跑 99 个 token + 当前 1 个;有 cache 只算 1 个 token 的 forward。Decode 阶段每步几乎只与模型大小相关,与已生成长度弱相关。

四、KV Cache 的代价:显存占用

单个 token 在每层的 KV cache 大小: $2(K+V) \times n_{kv_head} \times head_dim \times dtype_size$ 字节。

总大小: $2 \times n_{layer} \times n_{kv_head} \times head_dim \times seq_len \times batch \times dtype$ 。

举例 LLaMA-3 70B(64 层、64 头、128 head_dim,FP16):

单 token KV = $2 \times 64 \times 64 \times 128 \times 2 = 2$ MB;

8K 上下文、batch=1 $\rightarrow$ 16 GB;batch=16 $\rightarrow$ 256 GB。

这就是为什么长上下文 + 大 batch 在 70B 上极吃显存,GQA/MQA、量化、PagedAttention 等都是为了优化 KV cache。

五、围绕 KV Cache 的优化技术:

- (1) GQA / MQA:减少 KV head 数,直接缩小 cache 大小(LLaMA-3 70B 用 GQA,KV head 只有 8 个);
- (2) PagedAttention(vLLM):把 KV cache 分页存储(类似操作系统虚拟内存),避免显存碎片化,提升 batch 吞吐 2-4 倍;
- (3) Prefix Caching:跨请求复用相同前缀的 KV cache(SGLang RadixAttention、vLLM Automatic Prefix Caching);
- (4) KV Cache 量化:把 KV 从 FP16 量化到 INT8 / INT4,显存减半甚至 1/4(KIVI、KVQuant);
- (5) KV Cache 卸载到 CPU / SSD:超长上下文(>100K)用,延迟略增但能跑;
- (6) Sliding Window / Sink Attention:只保留最近 N 个 token 的 KV,适合超长文本;
- (7) MLA(Multi-head Latent Attention,DeepSeek-V2):把 KV 压缩到一个低维 latent,显著降低 cache 大小;
- (8) Streaming / Incremental:边输入边算 prefill,降低首 token 延迟。

六、面试关键点:

- 没 KV Cache:解码复杂度 $O(n^2 \cdot d)$ per step,总 $O(n^3 \cdot d)$;
- 有 KV Cache:解码每步 $O(n \cdot d)$,总 $O(n^2 \cdot d)$;
- KV cache 是显存大户,显存优化几乎都围绕它;
- 现代推理框架(vLLM、SGLang、TensorRT-LLM)的核心创新都在 KV cache 管理。

Q2. Flash Attention 为什么快?

Flash Attention(Dao et al. 2022)是一种 IO-aware 的精确注意力算法,通过分块计算 + 算子融合 + 减少 HBM 读写,在不改变数值结果的前提下显著加速 attention,且大幅降低显存占用。已成为现代 LLM 训练 / 推理的标配。

一、传统 Attention 的瓶颈:

标准注意力计算流程:

- (1) $S = Q \cdot K^T$, 形状 $[N, N]$;
- (2) $P = \text{softmax}(S)$, 形状 $[N, N]$;
- (3) $O = P \cdot V$, 形状 $[N, d]$ 。

GPU 内存层级:

- HBM(High Bandwidth Memory,GPU 显存):大(几十 GB),但慢($\sim 1-2$ TB/s);
- SRAM(片上 shared memory):极小(每 SM 几十到几百 KB),但极快(~ 20 TB/s)。

传统实现的问题:

- 中间矩阵 S 和 P 的大小是 N^2 , $N=4K$ 时就有 16M 个元素 ≈ 32 MB(FP16), $N=32K$ 是 2 GB;
- 这些中间矩阵都被写回 HBM 再读取,每次都是巨大 IO;
- GPU 算力远超带宽,实际计算只占小部分时间,大部分时间在等内存。

所以 attention 是典型的 memory-bound,提升的关键不在 FLOPs,在减少 HBM 访问。

二、Flash Attention 的核心思想:

(1) Tiling(分块):

把 Q 、 K 、 V 切成小块(tile),每块装入 SRAM 计算完整的注意力分块,不再把巨大的 S 、 P 中间矩阵写回 HBM。

具体地:外层循环遍历 K 、 V 块,内层循环遍历 Q 块,每对 (Q_i, K_j, V_j) 块在 SRAM 中完成局部 attention,把结果累加到输出 O 中。

(2) Online Softmax:

Softmax 通常需要先扫一遍求 max 和 sum,再归一化——天然不能分块。

Flash Attention 用 online softmax(累积统计量)的数学技巧:每处理一个块,维护当前最大值 m 和归一化分母 ℓ ,新块到来时按公式融合到全局,而无需保留完整的 S 矩阵。

关键公式: $m_{\text{new}} = \max(m_{\text{old}}, m_{\text{tile}})$; $\ell_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) \cdot \ell_{\text{old}} + \exp(m_{\text{tile}} - m_{\text{new}}) \cdot \text{sum_tile}$; $O_{\text{new}} = (\exp(m_{\text{old}} - m_{\text{new}}) \cdot \ell_{\text{old}} \cdot O_{\text{old}} + \exp(m_{\text{tile}} - m_{\text{new}}) \cdot O_{\text{tile_unnorm}}) / \ell_{\text{new}}$ 。

数学上完全等价标准 softmax,但流式 + 可分块。

(3) Kernel Fusion(算子融合):

把 matmul、softmax、masking、dropout 全部融合在一个 CUDA kernel 里,中间张量不出 SRAM,极致减少 HBM 读写。

(4) Recomputation in Backward(反向重算):

反向传播需要 forward 中的中间结果。Flash Attention 不存 S 和 P (它们太大),而是在 backward 时重新计算——以"少量额外 FLOPs"换"少量 HBM 占用",净收益巨大。

三、加速效果:

- (1) 速度:相比 PyTorch 标准 attention, F1 加速 2-4 倍, F2/F3 进一步;
- (2) 显存:中间张量从 $O(N^2)$ 降到 $O(N)$,长序列省显存极多;
- (3) 长上下文:之前 $N=8K$ 就吃力, F1 之后训练 32K、128K 成为可能;
- (4) 训练 + 推理 都受益, LLaMA、Qwen、GPT 类模型实际部署都用。

四、Flash Attention 的版本演进:

FlashAttention-1(2022):提出 IO-aware + tiling + online softmax,达到 2-3 倍加速。

FlashAttention-2(2023):优化工作分配、减少非 matmul FLOPs、更好的并行化,在 A100 上比 F1 快 2 倍。

FlashAttention-3(2024):针对 Hopper (H100) 的 TMA、WGMMA、async copy 等特性深度优化,接近硬件理论峰值(740 TFLOPS BF16),比 F2 快 1.5-2 倍。

FlashDecoding(2023):针对推理 decode 阶段(seq_len=1,KV 长)做专门优化,把 KV 维度并行化,显著降低 decode 延迟。

FlashDecoding++(2023):进一步优化 attention sink、长 KV 场景。

五、与其他注意力优化的关系:

- (1) Flash Attention 是精确算法(数值与标准 attention 完全相同),不是近似;
- (2) 区别于稀疏 / 线性注意力(改变算法本身,有数值损失);
- (3) 与量化、KV cache 优化、GQA/MQA 互补,可叠加;
- (4) Triton、xFormers、PyTorch SDPA(2.0+ 默认调用 Flash Attention)、vLLM、SGLang 都内置。

六、使用注意:

- GPU 必须支持(A100、H100 / 4090 等较新架构,V100 上 F1 可用,F2 不行);
- 序列长度越长收益越大;短序列($N < 512$)收益有限;
- 注意 dtype:推荐 BF16 / FP16;
- causal mask 是原生支持的;复杂 mask 需要变种支持(Flex Attention);
- PyTorch ≥ 2.0 调 `nn.functional.scaled_dot_product_attention` 会自动选 Flash Attention(若可用)。

七、一句话总结:

Flash Attention 之所以快,是因为它意识到 attention 是 memory-bound,通过 tiling + online softmax + 算子融合,让中间矩阵不再落地 HBM,把内存访问从 $O(N^2)$ 降到 $O(N)$,并通过反向重算

节省显存——同样的数学结果,显著低的延迟和显存。

Q3. 量化有哪些方法?INT8、INT4、GPTQ、AWQ 如何取舍?

量化(Quantization)是把 LLM 权重 / 激活值从高精度(FP32 / FP16 / BF16)降低到低精度(INT8 / INT4 / FP8 等)表示,以显著降低显存占用、提升推理速度、减小模型体积——是部署 LLM 的核心优化手段。

一、量化基本概念:

(1) 量化对象:

- 权重量化(Weight-only Quantization):只压缩权重,激活值保持高精度。最常用,推理友好;
- 激活量化(Activation Quantization):同时压缩激活值,进一步加速但更难做对;
- KV Cache 量化:压缩 KV Cache,缓解长上下文显存;
- W4A16:权重 INT4,激活 FP16(主流权重量化);
- W8A8:权重 + 激活都 INT8(适合服务端);
- W4A4 / FP8:极致压缩。

(2) 量化粒度:

- per-tensor:整个张量一个 scale;
- per-channel / per-row:每一行/列一个 scale;
- per-group:每 32/64/128 个元素一组,各自 scale;粒度越细误差越小,存储略增。

(3) 对称 vs 非对称:

- 对称:零点 = 0,简单快速;
- 非对称:有 zero point,适合分布偏移的激活值。

(4) 静态 vs 动态:

- 静态:用校准集预先确定 scale;

- 动态:运行时按当前张量动态计算 scale。

二、主流量化方法:

(1) INT8(W8A8):

- 朴素 INT8:权重和激活都用 8 bit;
- LLM.int8()(Dettmers 2022):识别异常激活值用 FP16 算,其他用 INT8;
- SmoothQuant(2022):通过等价变换把激活 outliers"迁移"到权重侧,使激活更易量化;
- 显存约为 FP16 的一半;质量损失通常 < 0.5%;主流推理后端原生支持(TensorRT、vLLM)。

(2) INT4(W4A16,权重 4 bit、激活 16 bit):

- 显存约为 FP16 的 1/4,7B 模型可在 6GB 显存跑;
- 质量损失依赖方法和组别大小,典型 1-3% perplexity 上升;
- 4 bit 量化算法很多,主要有 GPTQ、AWQ、SmoothQuant+、SqueezeLLM、QuIP、HQQ 等。

(3) GPTQ(Frantar et al. 2022,GPT Quantization):

- 基于 OBP(Optimal Brain Quantization)的逐层量化算法;
- 思路:量化一层时,逐列处理权重,把量化引入的误差"补偿"到尚未量化的剩余权重上,使整体输出误差最小;
- 需要少量校准数据(几十到几百条样本);
- 后训练量化(PTQ,无需重训);
- 4 bit 下质量损失小,通常 <1% perplexity 上升;
- 推理端 GPTQ kernel 优化好,速度快(GPTQ-for-LLaMA、ExLlamaV2、AutoGPTQ);
- 缺点:对激活异常值不敏感处理,在某些场景下表现不如 AWQ;校准数据分布影响结果。

(4) AWQ(Activation-aware Weight Quantization,Lin et al. 2023):

- 核心洞察:不是所有权重同等重要——对应"大激活"的权重对最终输出影响大,应该被保护;
- 做法:用校准数据看每个权重对应的激活分布,识别"重要"权重,通过 per-channel scaling 把它们

的精度损失降到最低;

- 不需要反向传播,纯前向校准;
- 4 bit AWQ 在指令跟随、推理任务上常优于 GPTQ;
- 工具:llm-awq、AutoAWQ;
- vLLM、TensorRT-LLM 等推理框架原生支持。

(5) GGUF / GGML(llama.cpp 系):

- 包含多种量化变体:Q4_0、Q4_K_M、Q5_K_M、Q8_0、IQ2 等;
- K-quants:把权重分组,组内 + 块内多级量化;
- 强项:CPU 推理友好、内存映射、支持混合精度(重要层用高精度)、文件格式统一;
- 缺点:GPU 性能不如专业 GPU 量化方案。

(6) FP8(E4M3 / E5M2):

- H100 / 4090 等新硬件原生支持;
- 比 INT8 动态范围更大,适合训练 + 推理一体化;
- NVIDIA TransformerEngine、TensorRT-LLM 支持;
- 损失通常小于 INT8。

(7) 其他:

- BitsAndBytes(QLoRA 用):NF4 / FP4,微调时常用;
- SpQR、HQQ、QuIP#、OmniQuant、SqueezeLLM:学术与新方法,各有特色;
- INT3、INT2:极致压缩,质量损失明显,目前主要用于研究。

三、如何取舍(决策框架):

(1) 按显存预算:

- 显存富裕(A100 80G / H100):可不量化或 FP8 / INT8 求速度;
- 中等(A100 40G / 4090 24G):INT8 / W4A16 平衡;

- 紧张(消费级 16/12/8 GB):INT4(AWQ / GPTQ);
- 极致(端侧 4-8 GB):INT4 K-quants 或更激进。

(2) 按场景:

- 服务端高吞吐:FP8 或 INT8,vLLM / TensorRT-LLM;
- 单卡部署 13-70B:W4A16(AWQ / GPTQ);
- 边缘 / CPU:llama.cpp + GGUF Q4/Q5;
- 微调:QLoRA(NF4) / 标准 LoRA(BF16);
- 学术评测:不量化 / FP16 作 baseline。

(3) GPTQ vs AWQ 对比:

维度 / GPTQ / AWQ:

- 算法基础:误差补偿(OBQ)/ 激活感知缩放;
- 校准:需要 / 需要;
- 后训练:✓/✓;
- 速度:很快 / 很快(都有专门 kernel);
- 显存:相当;
- 质量:在多数 benchmark 上相当,AWQ 在指令任务略胜;
- 框架支持:更广(早期普及)/ vLLM / TGI / TRT-LLM 一线支持;
- 鲁棒性:对校准数据更敏感 / 相对稳健。

工程建议:两者都试,看自己业务评测谁更好;现代趋势 AWQ 略占上风。

(4) INT8 vs INT4:

- INT8:质量几乎无损,显存减半,服务端推理多用;
- INT4:显存减到 1/4,有少量质量损失(对推理 / 数学任务影响略大),消费级 / 单卡部署用;
- INT8 + 异常值保护(LLM.int8、SmoothQuant) > 朴素 INT8;
- INT4 + 现代方法(AWQ / GPTQ + group_size=128)接近 INT8 质量。

(5) 评估:

量化后必须评测:

- Perplexity:基础指标;
- Benchmark:MMLU、GSM8K、HumanEval 等,量化对推理 / 代码任务损失更明显;
- 业务指标:RAG、Function Calling 准确率;
- 长尾质量:幻觉率、复杂指令跟随;
- 推理速度 / 吞吐:tokens/s、显存峰值、首 token 延迟。

四、推理框架兼容性:

- vLLM:AWQ、GPTQ、FP8、Marlin、INT8;
- TensorRT-LLM:FP8、INT8、INT4(AWQ);
- llama.cpp:GGUF 全套;
- TGI (HuggingFace):AWQ、GPTQ、BnB;
- SGLang:AWQ、GPTQ、FP8;
- ExLlamaV2:GPTQ、自家 EXL2。

五、一句话总结:

服务端高吞吐:INT8 / FP8;

单卡部署中大模型:W4A16(AWQ 或 GPTQ);

消费级 / 端侧:GGUF Q4_K_M;

微调:QLoRA NF4;

量化是工程必修课,选型靠"显存预算 + 场景 + 自家评测"三要素决定。

Q4. 如何加速 LLM 推理?

LLM 推理加速是工程的核心问题,涉及算法、系统、硬件、模型多个层面。可以分两大场景:首 token 延迟(TTFT,prefill 主导)和吞吐量(tokens/s,decode 主导),不同优化点不同。

一、推理两阶段回顾:

- Prefill: 一次性处理 prompt, 计算密集型(compute-bound), 决定 TTFT;
- Decode: 逐 token 生成, 内存访问密集型(memory-bound), 决定吞吐 + 总延迟。

二、加速技术(按层次):

(1) 模型层 - 减少参数 / 计算量:

① 量化(INT8 / INT4 / FP8): 减少显存与计算, 2-4 倍速度提升, 见上一题;

② 剪枝(Pruning): 结构化(整列 / 整通道) + 非结构化稀疏。LLM 上效果有限, 推理端硬件支持不好;

③ 知识蒸馏(Distillation): 用大模型教小模型, 部署小模型; 典型如 DistilBERT、Qwen-distill、DeepSeek-R1-Distill;

④ 模型架构优化:

- GQA / MQA: 减小 KV cache (LLaMA-3 70B、Qwen);
- MLA (Multi-head Latent Attention, DeepSeek-V2): KV cache 压缩到低维 latent;
- MoE (Mixture of Experts): 激活参数远小于总参数 (Mixtral 47B 但激活 13B、DeepSeek-V3 671B 激活 37B);
- 线性 / 状态空间模型 (Mamba、RWKV): 突破 attention 的 $O(n^2)$;

(2) Attention 算法层:

① Flash Attention / Flash Attention 2/3: IO-aware, 减少 HBM 读写, 2-4 倍加速, 见上一题;

② Flash Decoding: 针对 decode 阶段优化, 长 KV 场景显著加速;

③ Paged Attention (vLLM): KV cache 分页存储, 显存碎片化降低, batch 吞吐 2-4 倍;

④ Radix Attention(SGLang):跨请求共享前缀的 KV cache,极致提升前缀复用率;

⑤ Sliding Window / Sink Attention:超长上下文只看近期 + 起始 token;

(3) KV Cache 层:

① Prompt Caching:复用相同前缀的 KV cache,prefill 几乎免费;

② KV cache 量化(INT8 / INT4):显存减半,长上下文友好(KIVI、KVQuant);

③ KV cache 卸载:超长上下文部分 KV 放 CPU / SSD;

④ KV cache 压缩 / 驱逐(H2O、StreamingLLM):动态丢弃不重要的 KV。

(4) 解码策略层:

① Speculative Decoding(投机解码):

- 用小模型(draft model)一次生成多个候选 token,大模型并行验证;
- 接受 ≥ 1 个就赚到(原 decode 是逐 token);
- 2-3 倍加速,数学正确(输出分布与大模型完全相同);
- 变种:Medusa(多头自我投机)、EAGLE、Lookahead、tree-attention 等;

② Parallel Sampling / Beam Search 优化:批量内并行;

③ 早停(Early Stopping):达到结束条件提前结束;

④ Constrained Decoding(JSON Schema):减少无效搜索;

(5) 系统 / Batch 层:

① Continuous Batching(连续批处理 / Dynamic Batching,vLLM 核心):

- 不等同批所有请求完成,某请求结束立即把新请求填进 batch 槽;
- 大幅提升 GPU 利用率;
- 比静态 batch 吞吐高 2-10 倍。

② Prefill / Decode 分离(Chunked Prefill 或 Disaggregated):

- Prefill 和 decode 计算特性差异大,放一起会相互拖累;
- 拆开调度可提升整体吞吐(DeepSpeed-FastGen、SplitWise);

③ Chunked Prefill:把长 prompt 的 prefill 分块插入 decode 间隙,降低尾延迟;

④ Speculative Streaming / Pipeline Parallel:多 GPU 流水线;

⑤ Tensor Parallel(张量并行):大模型多卡拆分,降低单卡延迟;

⑥ Expert Parallel(MoE 专家并行)。

(6) 硬件 / 编译层:

① CUDA Kernel 优化:Triton、CUTLASS 手写算子;

② 算子融合(Operator Fusion):减少 kernel launch + HBM 访问;

③ FP8 / Tensor Core 利用:H100 / 4090 上原生 FP8;

④ TensorRT-LLM / TRT 编译:深度图优化、kernel 自动选择;

⑤ AOT 编译:torch.compile、TVM、IREE;

⑥ 专用硬件:Groq LPU(超低延迟)、Cerebras WSE、SambaNova、华为昇腾、Google TPU。

(7) 工程 / 业务层:

① 缓存:精确 prompt 缓存 + 语义缓存(GPTCache);

② 短路:简单问题用规则 / 小模型直接答,不调大模型;

- ③ Prompt 简化:压缩 prompt(LLMLingua、Selective-Context);
- ④ 模型分级路由:简单 query → 小模型,复杂 query → 大模型(LangChain RouterChain);
- ⑤ 异步生成 + 流式返回:用户感知延迟降低(看到字就觉得快);
- ⑥ 预热与连接池:避免冷启动。

(8) 模型选型:

同样任务,小模型 + 微调可能优于大模型 zero-shot;选小而精的模型是最大优化。

三、综合实践:

高吞吐服务端推荐组合:

- 框架:vLLM / SGLang / TensorRT-LLM;
- 模型:用 GQA / MQA 的 base + AWQ INT4 量化(或 FP8);
- 注意力:Flash Attention 2/3;
- 调度:Continuous Batching + Chunked Prefill;
- 缓存:Prefix Caching;
- 解码:Speculative Decoding(可选,2-3x);
- 业务:Prompt 缓存 + 模型分级路由。

低延迟单请求推荐:

- 模型:小或蒸馏版;
- 推理:TensorRT-LLM 或 vLLM;
- Speculative Decoding;
- FP8 / INT8;
- 流式输出降低感知延迟。

四、性能指标:

- TTFT(Time To First Token):首 token 延迟,关键体验指标;

- TPOT(Time Per Output Token):每 token 时间;
- Throughput:tokens/s(吞吐),关键成本指标;
- 端到端延迟、并发数、QPS、P95/P99;
- 显存峰值。

五、面试要点:

记住"prefill 看计算,decode 看带宽" → 不同优化策略;

了解 vLLM 三大法宝:PagedAttention + Continuous Batching + Prefix Caching;

熟悉 Flash Attention、Speculative Decoding、量化、MoE 这几个高频概念;

能针对场景给出综合方案,而不是堆名词。

Q5. 端侧如何部署 7B 模型?

端侧部署(手机、PC、嵌入式设备)对 LLM 来说挑战巨大:显存少、算力低、能耗严格、内存带宽受限。7B 模型(原始 FP16 约 14 GB)要在 8-16 GB 内存设备上跑起来,需要量化 + 高效推理框架 + 模型选型多管齐下。

一、端侧约束:

- 内存 RAM:手机 4-16 GB,PC 8-32 GB;
- 算力:手机 GPU / NPU,PC iGPU / 消费级 GPU(M 系列芯片、4060 等);
- 带宽:消费级硬件远低于服务端(M2 Max ~400 GB/s vs A100 ~2 TB/s);
- 功耗 + 散热:持续推理对电池 / 散热不友好;
- 启动时间:用户期待秒级响应,不能预热几十秒;
- 模型文件大小:下载体验。

二、关键优化手段:

(1) 量化是第一关键:

- 必须用 4-bit 量化:7B FP16 14GB → INT4 ~4 GB,才能装进手机内存;

- 推荐:GGUF Q4_K_M(质量与体积平衡好),或 AWQ INT4;
- 极端:Q3_K_M(~3 GB)、IQ2_XXS(~2 GB)用于内存特小设备,质量损失明显;
- KV cache 也量化(INT8 / INT4),长对话场景必要。

(2) 推理框架:

① llama.cpp:

- 端侧推理事实标准,纯 C/C++,跨平台(iOS / Android / Windows / Mac / Linux);
- 支持 GGUF 格式,CPU + GPU + Metal + Vulkan + CUDA;
- mmap 加载,启动快;
- 内存高效,可流式输出;
- 生态丰富(Ollama、LM Studio、Jan、GPT4All 等基于它);

② MLC LLM(Machine Learning Compilation):

- 基于 TVM,把模型编译成各平台原生 kernel;
- 支持 iOS / Android / WebGPU / Vulkan / CUDA / Metal;
- 浏览器端 LLM(WebLLM)的核心;

③ MNN(阿里) / TNN(腾讯) / NCNN(腾讯) / Paddle Lite:

- 国产端侧推理引擎,移动端深度优化;
- 部分支持 LLM;

④ ExecuTorch(PyTorch 移动端):

- PyTorch 官方端侧方案,2024 起逐渐成熟;

⑤ Apple MLX:

- Apple 出品,Mac / iOS 上为 M 系列芯片深度优化;
- 利用 Unified Memory,内存即显存,大模型部署友好;

⑥ Qualcomm AI Hub / Genie SDK、MediaTek Neuron SDK:

- 手机芯片厂商的 NPU SDK,跑量化模型加速明显;

⑦ ONNX Runtime Mobile:

- 跨平台,适合需要复用 ONNX 生态。

(3) 硬件加速:

- 优先用 GPU / NPU:手机 NPU(高通 Hexagon、Apple Neural Engine、华为达芬奇)对量化模型有专门加速,远快于 CPU;
- Mac M 系列:Unified Memory + Metal,16GB MacBook 跑 7B 极顺;
- Windows PC:NPU(Intel AI Boost、AMD Ryzen AI、Snapdragon X Elite)逐渐普及;
- 浏览器:WebGPU(Chrome、Edge、Safari 部分支持)。

(4) 模型选型:

- 优先选"小而强"的现代模型,而非朴素的早期 7B;
- 推荐:Qwen2.5-7B-Instruct、Llama-3.1-8B、Phi-3.5-mini(3.8B,质量接近 7B)、Gemma-2-9B、Mistral-7B、MiniCPM 系列(国产端侧专用);
- 端侧优化版:Phi-3.5-mini、MiniCPM-V、Qwen2-VL-2B;
- 蒸馏小模型:DeepSeek-R1-Distill-Qwen-7B、Qwen-distill 等;
- 多模态端侧:Phi-3.5-vision、MiniCPM-V 系列。

(5) 上下文与 KV Cache:

- 端侧 KV cache 是显存大户(7B 模型 8K 上下文 KV cache 约 4GB FP16,量化到 INT4 ~1GB);
- 限制最大上下文(2K / 4K 通常足够端侧场景);
- 用 Sliding Window 或 KV cache 量化;
- 长对话用对话摘要替代全保留。

(6) 启动与加载优化:

- mmap 映射模型文件:不一次性加载到内存,按需读取;

- 模型缓存目录提前预热;
- 后台进程常驻 vs 按需启动权衡;
- 模型分块预下载 + 解压。

(7) 推理参数调优:

- thread 数:CPU 推理时设 = 物理核数(避免超线程);
- batch size:端侧通常 1,无需 batch 优化;
- 短输出:对话场景设 max_tokens 合理;
- 流式:首 token 出来后逐字显示,提升体验。

(8) 系统集成:

- 与 OS / APP 沙箱集成(iOS App Group、Android Foreground Service);
- 权限管理(后台运行、电量);
- 离线工作能力(不连网也能用,核心优势);
- 隐私:数据不出端,符合合规;
- 离线知识库可结合本地 RAG(SQLite + embedding)。

(9) 多模态考虑:

- 端侧 VLM(MiniCPM-V、Phi-3.5-vision、Qwen2-VL):图像编码 + 文本生成;
- 注意视觉 encoder 的额外开销;
- 端侧 STT(Whisper.cpp)、TTS(Piper、Coqui)配合,实现完整对话。

三、典型部署链路:

手机示例(Android,7B 模型):

- ① 选模型:Qwen2.5-7B-Instruct;
- ② 量化:GGUF Q4_K_M(~4.4 GB)或厂商 NPU 专用格式;
- ③ 推理:llama.cpp(JNI 调用)或 MLC LLM(LLVM 编译到 ARM);

- ④ GPU 加速:Vulkan / OpenCL,或厂商 NPU(QNN Genie);
- ⑤ KV cache:INT8 量化,上下文限 4K;
- ⑥ UI:Jetpack Compose / SwiftUI,流式显示;
- ⑦ 性能预期:首 token 1-3s,后续 5-15 tokens/s(取决于设备)。

PC 示例(MacBook 16GB,M2):

- ① Ollama 或 LM Studio 一键拉模型;
- ② llama.cpp + Metal 加速;
- ③ 7B Q4 → 4.5 GB 文件,推理时占用 5-6 GB 内存;
- ④ 性能预期:20-40 tokens/s,体验流畅。

四、端侧的局限与未来:

- 7B 量化后能力仍弱于云端大模型,复杂任务效果差;
- 长上下文受限;
- 适合:离线问答、本地写作、隐私场景、简单对话、本地 Agent 基础;
- 未来趋势:芯片厂商 NPU 持续升级、模型小型化(SmolLM、Gemma-3n)、混合云端架构(简单本地、复杂上云)。

五、一句话总结:

端侧 7B = INT4 量化 + llama.cpp / MLC / MLX + 合适小模型 + NPU 加速 + 受限上下文。核心是"内存 + 带宽 + 算力"的平衡,llama.cpp + GGUF Q4_K_M 是最广适的入门组合。

Q6. 如何设计亿级用户的聊天机器人系统?

亿级用户的聊天机器人是典型的高并发、超大规模 LLM 服务系统,涉及前端、网关、推理、数据、运维、安全全栈设计。下面给出生产级架构与关键决策。

一、需求与挑战:

- 峰值 QPS:百万级并发会话,日活上亿;



- 推理成本:7B-70B 模型,token 成本是核心;
- 延迟要求:首 token < 1s,流式;
- 高可用:99.99% SLA,跨区容灾;
- 多模态:文本 / 图片 / 语音;
- 个性化:记忆、上下文、用户偏好;
- 安全合规:内容审核、PII、监管要求;
- 国际化:多语言、多区域。

二、整体架构(分层):

层 1 · 接入层:

- CDN + 边缘节点:静态资源、地理就近;
- API 网关:鉴权、限流、SSL、协议转换;
- WebSocket / SSE:支持流式输出;
- 客户端 SDK:iOS / Android / Web / 桌面;
- DNS 智能解析:多区域调度。

层 2 · 业务层:

- 用户服务、会话服务、消息服务、订阅服务(分布式微服务);
- 会话状态管理:用户当前对话上下文、活跃 session 数;
- 消息路由:同一用户的请求尽量打到同一推理节点(亲和性,提升 KV cache 命中);
- 鉴权中心:OAuth / JWT,多端登录;
- 服务发现 / 注册:Consul / Nacos / Kubernetes service。

层 3 · LLM 网关层(关键):

- 统一对接多模型(自建 + 外购);
- 智能路由:按用户层级、任务复杂度、成本分发到不同模型;
- 限流 / 配额:全局 + 用户级;

- 缓存:精确 + 语义(GPTCache);
- 安全过滤:输入 / 输出;
- 计费统计:token、用户、业务线维度;
- A/B 灰度 / 模型切换。

层 4 · 推理层(核心):

- 推理框架:vLLM / SGLang / TensorRT-LLM;
- GPU 集群:H100 / A100 / 国产卡;
- 容器化:Kubernetes + KServe / Triton;
- 自动扩缩容(HPA 按 QPS / GPU 利用率);
- 模型多版本并存,流量分配;
- 跨可用区部署,故障转移;
- Continuous Batching + Prefix Caching;
- 量化模型部署(INT8 / FP8 / INT4);
- 模型分级:大模型(70B+) + 中模型(13-30B) + 小模型(7B) + 蒸馏模型(<3B)。

层 5 · 数据层:

- 用户数据:MySQL / PostgreSQL(分库分表);
- 对话历史:HBase / TiKV / Cassandra(海量写入);
- 会话状态 / 缓存:Redis 集群(session、限流计数器);
- 向量库:Milvus / Qdrant(用户记忆、知识库);
- 对象存储:OSS / S3(图片、语音、文件);
- 数据湖:Hive / Iceberg(日志、训练数据);
- 消息队列:Kafka / Pulsar(异步流转、训练数据回流);
- 搜索:Elasticsearch(全文检索)。

层 6 · 周边能力:

- RAG:全文 + 向量混合检索;

- 工具调用:Function Calling 工具集;
- 多模态:VLM、ASR、TTS、图像生成服务;
- 个性化记忆:用户偏好、长期记忆向量化存储;
- 知识库:产品 FAQ、客服文档;
- 外部 API:天气、地图、新闻。

层 7 · 可观测性 / 运维:

- 日志:ELK / Loki;
- 指标:Prometheus + Grafana;
- 链路追踪:Jaeger / OpenTelemetry;
- LLM 专属:LangSmith / Langfuse / Helicone;
- 告警:成功率、延迟、token 用量异常;
- 故障演练 + Chaos Engineering。

层 8 · 安全与合规:

- DDoS 防护(CDN + WAF);
- 输入内容审核(敏感词、prompt injection、政治、色情);
- 输出内容审核(政治、暴力、虚假信息、未成年保护);
- PII 脱敏:日志、训练数据中的个人信息;
- 数据合规:GDPR / 网安法 / 数据出境;
- 审计日志:满足监管;
- 内容备案 / 算法备案。

层 9 · 数据回流与模型迭代:

- 用户反馈(点赞 / 点踩 / 转人工)收集;
- Bad case 沉淀;
- 定期 fine-tune / RLHF / DPO;
- A/B 测试新版本;

- 离线评测体系(自动 + 人工)。

三、关键设计决策:

(1) 模型分级路由(节省成本的核心):

- 简单 query(问候、FAQ):规则 + 缓存,不调用 LLM 或调小模型;
- 中等 query(常规对话):中型模型(7-13B);
- 复杂 query(推理、代码、长文档):大模型;
- 用 Router(规则 + 小分类模型)前置判断;
- 整体成本可降低 50-80%。

(2) 会话亲和性 + Prefix Caching:

- 同一用户的请求尽量路由到同一 GPU 节点;
- 历史对话 KV cache 复用,显著提速 + 省钱;
- 配合 SGLang RadixAttention / vLLM Automatic Prefix Caching。

(3) 流式输出 + 异步:

- SSE / WebSocket 流式,降低用户感知延迟;
- 长任务(写作、Agent)用异步 task,完成后推送。

(4) 多区域部署:

- 用户就近接入(GeoDNS);
- 数据本地化(用户数据不跨区);
- 跨区灾备(主区故障切备区)。

(5) 自动扩缩容:

- 按 GPU 利用率、队列长度、TTFT 触发扩容;
- 模型加载慢(几十秒),需要预扩容 + 温启动节点池;
- 离线低峰缩容降成本。

(6) 限流与降级:

- 用户级限流(免费用户、付费用户配额不同);
- 全局熔断(防爆冲);
- 过载时降级到小模型 / 排队 / 拒绝;
- 高优先级请求队列。

(7) 缓存策略:

- 精确 **prompt** 缓存(常见问题、确定性回答);
- 语义缓存(用 **embedding** 找相似 **prompt**);
- 命中率 10-30% 可显著省钱;
- 注意时效性敏感问题不缓存(天气、新闻)。

(8) 安全防护:

- 输入过滤前置,推理资源不浪费在恶意请求;
- 输出过滤,流式审核(边出边审,违规立刻截断);
- 越狱检测(Prompt Injection);
- 滥用检测(疑似自动化、薅羊毛行为)。

(9) 成本控制:

- 量化模型(FP8 / INT8)降推理成本一半;
- 选小模型 + 微调替代大模型 **zero-shot**;
- 自研模型替代付费 **API**(规模化后必走);
- 用 **Spot** 实例 / 抢占式 **GPU**(对延迟不敏感的批任务);
- **Prompt** 优化降 **token** 量。

(10) 高可用:

- 推理服务多副本 + 健康检查;

- 跨可用区 / 跨区双活;
- 模型多版本并存,故障回滚秒级;
- 数据库主从 + 异地容灾;
- 关键链路全链路压测 + 故障演练。

四、容量估算示例:

日活 1 亿,人均 10 次对话,平均每次 800 token 输出;

日总 token = 1 亿 $\times$ 10 $\times$ 800 = 8000 亿 token;

若用 H100 跑 70B 量化模型,单卡约 3000 tokens/s,需要 ~3000 卡持续运转;

考虑峰值 5 倍 + 冗余,需要约 15000-20000 卡集群;

加上 RAG、记忆、多模态等周边服务,GPU 集群需求 20000+ 卡。

这种规模一般是头部公司(OpenAI、Anthropic、Google、字节、阿里、腾讯、百度)级别。中型公司更可能用云 API + 部分自建组合。

五、面试要点:

- 强调"分层 + 微服务 + 多模型路由 + Prefix Caching + 自动扩缩"几个核心;
- 成本敏感性永远是第一位;
- 不要只讲模型,数据 / 安全 / 可观测同等关键;
- 给出明确的延迟、QPS、成本估算思路。

Q7. 如何做大模型应用的成本优化?

LLM 应用成本主要来自:推理算力(token 计费 / GPU 时长)、向量库存储、外部 API、人工标注、运维。成本优化是规模化生产必修课,可从模型、prompt、缓存、架构、商务多维度入手。

一、明确成本结构(先量化再优化):

- Token 成本:input + output 各自计费,output 通常贵 3-5 倍;
- 模型选择:GPT-4 比 GPT-4o-mini 贵 20 倍以上;

- 调用次数:总 QPS;
- 平均 token 长度;
- 计算 ARPU(每用户成本) 或 单次任务成本。

工具:建立成本 dashboard,按模型 / 用户 / 业务线 / 功能维度统计。

二、模型层优化:

(1) 模型分级路由:

- 简单任务(分类、抽取、改写、FAQ)用小模型(GPT-4o-mini / Haiku / Qwen-turbo / DeepSeek);
- 中等复杂(对话、摘要、轻推理)用中型模型;
- 复杂任务(深度推理、代码、Agent)才用顶级模型(GPT-4o / Claude Sonnet / Opus / o1);
- 用规则或小分类器前置选模型;
- 整体成本可降 50-80%。

(2) 模型微调替代大模型:

- 用小模型 + LoRA 微调在专业任务上达到大模型水平;
- 一次性微调成本几百美元,推理成本砍 10-20 倍;
- 适合任务边界明确的场景(分类、抽取、特定 QA)。

(3) 蒸馏:

- 大模型作老师,小模型作学生,在特定数据上蒸馏;
- 推理成本大降;
- 适合数据量大的场景。

(4) 量化部署(自建):

- FP8 / INT8 / INT4 量化,显存和算力需求大幅下降;
- 自建场景下 4-bit 推理比 FP16 节省 60-70% 显卡成本。

(5) 自建 vs API 平衡:

- 量小 → 用 API, 无运维成本;
- 量大 → 自建, 长期看每 token 成本可低 5-10 倍;
- 关键拐点: 月支出超过 \$5K-10K 一般可以考虑自建。

三、Prompt 层优化:

(1) 精简 prompt:

- 砍掉冗余指令、重复说明、过多 few-shot;
- 用专业术语而非啰嗦白话;
- few-shot 示例从 10 个砍到 3 个, 效果差不多;
- 实测每砍 100 token 节省 N%;

(2) Prompt Caching:

- 长 system prompt / 工具 schema / 长文档放前面, 启用厂商缓存;
- Claude / OpenAI / Gemini 都支持, 缓存命中部分计费打 1-5 折;
- 适合 system prompt 反复使用的场景, 节省 50-90%;

(3) Prompt 压缩:

- LLMingua、Selective-Context 等工具压缩长 prompt;
- 把 long context 压缩 2-10 倍, 信息损失小;
- 适合长文档场景;

(4) 控制 output 长度:

- output 比 input 贵 3-5 倍, output 长度是成本大头;
- 明确指令"用 200 字以内"、"用要点列出"、"省略解释";
- max_tokens 严格设;
- 避免让模型重复用户问题、加多余客套话;

(5) 结构化输出:

- 让模型输出 JSON 而非长文本,token 数更可控;
- 后端把 JSON 渲染为用户看的内容,前后端分离;

(6) 多步任务拆分:

- 一次性复杂任务可能用上 5K+ token;
- 拆成多步小任务,每步只看必要信息,总 token 反而少。

四、缓存层优化:

(1) 精确缓存:

- 完全相同的 prompt 直接返回历史结果(Redis);
- 命中率取决于业务,客服 FAQ 可达 30-50%,开放对话 10-20%;

(2) 语义缓存:

- 用 embedding 找相似问题(GPTCache、Redis Vector);
- 命中阈值要严,避免误命中导致回答不切题;

(3) Prompt 中间结果缓存:

- RAG 检索结果缓存(同一 query 30 分钟内 反复出);
- Function Calling 工具结果缓存;
- 长文档预处理结果(切分、embedding)缓存。

五、架构层优化:

(1) 非 LLM 替代:

- 能用规则、正则、传统 NLP(分类、抽取)、检索做的,先用,不调 LLM;
- LLM 只用在真正需要的地方。

(2) RAG 优化:

- 减少召回 top-k(从 20 砍到 5);
- 用 Reranker 精排,只给模型最相关的几条;
- 切分大小调优,减少冗余。

(3) Batch + 异步:

- 离线 / 非实时任务用 batch API(成本打折,如 OpenAI batch 5 折);
- 异步队列削峰;

(4) Spot / 抢占式实例(自建):

- 对延迟不敏感的训练 / 批推理用 Spot,价格 1/3;
- 在线推理用 on-demand;

(5) 自动扩缩容:

- 低峰自动缩容,避免 GPU 闲置;
- 但要预留冗余防峰值。

六、商务层优化:

(1) 多厂商比价:

- OpenAI、Anthropic、Google、Azure、AWS、阿里、火山、DeepSeek 等价格差异大;
- 同等质量下选最便宜;
- DeepSeek-V3 / Qwen 等性价比高;

(2) 谈判 / 包年:

- 大客户与厂商谈 enterprise 价格,折扣 20-50%;
- 包年承诺量获折扣;



(3) 替代厂商:

- 国产模型在中文任务上常比国外更便宜更好;
- 开源模型自部署(Llama-3、Qwen 系列)。

七、监控与归因:

(1) 成本可视化 Dashboard:

- 按用户 / 业务 / 模型 / 功能维度;
- 异常飙升告警;

(2) 单次任务成本:

- 每次请求都打标签,精确归因;
- LangSmith / Langfuse 自带 cost 字段;

(3) 滥用检测:

- 异常账号 / 自动化攻击导致 token 爆炸;
- 设单用户配额 + 异常告警。

八、典型节省套路与预期效果:

- (1) 模型分级路由:节省 50-80%;
- (2) Prompt Caching:节省 50-90%(适合长 system prompt 场景);
- (3) 精简 prompt + 控制 output:节省 20-40%;
- (4) Batch API:节省 50%;
- (5) 自建 vs API(规模化):节省 80-95%;
- (6) 缓存:节省 10-30%(取决于业务);
- (7) 小模型微调替代:节省 90%+。

组合起来,百万级月支出可降到十万级 - 几十万级。

九、避坑:

- 不要盲目追求最强模型——明确"质量上限" vs "够用"的差距,业务能用就行;
- 不要忽视 output 成本——output 长往往是隐形成本黑洞;
- 不要忽视 retry / failure—— retries 占成本可能 10-20%;
- 监控先行——没有数据就没法优化。

一句话:LLM 成本优化是"小步快跑的工程战"——模型分级 + Prompt Caching + 精简输出 + 缓存四件套是基础,规模化后自建 + 微调 + 蒸馏是终极武器。

Q8. 生产环境如何监控 LLM?

LLM 生产监控比传统服务复杂得多——既要监控系统指标(QPS、延迟、错误率),又要监控模型质量(回答好坏、幻觉、漂移),还要监控成本和安全。需要分层、自动化、可告警的完整体系。

一、监控维度(四大类):

(1) 系统指标(传统 SRE):

- QPS / 并发数;
- 端到端延迟:P50 / P95 / P99,TTFT(首 token)、TPOT(每 token);
- 成功率 / 错误率:5xx、429、超时;
- GPU 利用率、显存占用、batch size;
- KV Cache 命中率(Prompt Caching);
- 队列长度、排队时间;
- 上下游依赖:向量库、外部 API 健康度。

(2) 业务指标:

- 用户数(DAU / MAU)、会话数、消息数;
- 用户满意度:CSAT、NPS、点赞/点踩比;

- 转人工率、任务完成率、会话长度;
- 留存、活跃曲线;
- 业务转化(订单、注册、付费)。

(3) 模型质量指标:

- 回答相关性(Answer Relevance);
- 忠实度 / 幻觉率(Faithfulness):RAG 场景关键;
- 拒绝率 / 过度拒绝(Over-refusal);
- 工具调用准确率:工具选对率、参数填对率;
- 安全合规违规率(政治、色情、暴力检测);
- 格式合法率(JSON / 结构化输出);
- 多样性 / 重复率;
- Bad case 比例;
- Drift:回答风格 / 质量随时间漂移。

(4) 成本指标:

- Token 使用量:input / output / cache (创建 / 命中);
- 单次请求成本、单用户成本、单业务成本;
- 模型分布(各模型占比);
- 缓存命中率;
- 异常飙升告警。

二、监控方法与工具:

(1) 全链路追踪(Tracing):

- 每次请求生成唯一 trace_id,贯穿前端 → 网关 → LLM → 工具 → 返回;
- 记录每个 span:prompt、completion、token、延迟、错误;
- 工具:LangSmith、Langfuse、Helicone、Arize Phoenix、Honeycomb、Datadog APM、

Jaeger;

- 关键能力:复现 bad case(用户反馈"AI 答错了" → 立刻在 trace 找到完整上下文)。

(2) 指标聚合(Metrics):

- Prometheus + Grafana 是事实标准;
- 自定义 LLM 指标:token、延迟、命中率、成本;
- 多维度切片(模型、用户层级、地区);
- 实时 Dashboard。

(3) 日志聚合:

- ELK Stack(Elasticsearch + Logstash + Kibana) / Loki + Grafana / Splunk;
- 结构化日志(JSON),便于查询;
- 注意 PII 脱敏!不要把用户敏感信息明文存日志。

(4) 告警:

- Prometheus AlertManager / PagerDuty / Opsgenie;
- 关键告警:成功率 < 99%、P95 延迟 > 阈值、Token 消耗异常飙升、违规率 > X%;
- 分级:critical → 电话叫醒,warning → 群通知;
- 避免噪声(动态阈值、抖动过滤)。

(5) 自动化质量评测(在线 + 离线):

在线评测:

- 实时抽样请求(1% / 5%)做 LLM-judge 评分;
- 用户反馈(点赞/点踩)即时统计;
- 异常检测(回答过短、重复、错误格式);
- 工具调用失败率;

离线评测:

- 每天 / 每周跑评测集(数百-数千条 ground-truth);
- 对比版本间质量,捕捉退化;
- 周期性人工抽检 100-500 条;
- RAGAS / DeepEval / 自建 LLM-judge。

(6) Drift 监控:

- 输入分布漂移:用户 query 分布变化(用 embedding 聚类 / KS 检验);
- 输出分布漂移:回答平均长度、风格、关键词分布;
- 性能漂移:相同评测集随时间的得分下降;
- 上游 API 表现漂移(厂商升级模型后效果变了)。

(7) 安全监控:

- 实时违规检测:政治、色情、暴力、虚假信息(用专门审核模型或规则);
- Prompt Injection 检测:可疑指令模式;
- 异常账号:疑似自动化、滥用、薅羊毛;
- 越狱检测:用户 prompt 模式匹配已知越狱模板;
- 内容审计:输出违规率 / 拒答率;
- 数据泄漏检测:输出中是否包含训练数据中的敏感片段。

(8) 成本监控:

- 实时 token 计数;
- 按业务 / 用户 tag 归因(LangSmith / Langfuse 内置);
- 预算告警(单日 / 单月超限);
- 异常用户:单用户每分钟 token 飙升报警;
- 模型分布看是否路由失常(全打 GPT-4 没分流到小模型)。

三、用户反馈闭环:

(1) 显式反馈:

- 点赞 / 点踩 / 评分按钮;
- "纠正错误"按钮:用户标注更好答案;
- 满意度调查问卷;

(2) 隐式反馈:

- 用户没继续追问 → 大概率满意;
- 用户重复问相似问题 → 大概率没答对;
- 转人工 → 强信号;
- 离开会话 → 可能挫败;

(3) 反馈→数据集:

- Bad case 自动入库;
- 定期 review,改 prompt / 加知识 / 微调;
- 形成"数据飞轮":越用越好。

四、LLM 监控特殊工具(LLMOps):

(1) LangSmith(LangChain 出品):

- 全链路 trace、评估、prompt 管理;
- 与 LangChain 深度集成;

(2) Langfuse(开源):

- 开源 LangSmith 替代;
- 自部署,适合数据敏感场景;
- 支持 trace、score、dataset、prompt;

(3) Helicone(开源 + SaaS):

- 1 行代理代码集成;



- 侧重成本和性能;

(4) Phoenix(Arize):

- 开源 LLM observability;
- 强项:drift 检测、embedding 可视化;

(5) WhyLabs LangKit、TruLens、Confident AI:

- 评测 + 监控一体化;

(6) Datadog / New Relic LLM monitoring:

- 传统 APM 厂商的 LLM 扩展。

五、监控的最佳实践:

(1) 黄金 Dashboard:

- 一屏看核心指标:QPS、成功率、P95、token、成本、违规率;
- 关键漏斗:接入 → LLM → 工具 → 返回的转化率;
- 顶部异常告警一目了然;

(2) 三层告警:

- 系统级(SRE 接管):服务挂了;
- 质量级(产品 / ML 接管):效果下降;
- 业务级(运营接管):用户量下跌;

(3) 回归测试 + CI:

- 每次发版自动跑核心评测集;
- 关键指标退化 → 阻塞上线;

(4) Shadow Mode:

- 新模型上线先 shadow(不影响用户)跑一段时间;
- 真实流量看效果,验证 OK 再切流;

(5) PII / 合规:

- 日志脱敏:手机号、身份证、地址掩码;
- 数据保留期合规;
- 用户删除请求要能追溯并删除其数据。

(6) 数据飞轮:

- Bad case 持续回流;
- 评测集持续扩充;
- 模型 / Prompt 持续迭代。

六、面试要点:

- 强调 LLM 监控不是只看系统指标,质量 / 安全 / 成本同等重要;
- LangSmith / Langfuse 这类专属工具是现代 LLMOps 必备;
- 用户反馈闭环 + 离线评测 + 在线抽样三层结合;
- Drift 和 bad case 是长尾问题,需要专门设计。

Q9. 为什么要手搓 Agent,而不是直接用 LangChain/AutoGen?

"手搓"指不依赖重型框架,直接基于 LLM API + 业务代码实现 Agent。这是当前业界一个明显趋势:Anthropic、Cursor、Vercel 等都公开表达过对 LangChain 等重型框架的反思。手搓与框架各有适用场景,核心在于权衡。

一、Agent 框架的两种风格:

(1) 高抽象、组件化:LangChain、LlamaIndex(早期)、Haystack

- 提供 Chain、Agent、Tool、Memory、Retriever 等高层抽象;



- 内置数百个集成(各 LLM、向量库、工具);
- 上手快,demo 容易,但抽象太深,生产排查难。

(2) 状态图 / 显式控制:LangGraph、AutoGen、CrewAI、Pydantic AI

- 更接近显式有限状态机或多 Agent 协作模型;
- 中等抽象,平衡灵活与简洁;
- LangGraph 是 LangChain 团队的"反思后产品",更显式。

(3) 最小框架 / 直接 API:OpenAI SDK + 自写循环,或 Vercel AI SDK、Pydantic AI、Mastra 等

- 直接用裸 LLM API + 自定义控制流;
- 代码量少,完全可控。

二、为什么很多团队选择"手搓":

(1) 控制力:

- 框架的抽象隐藏了关键细节(prompt 拼装、错误处理、工具调用格式),出 bug 时不知道发生了什么;
- 手搓 = 完全掌握每个 prompt、每个 API 调用、每次状态变化;
- 生产事故排查时间从"几小时翻源码"降到"分钟级 grep"。

(2) 性能:

- 框架有不必要的开销(对象创建、序列化、回调链);
- 关键路径每毫秒都重要,手搓可以极致优化;
- 框架的并发 / 流式实现常常不够好。

(3) Prompt 透明:

- 框架背后的 prompt 模板对用户不可见,版本升级 prompt 可能改了你不知道;
- 手搓 = 自己掌握每个字,版本可控;
- 对 prompt engineering 至关重要。

(4) 升级与依赖:

- 框架快速迭代,break change 频繁(LangChain 0.0 → 0.1 → 0.2 → 0.3 多次大改);
- 升级一个版本可能要改一周代码;
- 手搓只依赖 LLM SDK,稳定得多。

(5) 集成深度:

- 框架的"通用集成"为了泛用而失去深度;
- 业务专属逻辑(权限、租户、审计、个性化)很难塞进框架抽象;
- 手搓自然贴合业务。

(6) 调试与可观测:

- 框架的 trace 不一定满足业务需求;
- 手搓直接对接公司既有的 logging / metrics / tracing 体系;
- 用 LangSmith / Langfuse / 自家平台,均可自由选择。

(7) 学习理解:

- 用框架几个月,可能对底层"Agent 究竟怎么 work"理解不深;
- 手搓一遍,Agent 范式刻进骨子里;
- 团队心智模型更清晰。

(8) 模型能力跟上:

- 早期模型弱,需要框架的复杂编排;
- 现代模型(GPT-4 / Claude 4 / Gemini 2)的指令跟随、tool use 能力极强,大量框架做的事直接 prompt 就能搞定;
- 框架抽象的边际收益降低。

(9) Anthropic 的明确建议:

官方文档《Building Effective Agents》(2024) 直接建议:"开始时直接用 LLM API,不要立刻引入

框架;只在确实需要时再引入。”并展示了大量“几十行代码搞定的 Agent 模式”,证明手搓的简洁性。

三、什么时候框架仍有价值:

(1) 快速原型 / Demo / Hackathon:

- LangChain 几分钟搭 demo;
- 不要求生产质量。

(2) 团队成员不熟 LLM:

- 框架降低上手门槛;
- 但代价是后期维护负担。

(3) 多模态 / 多工具集成生态:

- 框架提供数百个现成集成(各种向量库、API、工具);
- 自己实现 connector 工作量大;

(4) 复杂 Multi-Agent 编排:

- LangGraph、AutoGen、CrewAI 提供状态机、消息路由、协作模式;
- 自己实现这些抽象成本不小;
- 但简单 Multi-Agent 仍可手搓。

(5) 生态工具配套:

- LangSmith 与 LangChain 深度集成;
- LlamaIndex 在 RAG 上有专门优化。

四、实践建议(中间路线):

(1) 核心循环手搓 + 局部用框架:

- Agent 主控制流自写;
- RAG / 向量库用 LlamaIndex 或简单 SDK;
- Tool 描述格式直接用 OpenAI/Anthropic SDK 原生 schema;
- 监控用 LangSmith / Langfuse(SDK 集成,非框架强绑定)。

(2) 用轻量框架:

- Pydantic AI(类型友好、显式 prompt);
- Vercel AI SDK(JS 生态、UI 集成强);
- OpenAI Agents SDK(2025,官方轻量);
- Mastra、Inferable 等新一代轻量化框架。

(3) 自建"框架"作为公司内部库:

- 把公司业务通用的 Agent 模式抽象成内部 SDK;
- 比通用框架贴合业务,可控性强;
- 大公司常走这条路(字节、阿里都有内部 LLM 平台)。

五、手搓 Agent 的最小骨架(示意):

```
def agent_loop(user_msg, tools, max_steps=10):
    messages = [system_prompt, user(user_msg)]
    for _ in range(max_steps):
        resp = llm.create(messages=messages, tools=tools)
        if not resp.tool_calls:
            return resp.content
        messages.append(resp)
        for call in resp.tool_calls:
            try:
                result = execute_tool(call.name, call.args)
```

except Exception as e:

result = f"Error: {e}"

messages.append(tool_msg(call.id, result))

return "Max steps reached"

几十行代码,实现完整的 tool-use Agent,可控可调,生产可用。

六、面试要点:

- 不要无脑黑 LangChain——它在生态、Demo、教学上仍有价值;
- 强调"控制力、可调试、Prompt 透明"是手搓的核心收益;
- 提到 Anthropic 的官方建议,体现对业界趋势的了解;
- 给出中间路线:核心手搓 + 局部用工具,平衡灵活与效率;
- 体现工程审美:抽象不是越多越好,够用就好。

Q10. 用过哪些大模型网关框架,如 LiteLLM?

LLM 网关框架(LLM Gateway / Proxy)解决"一份代码对接多家模型"的问题,统一鉴权、路由、限流、缓存、监控。下面是主流方案的对比与实践经验。

一、LiteLLM:

(1) 定位:

Python 写的 LLM 统一 SDK + Proxy,把 100+ LLM 厂商的 API 抽象成 OpenAI 兼容格式。
BerriAI 出品,开源,广泛使用。

(2) 两种使用模式:

- SDK 模式:

```
from litellm import completion
```



```
response = completion(model="claude-3-5-sonnet-20241022", messages=[...])
```

```
response = completion(model="gpt-4o", messages=[...])
```

一行切换厂商,代码不变。

- Proxy 模式:

部署 LiteLLM Proxy(litellm --config config.yaml),业务方调一个 OpenAI 兼容 URL,Proxy 在后端做路由、鉴权、限流、缓存。

(3) 主要能力:

- 100+ LLM 厂商接入(OpenAI、Anthropic、Google、Azure、Bedrock、Cohere、各国产模型、本地 Ollama、vLLM、SageMaker 等);
- 统一 OpenAI Chat Completion API 格式;
- 路由:多 API key、多区域、负载均衡、故障切换、智能路由;
- 限流(全局 / 用户 / 模型 / TPM-RPM);
- 缓存(Redis 精确缓存);
- 重试 + 超时控制;
- 鉴权(虚拟 API key,业务方拿不到真 key);
- 成本追踪:按 user / team / project 标签计费;
- 日志 + 多平台对接(LangSmith、Langfuse、Helicone、Datadog、Slack 告警);
- 内容审核(可接 OpenAI Moderation、Lakera、自定义);
- Function Calling、Streaming、JSON Mode、Vision 多模态全支持;
- 嵌入 / Rerank / 图像生成等周边 API 也统一;
- 配置文件式管理(YAML 定义模型、key、限额、回退);
- 支持 fallback chain(主模型失败自动降级);

(4) 优点:

- 上手快(SDK 一行,Proxy Docker 几分钟起);
- 厂商覆盖最全;
- 社区活跃,迭代快;



- 开源 Apache 2, 可商用;
- 适合中小团队、混合多模型场景。

(5) 缺点:

- Python 实现, 大流量下吞吐有上限(几千 QPS, 要扩成多副本);
- 早期版本稳定性一般, 2024 后逐渐成熟;
- 高级功能(数据库 key、admin UI)需要 Enterprise / 自部署完整版;
- 对自托管模型的支持配置稍繁琐。

二、One API / New API(国内主流):

(1) 定位:

Go 语言开源 LLM 网关, songquanpeng 出品的 One API 是源头, 社区分叉 New API、FreeGpt-API、Chat-API 等。专门为国内场景设计。

(2) 主要能力:

- 多渠道管理: 统一接入 OpenAI、Claude、Gemini、文心、通义、智谱、Kimi、DeepSeek 等;
- 自带 Admin Web UI;
- 用户体系 + 配额管理 + 充值;
- 渠道权重、自动重试、可用性切换;
- Token 计费 + 报表;
- 兼容 OpenAI API;
- 部署超简单(单二进制 + SQLite/MySQL/PostgreSQL)。

(3) 优点:

- Go 实现, 性能优秀(数万 QPS);
- 国产模型支持全, 中文文档好;
- 自带 Admin UI, 无需额外搭建;
- 部署门槛极低;



- 国内中小团队 / 个人最爱。

(4) 缺点:

- 偏向"代理 + 计费",高级 LLM Ops(评测、tracing、Prompt 管理)较弱;
- 二次开发需要 Go 经验;
- 个人开发者维护,生产级支持不如商业产品。

三、Portkey:

(1) 定位:

商业 + 开源混合,功能丰富的 LLM Gateway,定位企业级。

(2) 能力:

- 200+ 模型集成;
- Gateway(路由 / fallback / cache / 重试);
- Observability(trace / cost / 评估);
- Prompt Management(版本化、A/B、模板);
- Guardrails(输入输出审核);
- Virtual Keys(分租户);
- 开源 portkey-ai/gateway 是网关核心,SaaS 是完整产品。

(3) 适合:中型团队、想要"一体化平台"的;

(4) 缺点:开源版功能受限,完整版需要付费。

四、Helicone:

(1) 定位:

主打 Observability 的 LLM Gateway,1 行 base_url 替换即可集成。



(2) 能力:

- Proxy 模式 + Async 日志模式;
- 强力的 Dashboard:请求列表、成本、延迟、用户分析;
- 缓存、限流、Retry;
- 自部署 + SaaS;
- Prompt 管理、评估、A/B。

(3) 适合:重视 Observability 的团队;

(4) 国际化为主,国内模型支持稍弱。

五、Cloudflare AI Gateway / Vercel AI Gateway / AWS Bedrock Gateway:

云厂商方案,与各自生态绑定,适合已经在用对应云的团队。

六、Kong AI Gateway / Higress AI Gateway:

基于传统 API Gateway(Kong / Nginx Ingress)的 AI 扩展。

- 优点:已有 API Gateway 体系的企业自然延伸;
- 高性能、企业级功能(认证、限流、可观测);
- Higress 是阿里基于 Envoy 的,中国厂商场景好;
- 缺点:LLM 专属特性(评测、prompt 管理)较弱。

七、商业方案:

- TrueFoundry、Truefoundry、Lakera、Vellum 等;
- 阿里云 PAI、火山引擎、AWS Bedrock、Azure AI Studio 都有内置网关能力。

八、自研方案:

大厂(字节、阿里、腾讯、Meta)通常自研。原因:

- 深度集成内部体系(SSO、IAM、监控、计费);
- 性能 / 安全极致要求;



- 不愿被第三方框架绑定。

九、选型决策框架:

(1) 个人 / 小团队 / 实验:

→ LiteLLM SDK + One API,或者直接 LiteLLM Proxy;

(2) 中小生产团队、混合模型:

→ LiteLLM Proxy + Langfuse(Observability);

(3) 国内场景为主、国产模型:

→ One API / New API;

(4) 企业级、完整 LLMOps:

→ Portkey 商业版 / Helicone / Vellum;

(5) 已有 API Gateway 体系:

→ Kong AI Gateway / Higress AI Gateway;

(6) 大厂或对性能 / 安全有极高要求:

→ 自研。

十、关键评估维度:

- 厂商覆盖广度;
- 性能(QPS、延迟开销);
- Observability 能力;
- 用户 / 配额 / 计费;
- 安全 / 审核 / Guardrails;
- 缓存 / 路由 / fallback 智能度;



- 部署运维成本;
- 社区活跃 + 长期维护;
- 二次开发能力(语言、扩展机制);
- 商业模式(开源 vs 商业,数据隐私)。

十一、实践经验小结:

- 起步用 LiteLLM 是最稳的选择,生态最全;
- 国内场景 One API 极简部署,小团队适合;
- 重视 observability → Langfuse 与上述任一搭配;
- 规模化后自研可能性大,LLM 网关是核心基础设施,不容易完全外包;
- 多个方案叠加也常见:LiteLLM 做路由 + Langfuse 做 trace + 自家系统做计费。

一句话:LiteLLM 是 LLM 网关界的"瑞士军刀",生态最全;One API 是国内开源界的"小钢炮",简单高效;企业级要考虑 Portkey / 自研。

第 7 章 多模态与 VLM

高频面试题详解 · Q1 ~ Q8

Q1. VLM 的核心挑战是什么?如何实现视觉和语言对齐?

VLM(Vision-Language Model,视觉-语言模型)指能同时理解图像/视频和文本、跨模态推理的模型,如 GPT-4o、Claude 3.5/4、Gemini、Qwen-VL、InternVL、LLaVA 等。核心目标是让"看"和"说"在同一个语义空间里融通。

一、核心挑战:

(1) 模态鸿沟(Modality Gap):

视觉是连续、稠密、空间结构性的(像素 / patch);语言是离散、稀疏、序列结构的(token)。两者天然分布差异巨大,直接拼一起 LLM 看不懂。

(2) 表征对齐:

同一个概念(一只猫)在图像里是像素分布,在文本里是 token "cat" 或 "猫"。如何让两者在共享语义空间里位置相近,是核心难题。

(3) 粒度匹配:

一张图包含数十到上千个 patch,但其中可能只有几个区域与当前问题相关。如何让模型聚焦关键区域(grounding)而忽略噪声,是难点。

(4) 信息密度差异:

一张高分辨率图等价于几千甚至上万 token,直接喂给 LLM 会爆炸,需要压缩 / 选择 / 多分辨率策略。

(5) 多任务统一:

VQA、Caption、OCR、Grounding、文档理解、UI 操作、视频理解,任务千差万别,需要统一架构和训练。

(6) 长尾视觉概念:

罕见物体、特殊场景、专业图(医学、电路、地图)训练数据少,模型理解差。

(7) 视觉幻觉:

模型"看到"图中不存在的物体、错认细节、复述常见模式而非真实图像内容,比文本幻觉更难检测。

(8) OCR / 文档:

图像中的文字理解(发票、屏幕截图、PPT)对实用价值极高,但需要既能识别又能理解版面结构,挑战大。

(9) 时空理解:

视频引入时间维度,需要建模动作、因果、状态变化,远比单图复杂。

(10) 训练数据与对齐成本:

高质量 (image, text) 对稀缺;噪声大;指令微调数据更稀缺;评测也难。

二、如何实现视觉-语言对齐(主流方法):

(1) Contrastive Learning(对比学习,CLIP 范式):

- 双塔结构:图像 encoder + 文本 encoder;
- 拉近匹配的 (image, text) 对在共享空间中的距离,推开不匹配的;
- 经典实现:CLIP、ALIGN、SigLIP;
- 优点:双塔可独立编码,适合检索;
- 缺点:不能直接生成文本,只能做匹配;粒度较粗;
- VLM 中作"视觉特征提取器"使用,与 LLM 配合。

(2) Adapter / Projector(投影器,LLaVA 范式):

- 用预训练好的视觉 encoder(CLIP ViT)提取图像特征,再用一个轻量投影模块(MLP / Q-Former

/ Resampler)把图像 token 映射到 LLM 的 word embedding 空间;

- 把这些"视觉 token"作为前缀塞进 LLM 的输入序列,让 LLM 像处理文本一样处理图像;
- 训练:先冻结视觉 + LLM,只训 projector(对齐);再做指令微调;
- 代表:LLaVA、MiniGPT-4、InstructBLIP、ShareGPT4V;
- 优点:轻量、可复用强大 LLM、训练成本低;
- 缺点:浅融合,细粒度对齐能力有限。

(3) Cross-Attention 融合:

- 在 LLM 内部插入 cross-attention 层,让文本 token 主动 attend 到图像特征;
- 代表:Flamingo(DeepMind,2022)、IDEFICS;
- 优点:深度融合,对齐更精细;
- 缺点:需要改 LLM 结构,训练成本高。

(4) Q-Former(查询变换器,BLIP-2):

- 在视觉特征与 LLM 之间放一个小的 transformer,用一组可学习 query 向量"询问"图像特征;
- 输出固定数量的 query 向量作为视觉 token;
- 优点:压缩了视觉 token 数量,信息密度高;
- 缺点:训练复杂,后续模型(LLaVA 1.5+)发现简单 MLP 已足够;
- 现在 Q-Former 用得少了。

(5) Native Multimodal(原生多模态):

- 从预训练阶段就同时见图、见文(早期融合);
- 代表:GPT-4o、Gemini 1.5/2.5、Chameleon、Fuyu、Janus、Qwen2.5-VL;
- 视觉和文本 token 共享同一序列、同一 transformer、同一目标(next-token prediction);
- 优点:最深度的融合,能力上限最高,支持任意模态交错输入输出;
- 缺点:训练数据、算力极其昂贵。

(6) 各种 patch 处理策略:

- 固定网格 patch:简单但分辨率受限;
- 动态分辨率(Qwen-VL / InternVL):图像按宽高比切多块,每块独立 patch;
- 多尺度 patch(高低分辨率结合);
- AnyRes(LLaVA-NeXT):支持任意分辨率;
- 像素级压缩(token merging、token pruning)。

三、训练流程(主流三阶段):

阶段 1 · 视觉-语言预对齐:

冻结视觉 encoder 和 LLM,只训 projector,用海量 (image, caption) 对训练,让视觉 token 进入 LLM 的语义空间。

阶段 2 · 多模态预训练:

解冻 projector 和 LLM(可能也解冻部分视觉层),在更复杂的多模态数据上预训练(图文交错文档、OCR、grounding、文档、视频)。

阶段 3 · 视觉指令微调(Visual Instruction Tuning):

在 (image, question, answer) 形式的指令数据上微调,让模型学会按指令完成多模态任务 (VQA、Caption、OCR、推理)。可加 RLHF / DPO 进一步对齐人类偏好。

四、面试关键句:

VLM 的核心 = 视觉 encoder(看)+ 投影对齐(翻译)+ LLM(说)。挑战在模态鸿沟、粒度匹配、token 爆炸、视觉幻觉。主流对齐方法是 CLIP 风格的对比学习 + LLaVA 风格的 projector + 指令微调三件套;最先进的是 native multimodal,从预训练阶段就统一。

Q2. CLIP 的原理是什么?

CLIP(Contrastive Language-Image Pretraining,OpenAI 2021)是开创了多模态视觉-语言对齐范式的里程碑工作。核心思想是用大规模图文对做对比学习,让图像和对应文本在共享语义空间中距离近。

一、模型结构:

CLIP 是双塔(Two-Tower)结构:

- (1) 图像编码器(Image Encoder):ViT 或 ResNet,把图像编码为定长向量(如 512 维);
- (2) 文本编码器(Text Encoder):Transformer,把文本编码为同维度向量;
- (3) 共享投影空间:两路输出投影到同一空间,做 L2 归一化。

两个 encoder 独立工作,推理时可分别预计算图像或文本的 embedding。

二、训练:对比学习目标

训练数据:OpenAI 收集了 4 亿对 (image, text) 来自互联网。

训练目标(InfoNCE 损失):

在一个 batch 里有 N 个 (image, text) 对。每张图都有一个真正配对的文本(正例),其余 N-1 个文本都是负例。模型学到:

- 正例对 (image_i, text_i) 的相似度 $\cos(v_i, t_i)$ 尽量高;
- 负例对 (image_i, text_j), $j \neq i$ 的相似度尽量低。

损失函数:

$$L = - (1/2) * [L_{i2t} + L_{t2i}]$$

$$L_{i2t} = \log[\exp(\text{sim}(v_i, t_i) / \tau) / \sum_j \exp(\text{sim}(v_i, t_j) / \tau)]$$

$$L_{t2i} = \log[\exp(\text{sim}(v_i, t_i) / \tau) / \sum_j \exp(\text{sim}(v_j, t_i) / \tau)]$$

其中 τ 是温度参数(可学习)。

本质:把对比学习视作 batch 内 N 路分类——给定图像,从 N 个候选文本中选出正确的;反之亦然。

大 batch size(32K+)对效果至关重要——负例越多对比信号越强。

三、关键设计点:

- (1) 简单的训练目标:不用复杂的预测头,纯对比;
- (2) 自然语言监督:用文本"caption"作监督信号,比传统标签丰富得多(零样本能力强);
- (3) 大规模:4 亿对图文,从互联网爬取;
- (4) 双塔可解耦:推理时图像和文本独立 embedding,适合做检索;
- (5) 模态无关共享空间:任意视觉与任意文本可计算相似度。

四、能力与应用:

(1) 零样本图像分类:

把每个类名构造成 prompt("a photo of a cat"),计算图像与各 prompt 的相似度,取最大类。无需任何 fine-tune,在 ImageNet 上达 76% 准确率,接近全监督 ResNet-50。

(2) 图文检索:

给文本找图、给图找文,大规模库下高效(预算 embedding + 向量检索)。

(3) 多模态搜索引擎:

电商图搜、相册搜索、内容审核。

(4) 作为视觉特征提取器:

CLIP 学到的视觉表征语义性强,被广泛用作下游任务的"视觉骨干":

- VLM(LLaVA、MiniGPT-4):用 CLIP ViT 提取视觉特征再喂 LLM;
- 文生图(Stable Diffusion、DALL-E):用 CLIP text encoder 把文本编码为生图条件;
- 视频理解、3D 等下游任务。

(5) Zero-shot detection / segmentation:

配合 SAM、GroundingDINO 等做开放词汇检测分割。

五、局限与改进:

局限:

- 粒度粗:只学了图整体与文本整体的对应,不擅长细粒度对齐(精确定位某物);
- 数据偏见:网络爬取数据有偏见、噪声;
- 不会生成:只能匹配,不能生成文本或图像;
- 对长文本理解有限(默认 77 token 上限);
- 对计数、空间关系、属性绑定弱。

改进 / 后续工作:

- ALIGN(Google):更大规模噪声数据;
- SigLIP(Google):用 sigmoid 取代 softmax,可以不依赖大 batch;
- EVA-CLIP:更大、更强的 CLIP;
- OpenCLIP:开源社区复现 + 扩展;
- BLIP / BLIP-2:加上生成能力;
- LiT、CoCa、FILIP:不同对比策略;
- DFN(Apple):data filtering 提升数据质量。

六、面试关键点:

- CLIP = 双塔 + InfoNCE + 大规模图文对;
- 核心贡献:用自然语言"caption"代替"类别标签",解锁零样本和开放词汇能力;
- 通过对比学习把图文映射到共享语义空间;
- 是当代 VLM、文生图、多模态检索的基石;
- 局限是粒度粗、不能生成,后续工作(BLIP、Flamingo、LLaVA)在此基础上扩展。

Q3. LLaVA / MiniGPT-4 如何连接视觉编码器和 LLM?

LLaVA(Large Language-and-Vision Assistant,Haotian Liu 等 2023)和 MiniGPT-4(Deyao

Zhu 等 2023)是开源 VLM 的两个开创性工作。它们的核心思想都是:复用强大的视觉 encoder(CLIP)+ 强大的 LLM(Vicuna / LLaMA),中间加一个轻量"翻译模块"做对齐,以极低成本得到一个能看图聊天的模型。

一、共同架构(Vision-Encoder + Projector + LLM 三件套):

(1) 视觉 encoder:

- 用预训练好的 CLIP ViT-L/14(或类似 backbone);
- 输入图像(224x224 或 336x336);
- 输出一组 patch 特征,例如 $24 \times 24 = 576$ 个 token,每个 1024 维;
- 通常冻结,不参与训练。

(2) 投影模块(Projector):

- 把视觉 token 投影到 LLM 的 word embedding 空间(LLM 的 hidden_size,例如 4096 维);
- 这是关键的"翻译层",决定了视觉信息能否被 LLM 理解。

(3) LLM:

- LLaVA 用 Vicuna(基于 LLaMA 微调的对话模型);
- MiniGPT-4 用 Vicuna(初版)或 LLaMA-2;
- 把视觉 token 当作"软 prompt"插入到文本 token 之前(或者交错);
- LLM 像处理文本一样处理视觉 token,生成回答。

输入序列格式(简化):

[BOS] <视觉 token 1> <视觉 token 2> ... <视觉 token N> "请描述这张图片" → LLM 生成回答

二、LLaVA 的关键设计:

(1) Projector 选择:

- LLaVA-1:用一个简单的线性层(1 个矩阵)把 1024 维投影到 4096 维;

- LLaVA-1.5:升级为 2 层 MLP(linear → GELU → linear),效果显著更好;
- LLaVA-NeXT:支持 AnyRes 高分辨率,把图切多块,每块过 CLIP,投影后拼接;
- 关键思想:projector 不需要很复杂,简单的 MLP 就够,关键是足够多的训练数据。

(2) 两阶段训练:

- 阶段 1 · 特征对齐(Pre-training):
 - 冻结 vision encoder + LLM,只训 projector;
 - 数据:CC3M 等图文 caption 数据(595K 对);
 - 目标:让 projector 学会把视觉 token 翻译到 LLM 能"听懂"的语义;
 - 训练快,几小时-一天。
- 阶段 2 · 视觉指令微调(Visual Instruction Tuning):
 - 解冻 projector + LLM(视觉 encoder 仍冻结);
 - 数据:LLaVA-Instruct(用 GPT-4 生成的高质量视觉指令数据,158K 对);
 - 目标:让模型学会按指令做 VQA、详细描述、推理等任务;
 - 训练时间稍长。

(3) 训练数据构造(LLaVA 的核心贡献):

- 用 GPT-4(纯文本)+ 图像的 caption / bbox / objects 信息,生成三类指令数据:
 - ① 对话式问答(Conversation):用户与 AI 围绕图像的多轮对话;
 - ② 详细描述(Detailed Description):一段对图像的细致描述;
 - ③ 复杂推理(Complex Reasoning):基于图像的多步推理问答;
- 这种"用 LLM 生成多模态指令数据"开创了 visual instruction tuning 范式,被广泛沿用。

(4) LLaVA 系列演进:

- LLaVA-1.0:基础架构;
- LLaVA-1.5:MLP projector + 更高分辨率 + 更多数据,效果跃升;
- LLaVA-NeXT(1.6):动态高分辨率 + 更强 LLM 选项;

- LLaVA-OneVision:统一图像、多图、视频。

三、MiniGPT-4 的关键设计:

(1) Projector:

- 用 Q-Former(BLIP-2 中的查询变换器)+ 一个线性层;
- Q-Former 用 32 个可学习 query 从视觉特征中提取信息,输出 32 个紧凑视觉 token;
- 再线性投影到 LLM 维度;
- 视觉 token 数量比 LLaVA 少得多(32 vs 576),节省 LLM 上下文。

(2) 训练:

- 阶段 1:在 5M 图文对上预训练,只训 Q-Former 后的线性层;
- 阶段 2:用一小部分高质量(图像 + 长描述)对做指令微调,显著提升输出质量;
- MiniGPT-4 当时的关键发现:第一阶段后输出有时颠三倒四,需要少量高质量指令数据"教模型说人话"。

(3) Q-Former 后被 LLaVA 的 MLP 取代:

- LLaVA 1.5 论文证明简单 MLP + 更多数据效果反超 Q-Former;
- 现在主流走 MLP 路线,Q-Former 用得少了。

四、两者对比:

维度 / LLaVA / MiniGPT-4:

- Projector:简单 MLP / Q-Former + Linear;
- 视觉 token 数:576(高分辨率多块更多)/ 32;
- 训练数据:GPT-4 生成的指令数据 / Caption + 长描述;
- LLM:Vicuna / Vicuna(初)→ LLaMA-2;
- 优势:简洁、可扩展、效果强 / 视觉 token 少、上下文友好;
- 后续影响:开创 visual instruction tuning,LLaVA 范式成为开源 VLM 主流 / 探索 Q-Former 路



线,启发后续工作。

五、核心启示:

(1) 复用 + 对齐 = 高性价比 VLM:

不需要从头训练巨大的多模态模型,把预训练好的视觉 **encoder** 和 **LLM** 用一个轻量 **projector** 对齐,就能得到强 **VLM**。这是 2023 年开源 **VLM** 爆发的关键。

(2) 数据质量 > 架构复杂度:

LLaVA-1.5 用简单 **MLP** 替代 **Q-Former** 反而效果更好,关键在用 **GPT-4** 生成的高质量指令数据。",

(3) 两阶段训练:

"对齐 + 指令微调"成为开源 **VLM** 标准流程。

(4) 局限:

- 视觉 **encoder** 冻结,无法学习视觉 **encoder** 没见过的概念;
- 分辨率有限(早期 224/336),小字、细节看不清;
- 浅融合,深度多模态推理不如 **native multimodal** 模型;
- 视觉 **token** 数限制了图像信息量。

后续工作改进:解冻视觉 **encoder**、**AnyRes** 多分辨率、增加视觉 **token**、动态切块、多图视频统一等。

六、面试关键句:

LLaVA / MiniGPT-4 = **CLIP ViT**(看)+ 轻量 **Projector**(翻译)+ **Vicuna**(说)。**Projector**(**MLP** 或 **Q-Former**)把视觉特征投影到 **LLM** 词嵌入空间,作为软 **prompt** 喂进 **LLM**。两阶段训练(对齐 + 视觉指令微调)是开源 **VLM** 标准流程。**LLaVA** 系列证明"简单 **MLP** + 高质量数据"路线最优,成为后续 **Qwen-VL**、**InternVL** 等的范式基础。

Q4. 什么是视觉指令微调?

视觉指令微调(Visual Instruction Tuning,VIT)是 VLM 训练的关键环节:在已对齐视觉-语言表征的基础上,用大量"图像 + 指令 + 期望回答"形式的数据进一步微调,让模型学会按用户指令完成多模态任务(描述、问答、推理、定位、OCR 等)。

由 LLaVA(Liu et al. 2023)首次系统化提出并大规模实践,成为现代开源 VLM 的标准做法。

一、为什么需要视觉指令微调:

阶段 1 视觉对齐之后,模型理解了图像内容,但行为还是"caption 模式"——看到图就生成描述,不擅长"按用户指令回答"。例如:

- 用户问"图里的人在做什么?"——未微调的模型可能输出整张图的描述,而非聚焦到人物动作;
- 用户问"图里有多少只猫?"——模型不会数;
- 用户给出复杂指令(先描述再推理)——模型无法跟随。

VIT 的目标:把视觉理解能力与"指令跟随"能力对齐,产出一个真正的多模态对话助理。

二、关键数据形式:

一条 VIT 样本通常是:

```
{  
  "image": <图像>,  
  "conversations": [  
    {"from": "human", "value": "<image>\n 这张图里发生了什么?"},  
    {"from": "assistant", "value": "图中是一位男士在公园里遛狗..."},  
    {"from": "human", "value": "狗看起来像什么品种?"},  
    {"from": "assistant", "value": "看起来像是金毛寻回犬..."}  
  ]  
}
```

可能是单轮或多轮,可能是 VQA、Caption、推理、OCR、Grounding(定位)、计数等多种任务混合。

三、典型 VIT 数据类别:

(1) 详细描述(Detailed Description):

"请详细描述这张图。" → 一段几百字的细致描述,包含主体、动作、背景、氛围。

(2) 对话式 VQA(Visual Q&A):

"图中有什么?" / "这是什么车?" / "现在是什么季节?" → 简洁问答。

(3) 复杂推理(Complex Reasoning):

"为什么这个人看起来很赶?" / "图中场景可能发生在什么时间?为什么?" → 需要常识 + 视觉信息综合推理。

(4) OCR / 文档理解:

"图中文字写了什么?" / "发票总金额是多少?" / "图中表格第二行内容是?" → 识别 + 结构化提取。

(5) Grounding(视觉定位):

"图中的猫在哪?" → 输出坐标 $[x1, y1, x2, y2]$ 或相对位置描述。

Referring:"图中红色衣服的人在做什么?";

Region Caption:"描述左上角区域。"。

(6) 计数 / 比较 / 属性:

"图中有多少个苹果?" / "哪个更大,A 还是 B?" / "瓶子是什么颜色?"

(7) 多图任务:

"图 1 和图 2 有什么区别?" / "按时间顺序描述这一组图。"

(8) 指令跟随(Format Following):

"用 JSON 格式列出图中所有物品" / "用三个 bullet 总结" / "用诗歌形式描述"。

(9) 拒答 / 安全:

"图中的人是谁?"(隐私)→ 拒绝识别身份;暴力 / 违禁 → 安全应对。

(10) 视频指令(Video Instruction Tuning):

"这个视频中发生了什么?" / "在哪个时间点出现了 X?"

四、VIT 数据怎么构造(数据是 VIT 的生命线):

(1) LLaVA 路线 - GPT-4 合成:

- 已有图像 + 该图的 caption / 物体框 / 检测结果(纯文本描述);
- 把这些文本喂给 GPT-4(纯文本模型),让它"扮演用户和助手"围绕图像生成多轮指令对话;
- 生成详细描述、复杂推理、对话三大类样本;
- 优点:成本低、可扩展、质量高;
- 缺点:GPT-4 没真正看图,生成内容可能与图像细节不一致;现在更多用 GPT-4V / Claude / Gemini 直接看图生成。

(2) 公开数据集转换:

- VQA、COCO Caption、OK-VQA、TextVQA、DocVQA、ChartQA、OCR-VQA 等现成数据集;
- 把它们转成统一指令格式(添加 prompt 模板);
- 量大、覆盖广,但格式略呆板。

(3) 真实人工标注:

- 用户提问 → 人工写回答,质量最高,成本最贵;
- 用于关键场景(医疗、法律、安全);

- 商业 VLM(GPT-4o、Claude)大量使用。

(4) Bootstrapping(模型自蒸馏):

- 用已训练好的强 VLM 给新图生成 (q, a) 对,作为新数据;
- 配合人工筛选,质量与规模兼顾。

(5) RLHF / DPO:

- 在 VIT 之后,用偏好数据进一步对齐人类喜好;
- 提升回答的有用性、准确性、安全性;
- 视觉 RLHF 数据更稀缺,但工业模型(GPT-4o、Claude、Gemini)都用了。

五、训练设置:

(1) 解冻策略:

- 视觉 encoder:通常冻结(LLaVA-1.5);新模型有时部分或完全解冻(InternVL、Qwen2.5-VL),让视觉表征也能更新;
- Projector:训练;
- LLM:训练(全参或 LoRA)。

(2) Loss:

- next-token prediction,但 loss 只在 assistant 部分计算;
- 用户消息和视觉 token 不算 loss。

(3) Batch 与上下文:

- 视觉 token 多,batch size 受限;
- 高分辨率切块时 token 数巨大,需要 Flash Attention、ZeRO 等。

(4) 学习率:

- Projector 学习率高($1e-3$ 量级);

- LLM 学习率低($2e-5$ 量级,防止灾难性遗忘)。

六、数据规模演进:

- LLaVA-1.0:158K 条指令数据;
- LLaVA-1.5:665K 混合数据;
- Qwen-VL / Qwen2-VL:数百万-千万级;
- 现代 SOTA 模型(GPT-4o、Gemini、Claude):无法公开,估计千万到亿级,且大量是人工 + RLHF 标注。

数据质量与多样性比绝对数量更关键,LLaVA-1.5 用 600K+ 高质量数据已能与早期百万级模型抗衡。

七、效果与影响:

- VIT 之后,模型从"图像描述器"升级为"多模态对话助理";
- 视觉指令跟随、推理、OCR、复杂任务能力质变;
- 是开源 VLM 与商业 VLM 共同的核心训练阶段。

八、挑战与陷阱:

- GPT-4 合成数据的"幻觉传染":GPT-4 没看图编内容,模型学到错的;
- 数据偏置:caption-heavy 数据让模型啰嗦,缺少简洁回答的样本;
- 灾难性遗忘:微调后纯文本能力下降,需要混入文本数据;
- 长尾任务(OCR 表格、医学图)需要专项数据;
- 多语言、多文化覆盖。

九、面试关键句:

视觉指令微调 = 在已对齐 VLM 上,用大量(图像, 指令, 回答)数据继续微调,让模型从"看图说话"升级为"按指令做多模态任务"。LLaVA 开创范式,GPT-4 合成数据是关键工程突破。数据多样性 + 质量 + 适当的解冻策略 + 与文本数据混合是成功关键。

Q5. 视频多模态相比图像多模态多了什么问题?

视频是"带时间维度的图像序列",视觉模态从静态 2D 扩展到 2D + 时间。看似只多了一维,实际上引入了一系列特有挑战,把多模态模型推到更复杂的层级。

一、视频多了什么(本质差异):

(1) 时间维度:

一段视频不是几张图,而是动作、变化、因果的载体。模型需要建模:谁在做什么、动作的顺序、状态的转变、事件的因果。

(2) 信息量爆炸:

一段 30 秒视频按 1 fps 抽帧就有 30 张图,每张图按 LLaVA 风格是 576 token,共 17280 视觉 token——比一张图大 30 倍。1 分钟、10 分钟、1 小时视频呢?

(3) 冗余与稀疏并存:

相邻帧高度相似(冗余),但关键事件可能就 1-2 帧(稀疏),如何取舍是核心难题。

(4) 多模态交错:

视频通常有视觉 + 音频 + 字幕,真正的"多模态"。

二、具体挑战:

(1) Token 数量爆炸:

直接逐帧编码后塞 LLM,1 分钟视频就远超 LLM 上下文窗口。需要:

- 帧采样:均匀抽 K 帧、关键帧检测、自适应密度;
- 视频 token 压缩:Q-Former、Resampler、TimeSformer、token merging;
- 时序聚合:把相邻帧合并为一个 token;
- 分段处理 + 摘要:长视频先分段,每段独立摘要,再综合。

(2) 时序建模:

不只是看每一帧,还要建模帧间关系——动作演变、因果、状态转换。常见方法:

- Temporal Attention:在帧 token 间加注意力;
- 3D Convolution:经典视频 CNN;
- TimeSformer / Video Swin / VideoMAE:视频专用 transformer;
- 时间位置编码:给每帧 token 加时间戳信息;
- 在 VLM 中,把帧序列拍平后让 LLM 通过自注意力学时序。

(3) 长视频理解:

电影、监控、教学视频可能 1-10 小时,挑战极大。研究方向:

- 分层处理:先粗粒度(关键帧 + 摘要)、必要时再细粒度(回看局部段);
- Memory 机制:类似 MovieChat、LWM、LongVA,把长视频压缩进 memory;
- 检索 + 生成:RAG 范式,先检索相关片段再回答;
- Streaming Video:边播边处理,不需要全片预加载。

(4) 关键帧选择:

从几千帧里挑出"重要"的几十帧:

- 均匀采样:简单但可能错过关键时刻;
- 场景切换检测:用 PySceneDetect 等工具;
- 显著性 / 运动量:动作幅度大的帧更重要;
- 语义相关性:基于 query 检索相关帧;
- LLM 驱动:让 LLM 决定看哪些帧(agenting)。

(5) 视频 grounding(时序定位):

"X 发生在第几秒?" / "在哪个时间段出现了 Y?"——需要输出时间戳。难度大,数据稀缺,评测指标 (IoU) 严苛。

(6) 视频字幕生成:

生成准确、连贯、时序对齐的字幕,需要时序理解 + 语言流畅。

(7) 动作识别 / 行为理解:

"图里的人在做什么"和"这个人完成了什么动作序列"是两回事。后者需要时序逻辑(走过去→拿起来→打开→放下)。

(8) 因果与逻辑推理:

"为什么 X 发生?" / "如果不做 Y 会怎样?"——需要从视频中提取因果链,远超单图视觉问答。

(9) 多模态融合(视频 + 音频 + 字幕):

真实视频的关键信息可能在音频(对话、背景声)或字幕里,纯视觉模型会丢失。需要:

- 语音转文本(ASR);
- 音频 encoder(Whisper、AudioMAE);
- 字幕 OCR;
- 三路融合到 LLM。

GPT-4o、Gemini 2.0+、Claude 4 等是这类原生多模态模型。

(10) 评测困难:

- 数据稀缺:高质量视频问答数据集少(VideoQA、Activity Net、Ego4D、MVBench);
- 长尾:动作类别、场景多样,评测覆盖不全;
- 自动指标(BLEU 等)与人类判断差异大;
- LLM-judge 评估视频内容本身就难。

(11) 训练数据获取与标注:

- 视频远比图像难收集、难标注;
- 标注成本高(标 1 分钟视频可能要 5-10 分钟);
- 视频 + 详细描述 + 时序信息的高质量数据集稀缺;
- 现代做法:用 ASR + image captioner + LLM 合成视频描述。

(12) 计算成本:

- 视频训练数据量级大,GPU 时长翻倍;
- 推理时显存压力大,长上下文管理复杂;
- 端侧视频实时处理几乎不可能(没有量化 + 帧数限制)。

三、主流视频 VLM:

(1) Video-LLaVA / VideoChat / VideoChatGPT:

早期开源工作,在 LLaVA 基础上加视频帧抽样和时序聚合;

(2) Video-LLaMA、PandaGPT:

视频 + 音频多模态;

(3) Qwen2.5-VL、InternVL、MiniCPM-V:

原生支持图像 + 视频;

(4) LLaVA-Video / VideoLLaMA3:

专门优化视频任务;

(5) Long-VA、MovieChat、Vista-LLaMA:

长视频专项;

(6) Gemini 1.5/2.5:

原生百万 token 上下文,可直接处理 1 小时视频;

(7) GPT-4o:

原生多模态,支持视频 + 音频实时对话(语音通话式)。

四、典型评测 benchmark:

- MSR-VTT、ActivityNet-QA:经典视频 QA;
- MVBench、Video-MME:全面视频理解;
- VideoMME、TempCompass:时序推理;
- EgoSchema、Ego4D NLQ:第一视角长视频;
- VideoChatGPT-Bench:对话评测;
- Charades-STA、ActivityNet-Captions:视频 grounding。

五、设计考量:

- (1) 任务确定抽帧策略:动作识别需要密集,问答可稀疏;
- (2) 显存预算反推上下文长度:1 小时视频要 100K+ token,需选支持长上下文的 LLM;
- (3) 多模态优先级:视频内容为主还是音频?是否需要字幕?
- (4) 时间精度要求:秒级还是毫秒级 grounding?
- (5) 在线 / 离线:直播分析需要 streaming 模型,离线分析可以慢但准。

六、面试关键句:

视频比图像多了"时间维度",带来 token 爆炸、关键帧选择、时序建模、长视频处理、音视频多模态融合、训练数据稀缺等一系列挑战。主流方案:帧采样 + 时序聚合 + 长上下文 LLM + 多模态对齐;前沿方向:长视频流式处理、视频 + 音频 + 字幕原生多模态、视频 grounding。

Q6. VLM 幻觉和纯文本 LLM 幻觉有什么区别?

幻觉(Hallucination)指模型输出与事实或上下文不符的内容。VLM 幻觉是 LLM 幻觉的"升级版"——除了文本幻觉,还有"看到了不存在的事物"、"漏看真实事物"等独有的视觉幻觉,且更难检测。

一、LLM 文本幻觉(回顾):

主要表现:

- 编造事实(不存在的人物、事件、论文);
- 错误数字、日期、引用;

- 推理错误;
- 风格 / 立场幻觉;
- 主要根源:训练知识有限或过时、模型参数记忆模糊、推理短板。

二、VLM 幻觉的独有类型:

(1) 物体幻觉(Object Hallucination):

回答中提到图像中并不存在的物体。

例:图里只有一张桌子,模型说"桌子上有一个杯子"——杯子根本不存在。

这是 VLM 最常见、最显著的幻觉类型,有专门 benchmark(POPE、CHAIR、AMBER)评测。

(2) 属性幻觉(Attribute Hallucination):

把物体的属性说错——颜色、数量、形状、位置、状态。

例:狗是黑色,模型说"白色狗";只有 2 只猫,模型说"3 只猫"。

(3) 关系幻觉(Relation Hallucination):

把物体之间的关系搞错——空间("A 在 B 左边"还是右边)、动作("A 拿着 B"实际没拿)、归属。

(4) 场景幻觉(Scene Hallucination):

错认整体场景——以为是"沙滩"实际是"草地";以为是"公园"实际是"室内"。

(5) OCR 幻觉:

识别图中文字时,把没有的字读出来,或把模糊字幻觉成具体字。发票金额、表格数字尤其敏感。

(6) 推理幻觉:

基于错误视觉前提的推理——前提就错,后续推理全错。

(7) 反事实幻觉(Counterfactual):

用户给出错误前提("图中有 5 个人,描述他们")模型不质疑、不指出图里只有 3 个,反而配合幻觉出



5 个人。

(8) 风格 / 知识幻觉:

模型基于"这类图常见"的模式(先验)而非真实图像作答。

例:看到食物图片,模型按"日料店常见"模式输出,而实际是中餐。

(9) "看不见"的幻觉:

该看的没看到——重要细节被忽略;问"图里有什么特殊符号"模型说"没有",实际有。

三、VLM 幻觉 vs LLM 幻觉的本质区别:

(1) 多了"视觉感知"环节出错:

LLM 错在"知识 / 推理";VLM 还可能错在"看错 / 没看清",根因更复杂。

(2) 视觉与语言可能不一致:

模型可能正确看到了图,但语言层面没用上;或者用了不在图里的先验知识。

(3) 难检测:

文本幻觉可以通过查证(知识库、搜索)发现;视觉幻觉需要重新看图对比,自动化检测难。

(4) 难评测:

需要专门设计 **benchmark**,标注成本高,主观性强;LLM 幻觉评测相对成熟。

(5) 模态偏置:

常见现象——模型"过度依赖语言先验"(基于 **caption** 模式回答,而非真正看图)。这在指令微调阶段被无意强化(GPT-4 合成数据本身就是凭文本想象的)。

(6) 错误传染:

视觉感知错 → 后续推理错;一旦"看错",整个回答全错,没有兜底。

四、VLM 幻觉的成因:

(1) 视觉 encoder 能力有限:

低分辨率(224/336)看不清小字、细节;无法精确计数(超过 5 个就难);不擅长空间关系。

(2) Token 压缩损失:

为了塞进 LLM,视觉特征被压缩到少量 token,细节丢失。

(3) 训练数据偏置:

- 大量数据是"描述图像"模式,模型学到"该说什么"而非"该看什么";
- GPT-4 合成的指令数据自身就有幻觉(GPT-4 当时不真正看图);
- 公开 caption 数据噪声大,常常描述与图不符。

(4) 语言先验过强:

LLM 部分远比 vision encoder 强,模型倾向于"基于看到的词 → 想象图里有什么",而非真正基于视觉。

(5) RLHF / 偏好对齐反向激励:

人类标注员可能更偏好"详细、丰富、长"的回答 → 模型学会编更多细节 → 增加幻觉。

(6) 上下文长度限制:

高分辨率多帧场景下,信息被截断或压缩。

五、缓解视觉幻觉的方法:

(1) 提升视觉编码:

- 高分辨率支持(AnyRes、动态切块);
- 更强的视觉 encoder(SigLIP、EVA-CLIP);

- 多分辨率融合;
- 解冻视觉 encoder,让视觉表征也能更新。

(2) 数据策略:

- 高质量人工标注 / 真实多模态指令数据;
- 用真正"看图"的强 VLM(GPT-4o、Claude)合成数据,而非纯文本 LLM;
- 加入"否定样本"教模型说"没有 X";
- 反幻觉专项数据(POPE 风格 yes/no 问答)。

(3) 训练目标:

- DPO / RLHF 用反幻觉偏好数据;
- POVID、HA-DPO、RLHF-V 等专门针对幻觉的偏好对齐;
- 增强对"看到 vs 没看到"的明确区分。

(4) 推理时技术:

- VCD(Visual Contrastive Decoding):对比有图和无图的输出,惩罚共同部分;
- OPERA:检测过度依赖少量视觉 token 的注意力模式并惩罚;
- Visual Self-Correction:让模型先描述再回答,然后自我检查;
- Caption-then-answer:先 caption 再 QA,推理更踏实。

(5) 后处理 / 验证:

- 让模型给出引用("我看到红色物体在左下角");
- 用 detection 模型校验存在性;
- 用 OCR 模型独立校验文字;
- 多模型集成投票。

(6) Prompt 工程:

- 明确要求"只描述图中确实看到的,不要推测";

- 让模型先列出看到的物体,再回答问题(类似 CoT);
- 指令"如果不确定,请说不确定"。

(7) 评测驱动:

- 用 POPE、AMBER、HallusionBench 持续评测;
- Bad case 回流训练数据。

六、评测 benchmark:

- POPE(Polling-based Object Probing Evaluation):yes/no 问"图中有 X 吗",简单但有效;
- CHAIR(Caption Hallucination Assessment):评估 caption 中幻觉物体比例;
- AMBER:多维度幻觉评测;
- HallusionBench:复杂幻觉场景;
- MMHal-Bench:实际对话幻觉;
- VHTest、Bingo:视觉幻觉专项。

七、面试关键点:

- VLM 幻觉比 LLM 幻觉更"看得见"——物体幻觉、属性错认、关系搞反、OCR 失误;
- 根因复杂:视觉 encoder 弱 + token 压缩损失 + 语言先验过强 + 训练数据噪声;
- 难检测、难评测,但已经有 POPE、AMBER 等专门 benchmark;
- 缓解需要从数据、训练、推理、评测多管齐下;
- 工业上常用 DPO/RLHF + 反幻觉数据 + 推理时对比解码组合。

Q7. 如何评估 Grounding 能力?

Grounding(视觉定位)指 VLM 把语言描述与图像中的具体区域/物体精确对应起来的能力。是 VLM 从"看图说话"升级到"精确指代和操作"的关键能力,在指代理解、机器人、AI 智能体(屏幕操作)等场景至关重要。

一、Grounding 任务类别:

(1) Referring Expression Comprehension(REC,指代表达理解):

给定一段语言描述(如"穿红衣服的男人"),输出对应物体的 bounding box $[x1, y1, x2, y2]$ 。

主流数据集:RefCOCO、RefCOCO+、RefCOCOg、Visual Genome、Flickr30k Entities。

(2) Referring Expression Generation(REG):

给定图像 + bbox,生成一段唯一指代该物体的描述。

(3) Phrase Grounding:

给一句话中的多个短语,把每个短语链接到图像中的对应区域。

(4) Visual Question Answering with Grounding(VQA-Grounding):

VQA 同时要求模型指出回答的视觉依据区域。

(5) Open-Vocabulary Detection / Segmentation:

给任意类别名(包括训练时没见过的),检测/分割图像中所有此类物体。

主流模型:Grounding DINO、OWLv2、SAM + CLIP。

(6) Pixel-level Grounding(语义分割):

"指出穿红衣服的人" → 输出像素级 mask。

主流:LISA、Pixel-LM、GLaMM。

(7) Region-level QA:

"图中标注框 $[x1,y1,x2,y2]$ 内是什么?" → 描述指定区域。

(8) GUI Grounding(界面定位,新兴重要场景):

"点击'确认'按钮" → 输出按钮在屏幕上的坐标。

应用:Anthropic Computer Use、Claude Code、ChatGPT 浏览器 Agent、AutoGUI。

Benchmark:ScreenSpot、ScreenSpot-Pro、SeeClick。

二、主流评估指标:

(1) IoU(Intersection over Union):

预测框与 ground-truth 框的交集面积 / 并集面积。

$\text{IoU} = 1.0$ 完美重合, 0 完全不重合。

$\text{IoU}@0.5$ / $\text{IoU}@0.75$: 阈值标准, 常用 0.5 表示"基本正确", 0.75 表示"精确"。

(2) Accuracy@K(命中率):

一次预测的 box 与 GT 的 $\text{IoU} >$ 阈值, 记为正确; 计算正确率。

REC 任务上 $\text{Acc}@0.5$ 是最常报告的指标。

(3) Precision / Recall / F1(检测任务):

多个物体场景下, 与 GT 集合做匹配, 算 TP/FP/FN。

(4) mAP(mean Average Precision, 检测标准):

在不同 IoU 阈值下计算 AP 后平均(如 COCO 的 $\text{mAP}@[0.5:0.05:0.95]$)。

(5) Pointing Accuracy / Click Accuracy:

预测一个点(中心点 / 单击位置), 与 GT bbox 是否落入。

GUI grounding 常用, 因为点击只需要一个点。

(6) cloU(complete IoU):

考虑中心点距离 + 长宽比的扩展 IoU, 更全面。

(7) Mask IoU(像素级):

分割任务下, 预测 mask 与 GT mask 的 IoU。

(8) Distance-based 指标:

中心点距离、归一化距离,对 GUI 等场景适用。

(9) 语言匹配指标(REG):

BLEU、METEOR、CIDEr、SPICE 评估生成的指代表达质量。

三、Grounding 在 VLM 中的输出形式:

(1) 直接输出数值坐标:

"<box>[235, 412, 580, 720]</box>" 或 [x1, y1, x2, y2] 归一化到 [0, 1] 或 [0, 1000]。

主流 VLM(Qwen-VL、CogVLM、InternVL)的做法。

(2) Special Token 表示:

把坐标量化成 special token(<x_0> <y_0> ... <x_999> <y_999>),作为词表一部分。

Pix2Seq、Kosmos 路线。

(3) 输出 mask 索引:

配合 SAM 等分割模型,VLM 输出物体描述,SAM 出 mask。

(4) 输出点坐标:

只输出一个点(适合 GUI、单击场景),格式更简洁。

四、评估流程:

(1) 数据集准备:

- 测试集要有 GT(图像 + 描述 + GT box / mask);
- 主流:RefCOCO / +/g(自然图像)、ScreenSpot(GUI)、AITW(移动 App)、WebArena;
- 类别覆盖、难度分布要均衡。

(2) 模型推理:

- 给图像 + 文本指令;
- 解析 VLM 输出的坐标 / mask;
- 反归一化到图像像素坐标;
- 注意不同模型的坐标格式(归一化范围、xyxy vs xywh)。

(3) 指标计算:

- 与 GT 比较;
- 不同 IoU 阈值下统计准确率;
- 按类别 / 大小 / 难度切片分析。

(4) 错误分析:

- 偏移类:位置大致对但精度不够;
- 错认类:定位到错误物体;
- 漏检:没找到;
- 多检:同名物体多个时定位错(指代歧义)。

五、Grounding 评估的挑战:

(1) 语言歧义:

"红色的物体"图里有两个,模型选哪个都"对",评测难。

需要多个 GT 接受任一,或用更严格的描述。

(2) 坐标精度:

VLM 直接输出坐标的精度有限(通常 $\pm 5-10$ 像素以上),细小物体精度不够。

(3) 不同分辨率:

高分辨率图像和低分辨率上的精度不一样,要规范化对比。

(4) 输出格式差异:

不同模型坐标格式不同,需要 parser 统一。

(5) 多任务统一评测:

一个 VLM 既要 caption 又要 grounding,综合评测难。

(6) GUI grounding 的复杂性:

小按钮、密集元素、动态布局,挑战极大,精度普遍较低(早期 50%-,现在 70%+)。

六、Grounding 能力的提升方法(评估的同时也是研究):

(1) 数据增强:

加入大量 REC 数据(数百万级);

合成数据(自动从 detection 数据集生成 grounding 样本);

(2) 高分辨率支持:

小物体定位需要高分辨率(AnyRes、UReader 风格);

(3) 坐标 token 化:

把坐标作为 special token 学,精度提高;

(4) 强化视觉 encoder:

解冻或换更强 backbone;

(5) 引入专门 grounding 头:

Shikra、Ferret 等模型加专门的 grounding 模块;

(6) Chain-of-Pointing / 分步推理:

先描述要找的物体,再输出坐标;

(7) RLHF / DPO 优化:

用 IoU 或人类偏好作 reward 训练。

七、主流 grounding-enabled VLM 与性能:

- Qwen-VL / Qwen2-VL / Qwen2.5-VL: 坐标 token + 大量 REC 数据, 主流开源 SOTA;
- CogVLM / CogAgent: GUI grounding 强;
- Kosmos-2: special token 坐标;
- Shikra、Ferret、GLaMM: 专门 grounding;
- OS-Atlas、UI-TARS: GUI Agent 专门优化;
- 商业: Claude(computer use)、GPT-4o、Gemini 2 都有 grounding。

八、面试关键点:

- 核心指标 = IoU + Acc@IoU 阈值;
- 任务多样: REC、REG、Phrase Grounding、Open-Vocab Detection、Pixel Grounding、GUI Grounding;
- 新兴热点: GUI grounding(计算机操作类 Agent 的基石);
- 评估流程: 数据集 → 推理 → 坐标解析 → IoU 计算 → 分类型分析;
- 挑战: 语言歧义、坐标精度、不同模型输出格式、小物体定位。

Q8. 是否做过 VLM 微调? 用什么模型?

这是面试中考察实操经验的开放题。下面提供一个真实可用的 VLM 微调路线, 涵盖模型选型、数据准备、训练框架、效果评估、踩坑经验, 适合作为答题骨架(具体细节可结合自身项目调整)。

一、典型微调场景:

- 公司业务需要专业领域 VLM(医学影像、工业质检、文档理解、电商商品识别、UI 操作、特定 OCR);
- 通用 VLM 在长尾任务上表现不够, 需要专项微调;
- 端侧部署需要小模型 + 蒸馏;

- 中文场景下需要更好的中文图文能力。

二、模型选型(2024-2025 主流开源 VLM):

(1) Qwen2.5-VL(阿里):

- 规模:3B / 7B / 32B / 72B;
- 强项:中文支持好、动态分辨率、原生 grounding、视频支持、OCR 强、生态成熟;
- 训练框架支持:LLaMA-Factory、ms-swift、Qwen 官方仓库;
- 端侧友好(3B / 7B 量化后可端侧部署);
- 工业首选,推荐微调起点。

(2) InternVL 2.5 / 3(上海 AI Lab):

- 规模:1B 到 78B;
- 视觉 encoder InternViT 自研,质量极高;
- 学术 SOTA,多 benchmark 强;
- 微调生态稍弱但日益成熟。

(3) MiniCPM-V / MiniCPM-o(面壁智能):

- 端侧专精,8B 以下;
- 手机端实时跑,业内端侧标杆;
- 适合移动应用场景。

(4) LLaVA-NeXT / LLaVA-OneVision:

- 经典开源 VLM 系列;
- 架构清晰,适合学术 / 教学;
- 多图视频统一(OneVision)。

(5) DeepSeek-VL2:

- MoE VLM,推理高效;

- 学术任务表现强。

(6) Phi-3.5-vision / Phi-4-Multimodal(微软):

- 端侧小模型;
- 英文场景强。

(7) Llama-3.2-Vision、Gemma3-Vision、Mistral Pixtral:

- 国际厂商方案;
- 英文为主。

(8) 选型决策:

- 中文 + 综合 SOTA:Qwen2.5-VL-7B/72B;
- 端侧:Qwen2.5-VL-3B / MiniCPM-V;
- GUI / Agent:Qwen2.5-VL、CogAgent、OS-Atlas、UI-TARS;
- 学术研究:InternVL、LLaVA-OneVision;
- 限定预算 LoRA 起步,效果不够再考虑全参微调。

三、典型微调路线(以 Qwen2.5-VL-7B 为例):

阶段 1:数据准备(决定成败 80%)

(1) 数据构造:

- 收集业务图像:几千到几十万张;
- 标注 (图像 + 指令 + 期望回答) 三元组;
- 标注形式与 LLaVA / Qwen-VL 一致的 JSON 对话格式;
- 任务类别:简单问答、复杂推理、Caption、Grounding、OCR、结构化抽取;
- 多样性:同一类任务多种表述方式,避免过拟合 prompt 模板;

(2) 数据来源:

- 人工标注:质量最高,核心场景必用;
- 模型蒸馏:用强 VLM(Qwen-Max-VL、Claude、GPT-4o)对自家图像生成指令数据,再人工抽检;
- 公开数据集改造:COCO、Visual Genome、DocVQA、ChartQA 改写格式;
- 业务日志回流:用户真实问题。

(3) 数据规模建议:

- LoRA 微调:几千到几万条已能见效;
- 全参微调:几万到百万条;
- 数据质量 > 数量,优质 5K 优于劣质 50K。

(4) 数据混合:

- 业务数据(目标场景):60-80%;
- 通用 VLM 指令数据(防止退化):20-40%;
- 防止灾难性遗忘很重要,否则模型业务好但通用变差。

阶段 2:训练框架与配置

(1) 框架选择:

- LLaMA-Factory:界面友好、配置简洁、社区活跃,推荐入门 / 中小规模;
- ms-swift(魔搭):阿里官方,Qwen 系列适配最深;
- xtuner / InternEvo:InternVL 官方;
- LLaVA / Qwen-VL 官方训练脚本:深度定制;
- DeepSpeed / FSDP:大规模分布式训练。

(2) 微调策略:

- LoRA(rank 16-64,作用在 $q/k/v/o + \text{ffn}$):显存少、训练快、效果不错,推荐起步;
- QLoRA(4-bit base + LoRA):极致显存优化,单卡可微调 72B 量级;
- 全参微调:数据量大、显存足、追求上限时用;

- Vision encoder:通常冻结;数据多 / 领域差异大可解冻部分层;
- Projector:通常一起训。

(3) 关键超参:

- LR:LoRA $1e-4$ - $2e-4$;全参 $1e-5$ - $2e-5$;
- Batch size:看显存,梯度累积凑大 effective batch(128-512);
- Epoch:1-3 epoch 常见;监控 eval loss 防过拟合;
- Warmup:3-10%;
- LR scheduler:cosine 衰减;
- Max length:根据 token 数和图像 token 数综合(视觉 token 较多,常 4K-8K);
- DeepSpeed ZeRO-3 / FSDP 分布式;
- BF16 / FP16 训练,梯度 checkpoint 省显存。

(4) 显存预估(7B 模型,单卡):

- LoRA + BF16:24-40 GB(单 A100 40G 可跑);
- QLoRA(NF4):16-24 GB(消费级 24G 卡可跑);
- 全参 BF16:80GB+(多卡分布式)。

阶段 3:训练与监控

(1) 监控指标:

- 训练 loss、eval loss(过拟合检测);
- GPU 利用率、显存峰值;
- 中间 checkpoint 做小评测集快速评估。

(2) 训练时长(参考):

- 7B + LoRA + 5 万样本:1-2 天($8 \times A100$);
- 7B + 全参 + 50 万样本:1 周($8 \times A100$)。

阶段 4:评估

(1) 业务评测集:

- 自建几百到几千条业务 ground-truth 样本;
- 任务多样:VQA、Caption、Grounding 等;
- 自动指标 + LLM-judge + 人工抽检。

(2) 公开 benchmark(防止业务过拟合 + 通用能力对照):

- MMBench、MME、MMMU、MathVista、DocVQA、ChartQA、OCRBench、HallusionBench;
- POPE 测幻觉;
- RefCOCO 测 grounding;
- 中文:CCBench、SEED-Bench-Chinese。

(3) 关键对比:

- 微调前 vs 微调后:业务任务提升多少?通用任务退化多少?
- 不同 LoRA rank、不同数据配比、不同模型大小的横评;
- 错误分析,bad case 沉淀。

阶段 5:部署与持续优化

(1) 量化部署:

- AWQ INT4 或 GPTQ;
- vLLM / lmdeploy / SGLang 推理;
- 端侧用 GGUF + llama.cpp。

(2) Bad Case 回流:

- 上线后用户反馈、点踩、转人工的样本收集;

- 定期(每周/每月)新增到训练数据;
- 持续迭代版本。

(3) A/B 测试:

- 新版 vs 旧版分流;
- 关键业务指标对比。

四、常见踩坑经验:

(1) 数据质量胜过一切:

- 数据噪声(标错、不一致、模糊)直接导致模型学坏;
- 一定要人工抽检,即使是合成数据;
- 不同标注员风格不一致会让模型困惑。

(2) 灾难性遗忘:

- 只用业务数据微调,通用能力会大跌;
- 必须混入通用 VLM 数据(20-40%);
- 太少业务数据时倾向 LoRA 而非全参。

(3) Vision encoder 冻不冻:

- 默认冻结安全;
- 数据极多、领域跨度大(医学影像)时可考虑解冻末端几层。

(4) 视觉 token 数与上下文:

- 高分辨率切块后 token 数可能 2000+ / 张图;
- 一个对话多图时容易爆 context;
- 控制 max_pixels 或采样数。

(5) Prompt 模板一致性:

- 训练用的 chat template 必须与推理时完全一致(BOS、特殊 token、image placeholder);
- 不一致会导致效果腰斩,新手常见坑。

(6) 评测严谨性:

- 不要只看 loss 曲线;
- 真实评测集 + 人工抽检;
- 不要在评测集上过拟合(eval set 别参与训练)。

(7) 学习率敏感:

- LoRA LR 不要太低($< 5e-5$ 学不动)也不能太高;
- 全参微调 LR 一定要小($2e-5$ 量级)。

(8) 多任务平衡:

- 任务数据比例失衡 → 模型偏科;
- 用上下采样平衡;
- 监控每类任务的评测指标分别。

五、回答模板(面试用):

"我做过的 X 业务场景的 VLM 微调,基于 Qwen2.5-VL-7B(或 InternVL / MiniCPM-V,根据实情)。",

"数据上,我们标注了 N 条业务数据 + 混入 X% 通用 VLM 数据防遗忘。数据形式是 LLaVA 风格的图文指令对话。",

"训练用 LLaMA-Factory + LoRA(rank 32),冻结视觉 encoder,训练 projector 和 LLM 的 q/k/v/o 投影。BF16 + DeepSpeed ZeRO-3,8 张 A100 训练 2 天。",

"评估上,业务自建评测集 + 公开 benchmark(MMBench、POPE 等)横评,微调后业务指标提升 X%,通用指标退化 Y%(可接受)。",

"踩坑主要是数据质量(GPT-4o 合成数据要人工抽检)、灾难性遗忘(必须混通用数据)、template 一致性(导致一次效果腰斩)。"

六、面试要点:

- 体现完整工程链路(数据 + 训练 + 评估 + 部署),而非只讲模型;
- 明确说出选型理由(为什么选 Qwen2.5-VL 而非 LLaVA?);
- 量化效果:不要光说"提升了",给具体百分比;
- 真实踩坑:数据问题 / 模板问题 / 遗忘 / 评测 / 显存,任选 2-3 个深聊;
- 体现持续迭代思维(bad case 回流、A/B);
- 如果实际没做过,坦诚说"我研究过这个流程但没实际跑过完整微调",再讲清楚理论 + 工具链。